

SCHOOL OF ELECTRICAL ENGINEERING
AND TELECOMMUNICATIONS
THE UNIVERSITY OF NEW SOUTH WALES

Comparison and Implementation of Data Compression Methods for SoC-based Radar Systems

by

Jordan Mohindra

Student ID: z5418455

Thesis submitted as a requirement for the degree
Bachelor of Engineering (Electrical Engineering)

Submitted: September 2026

Supervisor: A/Prof. Beena Ahmed

Abstract

High-resolution cascaded MIMO radar produces data faster than a vehicle’s network and central processor can comfortably absorb it. Compressing that data at the sensor, before it enters the network, is an attractive way to buy back bandwidth — but only if the compressor is fast, cheap in silicon, and lossless, because a radar that quietly discards weak targets to save bits is not a safe radar.

This thesis implements, verifies and critically evaluates two lossless compressors for radar range-FFT data on an AMD Kintex UltraScale+ FPGA: *Doppler Redundancy Hybrid Encoding* (DRHE), a streaming polar-domain predictor, and a second-order *Linear Predictive Coder* (LPC) with Yule-Walker coefficients. Both share a category-plus-append entropy back end with a fixed Huffman dictionary. Both were developed in MATLAB against a double-precision reference, translated to C++ for Vitis HLS, verified bit-exactly over 50 frames of the real ColoRadar cascade dataset, and characterised through RTL co-simulation and Vivado placement and routing.

The headline results are not the ones that were expected. A systematic audit of the compression ratio found that *neither predictor was contributing anything*: coding the quantised FFT output through the identical entropy coder with no prediction whatsoever achieves a compression ratio of 3.489, against 3.319 for DRHE and 3.397 for LPC. Both predictors were making the data measurably harder to compress. The cause was traced to an assumption inherited from the single-transmitter literature. In a twelve-transmitter time-division MIMO cascade the ramp index interleaves transmitters, so “the previous ramp” belongs to a different physical antenna eleven times out of twelve, separated in space by the array baseline rather than in time by a pulse interval. Predicting instead from the previous ramp of the *same* transmitter — lag n_{Tx} rather than lag 1 — converts a 4.9% loss into an 8.2% gain. Measured in hardware simulation over the full dataset, compression rises from 3.310 to 3.751 for DRHE and from 3.385 to 3.697 for LPC, in both cases with every sample still reconstructed exactly. For DRHE the correction costs 40 additional block RAMs — 4% of the device — with logic and measured frame latency unchanged and the clock still well above 100 MHz.

Two implementation defects were also found and fixed, neither visible on real data and both caught by synthetic edge cases: a residual value of -32768 that the entropy code cannot represent and which aliased silently onto $+32767$, and a 128-bit to single-precision conversion in the Yule-Walker solver that returned zero for operands above 2^{64} . The reference design’s published throughput was also found to be a theoretical figure in gibibits rather than a measured one in gigabits; compared like for like, the gap to it is exactly the factor of two that one extra memory access per cycle costs.

Implemented through placement and routing, the designs overturned their own synthesis estimates: DRHE uses about a third of the logic HLS predicted, fewer LUTs than the published reference design, and every design meets 100 MHz where the estimates said none of them would. To account for every remaining difference from the reference, it was rebuilt from its description in the same tool flow and connected to our design by a ladder of single-change variants, each implemented and verified lossless. Each difference in block RAM, DSPs, pipeline depth and initiation interval has one measured cause, and most of the logic is the floating-point datapath: changing only the arithmetic to the reference’s fixed point removes 42% of the

logic and 67 % of the DSPs and compresses to within 0.07 % of the floating-point design. The reference’s own published logic count remains larger than its description requires, and only part of that difference could be explained.

The thesis closes with a decomposition of where the compression ratio genuinely comes from — $1.63\times$ from an under-filled fixed-point container, $2.14\times$ from spectral sparsity, and only then a modest contribution from prediction — and argues that reporting a no-prediction baseline alongside any such ratio should be standard practice.

Acknowledgements

I would like to thank A/Prof. Beena Ahmed for her guidance throughout this project, and Dr. Mostafa Shahin for his assessment and feedback. I am also indebted to Lukas Kiem, whose master's thesis provided the algorithmic baseline this work builds on and corrects; the fact that this thesis identifies a limitation in applying that work to MIMO radar should be read as a measure of how carefully specified it was, since a vaguer description could not have been tested against measurement at all.

Abbreviations

ADAS	Advanced Driver Assistance Systems
ADC	Analog-to-Digital Converter
AR	Autoregressive
ASIC	Application-Specific Integrated Circuit
AXI	Advanced eXtensible Interface
BAQ	Block Adaptive Quantization
BFP	Block Floating Point
BRAM	Block Random Access Memory
CFAR	Constant False Alarm Rate
CR	Compression Ratio
CSIM	C-Simulation
DDR	Double Data Rate (memory)
DPCM	Differential Pulse-Code Modulation
DRHE	Doppler Redundancy Hybrid Encoding
DSP	Digital Signal Processing / DSP slice
ECU	Electronic Control Unit
EGE	Exponential-Golomb Encoding
EVM	Evaluation Module
FDR	False Discovery Rate
FF	Flip-Flop
FFT	Fast Fourier Transform
FMCW	Frequency-Modulated Continuous Wave
FN	False Negative
FNR	False Negative Rate
FP	False Positive
FPGA	Field-Programmable Gate Array
FPU	Floating-Point Unit
FX16	16-bit Fixed-point representation
HDL	Hardware Description Language
HLS	High-Level Synthesis
IF	Intermediate Frequency
IIR	Infinite Impulse Response
II	Initiation Interval
I/Q	In-phase / Quadrature
LPC	Linear Predictive Coding
LSB	Least Significant Bit
LUT	Look-Up Table
MIMO	Multiple-Input Multiple-Output
MSB	Most Significant Bit
PRI	Pulse Repetition Interval
RTL	Register-Transfer Level

RX	Receive / Receiver
SAR	Synthetic Aperture Radar
SerDes	Serialiser/Deserialiser
SNR	Signal-to-Noise Ratio
SoC	System-on-Chip
TDM	Time-Division Multiplexing
TX	Transmit / Transmitter
URAM	UltraRAM

Contents

Abstract	i
Acknowledgements	iii
Abbreviations	iv
Contents	vi
List of Figures	xi
List of Tables	xii
1 Introduction	1
1.1 The problem in one paragraph	1
1.2 Motivation	1
1.3 Why the compressor must be lossless	2
1.4 Objectives	2
1.5 Contributions	3
1.6 How to read this report	3
2 Background	5
2.1 How an FMCW radar measures range	5
2.1.1 The chirp	5
2.1.2 From beat frequency to a range profile	5
2.1.3 Slow time and Doppler	6
2.2 Angle, MIMO and the virtual array	7
2.2.1 Why more antennas	7
2.2.2 MIMO: buying antennas cheaply	7
2.2.3 Time-division multiplexing, and why it matters here	7
2.3 The radar data cube, and why it is large	8
2.4 Lossless compression: how it actually works	8
2.4.1 The two categories	8
2.4.2 The bound: entropy	9
2.4.3 The standard two-stage structure	9
2.4.4 Open-loop versus closed-loop prediction	9
2.5 The sensor and the dataset	10
2.5.1 AWR2243 and the MMWCAS-RF-EVM	10
2.5.2 ColoRadar	10
2.6 Number representation and quantisation	11
2.6.1 Fixed point versus floating point	11
2.6.2 Quantisation error, and the FX16 step	11
2.7 FPGAs and high-level synthesis	11

2.7.1	What an FPGA is	11
2.7.2	The four currencies: LUT, FF, BRAM, DSP	12
2.7.3	Pipelining, initiation interval and $F_{\max}$	12
2.7.4	High-level synthesis	13
2.7.5	AXI4-Stream	13
2.8	Related work and the baseline	13
2.8.1	Deterministic industrial schemes	13
2.8.2	Block Adaptive Quantization	13
2.8.3	Neural and spectral approaches	13
2.8.4	The baseline: Kiem's thesis	13
2.9	Summary of what the later chapters assume	14
3	The Two Algorithms	15
3.1	The shared entropy coder: turning residuals into bits	15
3.1.1	What a residual is	15
3.1.2	Why a re-expression can be worth anything	16
3.1.3	The parsing problem	16
3.1.4	The category S_4	16
3.1.5	The APPEND bits	17
3.1.6	Cost, and the shape of the dictionary	18
3.1.7	Worked example 1: one value, in full	18
3.1.8	Worked example 2: sixteen real samples, both ways	19
3.1.9	Decoding the same stream	21
3.1.10	Where the bits actually go	22
3.1.11	How good is the coder itself?	23
3.1.12	A representability hole	23
3.2	DRHE: Doppler Redundancy Hybrid Encoding	24
3.2.1	Why predict in polar coordinates	24
3.2.2	The two predictors	24
3.2.3	The streaming property	24
3.2.4	The assumption	25
3.3	LPC: linear prediction with Yule–Walker coefficients	25
3.3.1	Autoregressive modelling	25
3.3.2	Stability	25
3.3.3	Granularity and side information	26
3.3.4	The cost: LPC is not causal	26
3.4	Side-by-side	26
4	Methodology	27
4.1	The verification ladder	27
4.2	The reference signal chain	28
4.2.1	Loading	28
4.2.2	Windowing	28
4.2.3	Range FFT	28
4.2.4	FX16 quantisation	28
4.2.5	Detection and metrics	29
4.3	Metrics, defined precisely	29
4.3.1	Compression ratio	29
4.3.2	Maximum absolute difference	29
4.3.3	Detection metrics, and one correction	29
4.3.4	What the detection metrics actually measure here	30
4.3.5	Order-0 entropy	30

4.4	Edge-case testing	30
4.5	Toolchain and reproducibility	30
5	Implementation	32
5.1	Common infrastructure	32
5.2	DRHE architecture	32
5.2.1	State and parallelism	32
5.2.2	Why it is expensive	32
5.2.3	The initiation-interval bottleneck	33
5.3	LPC architecture	33
5.3.1	Three passes	33
5.3.2	A useful asymmetry	33
5.3.3	Fixed-point choices	34
5.4	Two defects found and fixed	34
5.4.1	Defect 1: the -32768 residual	34
5.4.2	Defect 2: 128-bit to float conversion	35
5.5	The TDM-aware variants	35
5.5.1	DRHE	36
5.5.2	LPC, and a ring buffer trick	36
5.5.3	A restructure that pays for itself	36
5.5.4	Both baselines retained	36
6	Results	37
6.1	Phase 1: the MATLAB benchmark	37
6.1.1	Eight algorithms on real data	37
6.2	Phase 2: bit-exactness	37
6.2.1	The headline	37
6.2.2	How closely the hardware matches MATLAB	38
6.2.3	Edge cases	38
6.3	Phase 3: synthesis and co-simulation	39
6.3.1	Resources	39
6.3.2	Timing: the estimate said no, the implementation said yes	39
6.3.3	The expectation that did not survive, twice	39
6.3.4	Where LPC's cycles go	40
6.4	The audit: what the compression ratio actually measures	40
6.4.1	The question	40
6.4.2	The result	40
6.4.3	Where the ratio really comes from	41
6.5	The prediction axis	42
6.5.1	Diagnosis	42
6.5.2	Testing the hypothesis	42
6.5.3	The variant that does not work, and why	43
6.5.4	What the residuals look like	43
6.6	The corrected implementations	43
6.6.1	Compression, measured in hardware	44
6.6.2	What the correction costs	45
6.6.3	A caution about cycle models	46
6.6.4	Co-simulation	47
6.7	DRHE against LPC, after the correction	47
6.8	Summary of results	47

7	Accounting for the Hardware Numbers	49
7.1	Two ways of counting the same circuit	49
7.1.1	Running the implementation	49
7.1.2	What the implementation found	50
7.1.3	Why the logic estimates were so far off	50
7.2	Where the numbers come from, and how far to trust them	52
7.3	The yardsticks	52
7.4	Rebuilding the reference design	53
7.4.1	Why a replica is needed	53
7.4.2	One source, four switches	54
7.4.3	How close is the replica to Kiem's published design?	54
7.5	From our design to Kiem's, one change at a time	55
7.5.1	Block RAM: the width of a word	56
7.5.2	DSP slices: counted operator by operator	56
7.5.3	Pipeline depth: which operations wait for which	57
7.5.4	Initiation interval: one write too many	58
7.5.5	LUTs and flip-flops: two gaps, not one	58
7.5.6	Clock frequency and speed grade	62
7.6	Latency: three different things with the same name	63
7.6.1	Pipeline latency: how long one sample takes to go through	63
7.6.2	Frame latency: how long a whole frame takes, if it all arrives at once	64
7.6.3	Added latency: how much later the answer arrives because of compression	64
7.6.4	When does latency become a problem?	64
7.7	Throughput and initiation interval	65
7.8	Resources against the device	67
7.9	Power	68
7.10	Compression ratio: what counts as good	69
7.11	Every difference from Kiem, accounted for	70
8	Discussion	72
8.1	Comparison with the baseline design	72
8.1.1	Three corrections to the comparison	72
8.1.2	What the comparison finally says	72
8.2	What the compression ratio means elsewhere	73
8.3	Does the correction generalise?	73
8.4	Threats to validity	74
8.5	Lessons about verification	75
9	Conclusions and Future Work	76
9.1	What was done	76
9.2	What was found	76
9.2.1	The prediction axis	76
9.2.2	What a compression ratio actually measures	77
9.2.3	What the hardware numbers mean	77
9.2.4	Two defects, and two corrections to the record	77
9.2.5	Which algorithm	77
9.3	Future work	78
9.3.1	Immediate	78
9.3.2	Longer term	78
9.4	Closing remark	79
	Bibliography	80

Bibliography	80
A The Huffman Dictionary	82
B Bitstream Layouts	83
B.1 DRHE	83
B.2 LPC	83
C Reproducing the Results	84
C.1 Source files	84
C.2 Build commands	85
C.3 Co-simulation stimulus	85

List of Figures

1.1	Raw versus compressed radar transport	2
2.1	FMCW chirp and beat frequency	6
2.2	TDM-MIMO ramp interleaving	8
2.3	Radar data cube and processing chain	8
2.4	Two-stage lossless compressor	9
3.1	The sixteen samples, plotted	20
3.2	Bit budget by category	23
3.3	DRHE block diagram	25
4.1	The verification ladder	28
5.1	LPC three-pass architecture	33
6.1	Decomposition of the compression ratio	41
6.2	S4 category distributions	43
6.3	Residual magnitude	44
6.4	Bits per value across configurations	44
6.5	Per-frame compression ratio	45
6.6	Resource usage across the four designs	46
7.1	HLS estimate against implemented LUTs, per module	51
7.2	Resources at each step from DRHE-1 to the replica	55
7.3	Pipeline schedule of one sample	57
7.4	Latencies on one time axis	63
7.5	Throughput capacity against requirement	67

List of Tables

2.1	ColoRadar cascade capture parameters used throughout this thesis.	10
3.1	Raw values against residuals	16
3.2	S4 category ranges	17
3.3	Category 3 enumerated	17
3.4	Sixteen samples traced through both predictors	19
3.5	Why the two lags differ	21
3.6	Bit budget by category	22
3.7	DRHE and LPC compared	26
4.1	Synthetic edge cases	30
6.1	Phase 1 benchmark	38
6.2	LPC MATLAB-versus-hardware agreement	38
6.3	Phase 3 resources: estimate and implementation	39
6.4	Compression ratio decomposition	41
6.5	Prediction axis study	42
6.6	TDM-aware compression results	45
6.7	Cost of the TDM correction	46
6.8	Final head-to-head	47
7.1	HLS estimates against implemented results	50
7.2	Where each hardware number comes from	52
7.3	Sensor and system yardsticks	53
7.4	The replica against Kiem’s published design	54
7.5	From DRHE-1 to the replica, one change per step	55
7.6	How the predictor state is stored	56
7.7	Where DRHE-1’s DSP slices go	56
7.8	Where DRHE-1’s LUTs and flip-flops go	59
7.9	Testing explanations for the gap to Kiem’s LUT and FF counts	61
7.10	Throughput: measured, theoretical and required	66
7.11	Implemented resources as a fraction of two devices	67
7.12	Power and energy of the implemented designs	68
7.13	Every difference from Kiem’s design: cause and evidence	71
A.1	The fixed Huffman dictionary	82

Chapter 1

Introduction

1.1 The problem in one paragraph

A modern high-resolution automotive radar generates data faster than the vehicle it is bolted to can conveniently move it. The sensor at the centre of this thesis produces 3 MiB of raw digitised samples every frame, and it is one of several such sensors on a car. Getting that data to the central processor means either expensive shielded high-speed cabling, or compressing it at the sensor first. Compression is the cheaper answer, but it has to be done in a few hundred microseconds, in a small amount of silicon, and — because this data feeds a system that decides whether to brake — without losing anything. This thesis builds two such compressors, puts them on an FPGA, measures them properly, and finds that the accepted way of applying them to a MIMO radar is wrong.

1.2 Motivation

Radar earns its place in a vehicle’s sensor suite because it works when the other sensors do not. Cameras fail in fog, glare and darkness; lidar is degraded by rain and airborne dust; radar sees through all of them [1, 2]. That robustness is why radar has been in production vehicles for two decades, initially for simple distance warning and now for adaptive cruise control, automatic emergency braking, blind-spot detection and lane-change assistance [3].

What has changed recently is resolution. Supporting SAE Level 2+ autonomy needs more than “something is 40 m ahead”. It needs a map: how many objects, where, how fast, how wide, and what they probably are. Angular resolution is what separates “an object” from “a car and a pedestrian standing beside it”, and angular resolution comes from having many antennas. A radar with N_t transmitters and N_r receivers can synthesise an effective array of $N_t \times N_r$ elements, so resolution improves roughly with the product. The data volume, unfortunately, scales the same way.

At the same time, vehicle electrical architectures have been consolidating. Where a car once had a hundred small electronic control units each doing one job, the industry has moved to domain controllers, then to *zonal* architectures where sensors report to a nearby gateway, and toward fully centralised designs where one high-performance system-on-chip does everything [4, 5, 6]. Centralising the computation has obvious advantages, but it means the raw sensor data has to travel.

Figure 1.1 shows the consequence. A Serialiser/Deserialiser (SerDes) link [7, 8] carries the aggregated output of several radar modules at 2.5 Gbit/s or more. Those data rates are achievable, but they require shielded cable, they cost money and weight, and they put a matching ingest burden on the SoC’s memory system [9]. If instead the data is compressed at the radar module by a factor of 2.5 or better, the same information fits inside Gigabit

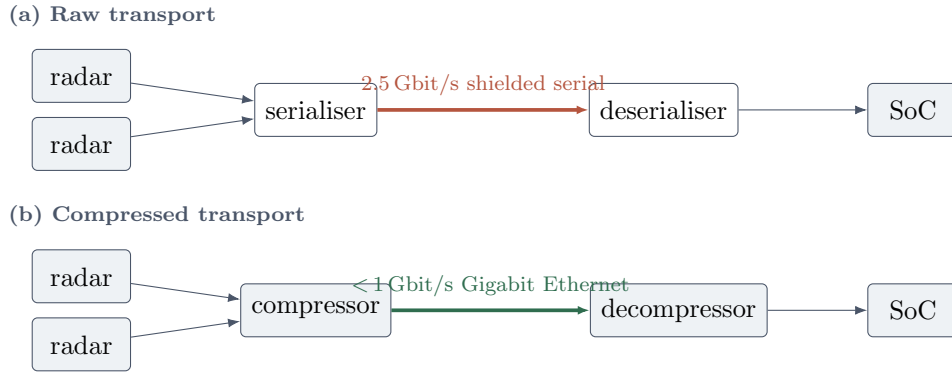


Figure 1.1: The economic argument for sensor-edge compression. Compressing by a factor of 2.5 or better at the radar module replaces an expensive shielded high-speed serial link with standard Gigabit Ethernet, without changing what the central processor receives.

Ethernet [10] — a cheap, standard, unshielded interconnect. That single substitution is the economic argument for this entire body of work.

1.3 Why the compressor must be lossless

It is tempting to relax the requirement. Lossy compression reaches much higher ratios, and radar data is noisy anyway, so surely a little extra noise is harmless?

It is not, and the reason is specific rather than philosophical. The compressor sits *upstream* of detection. Whatever it discards is discarded before the detector ever sees it, and the detector’s job is to find weak returns close to the noise floor — a pedestrian at range, a motorcycle between two trucks. Lossy compression raises the noise floor precisely where the margin is thinnest. Chapter 6 quantifies this: Block Adaptive Quantization, a respected lossy scheme, reaches a compression ratio comparable to the lossless coders studied here but misses nearly three times as many real detections and invents nearly nine times as many false ones.

Losslessness also has a practical virtue that matters enormously for a project of this kind: it makes verification unambiguous. The pass criterion is that every reconstructed sample equals its original exactly. There is no tolerance to argue about, no distortion metric to trade off. A single differing bit anywhere in 157 million input bits is a defect, and the test reports it as one. Section 5.4 describes two defects that were found exactly this way and would have been invisible under any tolerance-based criterion.

1.4 Objectives

The starting point for this thesis is the master’s work of Lukas Kiem [11], which evaluated a wide range of radar compression schemes and identified Doppler Redundancy Hybrid Encoding as the best of them, reaching a compression ratio of 3.2 at a latency of 0.08 ms on a Zynq UltraScale+ device. Kiem’s work left three openings, which became the objectives of this thesis:

1. **Validate on real data.** Kiem’s FPGA implementation was exercised with synthetic stimulus. This thesis drives both compressors with real captured data — 50 consecutive frames from a Texas Instruments MMWCAS-RF-EVM cascade, taken from the public ColoRadar dataset [12] — through the entire verification chain, from MATLAB to RTL co-simulation.
2. **Broaden the algorithm comparison.** DRHE is compared here not only against the schemes Kiem considered but against a second-order linear predictive coder with

Yule–Walker coefficients, and against a *no-prediction* baseline that turns out to be the most informative comparison of all.

3. **Implement the complete pipeline.** Kiem implemented compression only. This thesis implements compression *and* decompression for both algorithms, which is what makes bit-exact verification possible at every level, and which exposed a subtle encoder/decoder divergence described in Section 5.4.1.

1.5 Contributions

The substantive contributions of this thesis are:

1. **A working, bit-exact, fully verified FPGA implementation of two lossless radar compressors**, each with a matching decompressor, verified over 50 frames of real cascade radar data at three levels of abstraction (MATLAB reference, HLS C-simulation, RTL co-simulation) with zero reconstruction error at every level.
2. **The identification and correction of a prediction-axis error** in applying slow-time predictive compression to time-division MIMO radar. Both algorithms, applied the way the literature describes, performed *worse than no prediction at all*. Correcting the prediction lag to the transmitter period recovers the intended gain, lifting measured compression from 3.310 to 3.751 for DRHE and 3.385 to 3.697 for LPC. This is, to the author’s knowledge, not reported elsewhere.
3. **A decomposition of the compression ratio** into its three genuine sources — unused fixed-point container headroom, spectral sparsity, and prediction — which shows that the headline ratios reported in this field are substantially a property of sensor gain staging rather than of the algorithm.
4. **Two implementation defects found and fixed**, both invisible on real data and both caught by synthetic edge-case testing: an unrepresentable residual value that aliased onto a different value, and a wide-integer to floating-point conversion that silently returned zero.
5. **Three corrections to the surrounding literature and to the author’s own earlier reporting**: a throughput figure quoted in gibibits and compared against decimal figures; a detection metric labelled as a false positive rate that is in fact a false discovery rate; and two malformed source citations.

1.6 How to read this report

The report is written to be self-contained for a reader who knows some signal processing and digital design but nothing in particular about radar or about compression.

Chapter 2 builds up the necessary background from first principles: how an FMCW radar turns a frequency sweep into a range measurement, what “slow time” and Doppler mean, how MIMO and time-division multiplexing produce a virtual array, what is actually in the data that has to be compressed, how lossless compression works in general, and what an FPGA gives you and charges you for. A reader already fluent in these can skip to Section 2.8.

Chapter 3 specifies the two algorithms and the entropy coder they share, in enough detail to reimplement them.

Chapter 4 sets out the four-phase verification framework and defines every metric used, including several that are commonly used loosely.

Chapter 5 describes the hardware architectures and the two defects.

Chapter 6 reports results, and is where the audit and the prediction-axis correction appear.

Chapter 7 explains what each hardware number means: what it represents, why it is measured, what it should be compared against — the reference design where that is valid, the sensor, the link and the device where it is not — and at what point it becomes a problem.

Chapter 8 discusses what the numbers support and what they do not, and **Chapter 9** concludes.

Chapter 2

Background

This chapter assumes no prior knowledge of radar. It builds up, in order, what the radar measures, how it measures it, what the resulting data looks like, why there is so much of it, how lossless compression works in general, and what an FPGA can do about it. Each section ends with the specific fact that the later chapters depend on.

2.1 How an FMCW radar measures range

2.1.1 The chirp

Automotive radar is almost universally *frequency-modulated continuous wave* (FMCW). Instead of emitting a short pulse and timing the echo — which would demand impractically fast electronics for the distances involved — an FMCW radar transmits continuously, sweeping its frequency linearly upward over a short interval. One such sweep is called a *chirp*. The sensor used here sweeps 1.798 GHz of bandwidth around a 77 GHz centre frequency in 78.5 μ s.

When the chirp reflects off an object at range R , the echo arrives delayed by the round-trip time

$$\tau = \frac{2R}{c}, \quad (2.1)$$

where c is the speed of light. Because the transmitter has kept sweeping in the meantime, the echo's frequency at any instant is lower than what is currently being transmitted, by an amount proportional to the delay. If the chirp sweeps B hertz in T_c seconds, the sweep slope is $S = B/T_c$ and the frequency difference is

$$f_b = S\tau = \frac{2SR}{c}. \quad (2.2)$$

A mixer multiplies the transmitted and received signals, producing a low-frequency *beat* or intermediate-frequency signal at exactly f_b . That signal is low-pass filtered and digitised by an analog-to-digital converter (ADC) at a modest rate — 8 MHz here, rather than the tens of gigahertz the raw RF would need. This is the central trick of FMCW: it converts a timing measurement that would need picosecond resolution into a frequency measurement that an ordinary ADC can make.

2.1.2 From beat frequency to a range profile

Measuring f_b is a spectral estimation problem, so the natural tool is the Fourier transform. Taking an N_s -point FFT across the N_s ADC samples of a single chirp produces a *range profile*: bin n of the FFT corresponds, through Equation (2.2), to range

$$R_n = n \cdot \frac{c}{2B}, \quad (2.3)$$

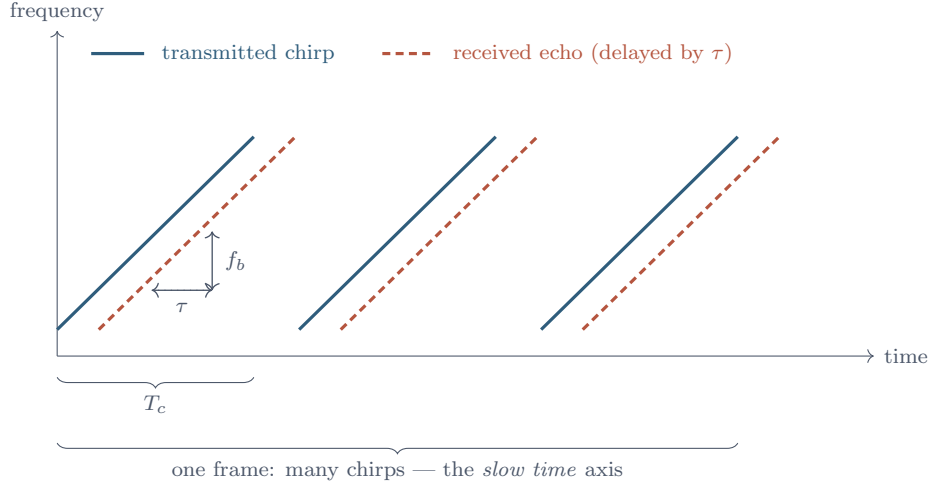


Figure 2.1: FMCW operation. Each chirp sweeps linearly in frequency. The echo is a delayed copy, so at any instant the transmitted and received frequencies differ by $f_b = S\tau$, proportional to target range. A frame consists of many such chirps in succession; the axis running across chirps is called *slow time* and is where Doppler information lives.

so the range resolution is

$$\Delta R = \frac{c}{2B} \approx 8.3 \text{ cm} \quad (2.4)$$

for the 1.798 GHz bandwidth used here. A peak in the range profile is an object. This first transform is called the **Range FFT**, and its output is the data this thesis compresses.

2.1.3 Slow time and Doppler

A single chirp gives range but not velocity. To get velocity, the radar transmits many chirps in rapid succession — a *frame* — and looks at how the *phase* of a given range bin evolves from chirp to chirp.

The reasoning is worth spelling out, because the whole of DRHE rests on it. A target at range R produces a range-FFT value whose phase is $\theta = -4\pi R/\lambda$, where λ is the wavelength. Over the chirp-to-chirp interval T , a target moving at radial velocity v changes range by vT , so its phase changes by

$$\Delta\theta = -\frac{4\pi vT}{\lambda}. \quad (2.5)$$

At 77 GHz, $\lambda \approx 3.9$ mm, so even a very slow target produces a measurable phase step — which is exactly why radar is good at velocity. Critically for compression, $\Delta\theta$ is *constant* for a target at constant velocity. The phase of a given range bin therefore advances linearly across chirps, and its magnitude barely changes at all. That is a highly predictable sequence, and predictable sequences compress.

The axis running across chirps within a frame is called **slow time**, as opposed to the *fast time* axis of samples within a chirp. Taking a second FFT along slow time resolves $\Delta\theta$ into velocity: this is the **Doppler FFT**.

Carry forward: slow time is where the redundancy is. A target at constant velocity produces a range-bin sequence whose magnitude is nearly constant and whose phase advances by a constant increment. Both DRHE and LPC exist to exploit exactly this.

2.2 Angle, MIMO and the virtual array

2.2.1 Why more antennas

Range and velocity are not enough to drive a car. The radar also has to know *where* a target is in angle, and how well it can separate two targets at the same range and velocity but different bearings.

Angle is measured by comparing the phase of the same echo at physically separated receive antennas. A wavefront arriving at angle ϕ reaches adjacent antennas, spaced d apart, with a path difference $d \sin \phi$, hence a phase difference

$$\Delta\psi = \frac{2\pi d \sin \phi}{\lambda}. \quad (2.6)$$

An FFT across the receive-antenna axis — the **Angle FFT** — turns those phase differences into a bearing estimate. The angular resolution improves with the physical size of the array, which in practice means with the number of elements.

2.2.2 MIMO: buying antennas cheaply

Building a large physical receive array is expensive. Multiple-Input Multiple-Output (MIMO) radar gets the same effect more cheaply. If N_t transmitters and N_r receivers are arranged suitably, and the signal from each transmitter can be separated at each receiver, then each of the $N_t \times N_r$ transmit–receive pairs behaves like an element of a *virtual array*. The sensor used here has $N_t = 12$ and $N_r = 16$, giving 192 virtual elements from 28 physical antennas.

2.2.3 Time-division multiplexing, and why it matters here

The separability requirement has to be met somehow. The simplest method, and the one this sensor uses, is *time-division multiplexing* (TDM): the transmitters fire one at a time, in a fixed rotation, so whatever a receiver hears during transmitter t 's slot must have come from transmitter t .

TDM is simple and robust, and it has one consequence that this thesis turns out to hinge on. Inside every repetition of the frame — every “chirp loop” — all N_t transmitters fire in sequence before the loop repeats. When the resulting data is laid out as a two-dimensional array of (ramp, range bin), which is how a streaming compressor sees it, the ramp index is

$$m = c N_t + t, \quad c \in [0, N_c), \quad t \in [0, N_t), \quad (2.7)$$

where c indexes the chirp loop and t the transmitter. The transmitter index varies *fastest*.

This is the correct physical time order. It is also, as Figure 2.2 makes visual, a trap. Ramp m and ramp $m - 1$ are different physical transmitters $N_t - 1$ times out of N_t . They are separated in *space* by the array baseline, not in *time* by a pulse interval, and the phase relationship between them is governed by Equation (2.6) — the target's bearing — not by Equation (2.5) — the target's velocity. The genuine slow-time neighbour of ramp m , the one whose phase differs by a Doppler step, is ramp $m - N_t$.

Carry forward: on a TDM-MIMO radar the ramp axis is transmitter-interleaved. Any algorithm that assumes consecutive ramps are consecutive *in time on the same antenna* — which includes DRHE as published, and a plain lag-1/lag-2 linear predictor — is operating on the wrong axis. This is the central finding of Chapter 6.

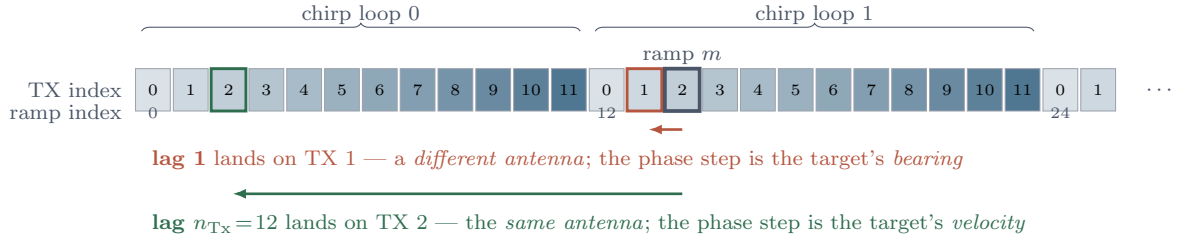


Figure 2.2: Why “the previous ramp” is the wrong neighbour on a TDM-MIMO cascade. Each cell is one ramp, labelled with the transmitter that produced it; the twelve transmitters rotate inside every chirp loop. Predicting ramp m from ramp $m - 1$ (red) crosses a transmitter boundary eleven times out of twelve, so the two ramps differ by the target’s array steering phase. Predicting from ramp $m - n_{\text{Tx}}$ (green) stays on the same physical antenna and therefore on the genuine Doppler axis.

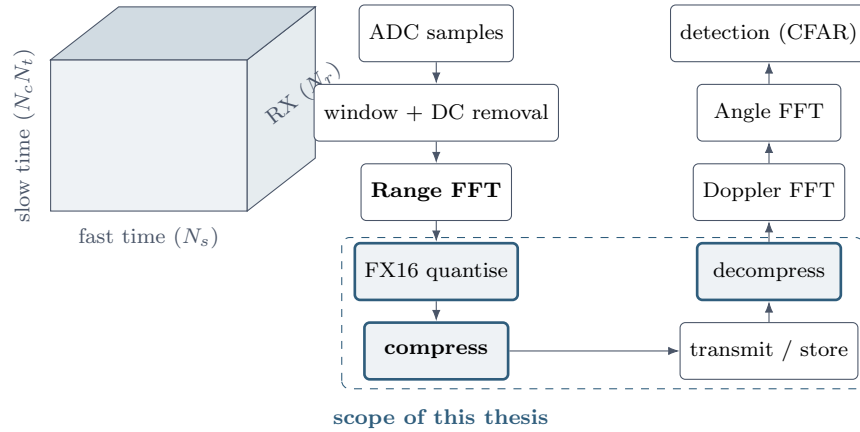


Figure 2.3: The radar data cube and the processing chain. The compressor sits immediately after range-FFT and fixed-point quantisation, before the data leaves the sensor. Everything downstream — Doppler FFT, angle FFT, detection — sees the reconstructed data, which for a lossless compressor is bit-identical to the original.

2.3 The radar data cube, and why it is large

Putting the three axes together gives the *radar data cube*: fast time (samples within a chirp) $\times$ slow time (ramps) $\times$ receive channel. Three FFTs are applied along the three axes in turn, yielding range, velocity and angle. Figure 2.3 shows the arrangement and where in the chain this thesis intervenes.

The volume follows directly. For the sensor used here, one frame is

$$N_s \times N_c \times N_r \times N_t = 256 \times 16 \times 16 \times 12 = 786,432 \quad (2.8)$$

complex samples. Each is two 16-bit integers, so a frame is 3,145,728 B — exactly 3 MiB. At a typical frame rate this is gigabits per second from a single sensor, and a vehicle carries several.

2.4 Lossless compression: how it actually works

2.4.1 The two categories

Compression divides into *lossless*, where the decompressed data is bit-identical to the input, and *lossy*, where some information is permanently discarded in exchange for a smaller payload [14]. Lossy compression can reach far higher ratios; lossless compression is bounded by the information content of the data itself. Section 1.3 explained why this application needs the lossless kind.

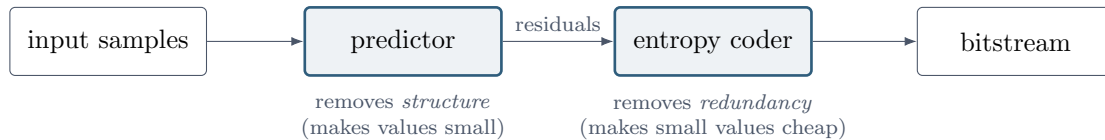


Figure 2.4: The standard structure of a lossless compressor. The two stages do different jobs and can fail independently — which is precisely what Chapter 6 finds had happened.

2.4.2 The bound: entropy

There is a hard limit on lossless compression, and knowing it is what makes the results in Chapter 6 interpretable. If a source emits symbols x with probabilities $p(x)$, its *order-0 entropy* is

$$H = - \sum_x p(x) \log_2 p(x) \quad \text{bits per symbol}, \quad (2.9)$$

and no code that treats symbols independently can average fewer bits than H [14]. This gives a concrete yardstick: measure H for the data, compare it against what the coder actually achieves, and the gap is the coder’s inefficiency. Section 6.4 does exactly this, and finds the entropy coder used here runs only 6.2% above the order-0 entropy of its input — which is good, and which means the room for improvement is not in the entropy coder.

2.4.3 The standard two-stage structure

Almost every practical lossless compressor has the same shape (Figure 2.4): a *decorrelation* stage followed by an *entropy coding* stage.

The **decorrelation** stage exploits structure. If consecutive samples are similar, predict each one from its neighbours and transmit only the *residual*, the difference between prediction and truth. When the prediction is good, residuals cluster tightly around zero even though the original samples spanned the full dynamic range. Nothing is lost: given the residual and the same prediction rule, the decoder reconstructs the original exactly.

The **entropy coding** stage exploits the resulting skew in the distribution. Values that occur often get short codewords; values that occur rarely get long ones. Huffman coding [15] is the classic construction and produces the optimal prefix-free code for a known symbol distribution.

Crucially, the two stages are independent. A predictor that fails — that produces residuals no more skewed than its input — does not break the coder; the coder simply carries on compressing whatever it is handed, and the compression ratio can still look respectable. Detecting this requires deliberately measuring what the coder achieves *without* the predictor, which is not a standard practice and which is how the central finding of this thesis came to light.

2.4.4 Open-loop versus closed-loop prediction

One subtlety in predictive coding matters enough to state now, because it caused a defect described in Section 5.4.1.

The encoder predicts from past samples. So does the decoder — but the decoder does not have the past *original* samples, only the past *reconstructed* ones. In *open-loop* prediction the encoder uses the originals; in *closed-loop* prediction (classically, differential pulse-code modulation, DPCM) the encoder reconstructs each sample exactly as the decoder will, and predicts from that.

When compression is perfectly lossless the two are identical, because reconstruction equals the original. The moment anything — a saturation, a rounding, a clamp — makes

Table 2.1: ColoRadar cascade capture parameters used throughout this thesis.

Parameter	Symbol	Value
ADC samples per chirp	N_s	256
Chirp loops per frame	N_c	16
Receive channels	N_r	16
Transmit channels (TDM)	N_t	12
Ramps per frame	$N_c N_t$	192
Fast-time sample rate	f_s	8 MHz
Chirp bandwidth	B	1.798 GHz
Chirp-to-chirp interval	T_c	78.5 μ s
Centre frequency	f_c	77 GHz
Range resolution	ΔR	8.34 cm
Frame size (raw)	—	3,145,728 B

reconstruction differ from the original by even one bit, an open-loop encoder and its decoder begin predicting from different histories and *diverge*. With an IIR predictor, as in DRHE, that divergence is then carried forward through every subsequent ramp. Closing the loop confines the error to the affected sample.

Carry forward: a compression ratio is the product of two independent effects, and a healthy-looking ratio does not imply a working predictor. Always measure the entropy coder alone as a baseline.

2.5 The sensor and the dataset

2.5.1 AWR2243 and the MMWCAS-RF-EVM

The radar front end assumed throughout is the Texas Instruments AWR2243, a 76–81 GHz FMCW transceiver with three transmit and four receive channels [13]. Importantly for this work, the AWR2243 performs no signal processing of its own: it transmits, receives, mixes and digitises, then streams raw ADC samples out over a MIPI CSI-2 interface. It therefore exposes exactly the payload that creates the transport bottleneck.

Four AWR2243 devices are cascaded on the MMWCAS-RF-EVM evaluation module, giving 12 transmit and 16 receive channels on one synchronised board — the 192-element virtual array of Section 2.2.

2.5.2 ColoRadar

Rather than synthetic stimulus, this thesis uses real captures from the public ColoRadar dataset [12], which includes cascade recordings from exactly this module across indoor, outdoor and subterranean environments. Table 2.1 lists the relevant parameters. Fifty consecutive frames were used throughout; that is 39,321,600 complex ADC samples, or 150 MiB of raw data, which is enough to make a claim of losslessness meaningful.

Each complex sample is stored as two contiguous `int16` values. The index of the sample at (fast-time s , chirp loop c , receiver r , transmitter t) in the file is

$$i = 2(s + N_s(c + N_c(r + tN_r))), \quad (2.10)$$

so the layout from slowest to fastest varying is $[t, r, c, s]$. Reshaping this into the (fast time, ramp, receiver) cube the processing chain expects is what produces the transmitter-interleaved ramp axis of Equation (2.7).

2.6 Number representation and quantisation

2.6.1 Fixed point versus floating point

Hardware has to represent numbers somehow, and the choice is consequential.

Fixed-point representation allocates a constant number of bits to the integer and fractional parts, commonly written $Qm.n$. Because the binary point never moves, addition, subtraction and multiplication are ordinary integer operations: cheap, single-cycle, and small in silicon [16]. The price is a fixed dynamic range. Values too large wrap around; values too small round to zero.

Floating-point representation stores a sign, exponent and mantissa, so the binary point moves with the magnitude [17]. This buys enormous dynamic range but costs a great deal: every operation requires exponent alignment, normalisation and special-case handling, which needs a dedicated floating-point unit and takes multiple clock cycles. Chapter 7 measures this concretely: in the implemented DRHE design, the floating-point operators account for about two thirds of the LUTs and every one of the DSP slices, and replacing them with fixed point, with nothing else changed, removes 42 % of the LUTs, 61 % of the flip-flops and 67 % of the DSPs.

2.6.2 Quantisation error, and the FX16 step

The range-FFT output is naturally a complex floating-point quantity. Before it can be compressed by a fixed-point pipeline it has to be quantised to 16-bit integers — the **FX16** step. This is a scaling followed by rounding:

$$\sigma = \frac{32767}{A_{\max} 2^{\text{shift}} \sum_k w_k / (2N_s)}, \quad x_{\text{FX16}} = \text{round}(\sigma \cdot x), \quad (2.11)$$

where $A_{\max}$ is the full-scale ADC amplitude and w is the window applied along fast time.

Two things about Equation (2.11) matter later. First, the rounding introduces an error bounded by half a least-significant bit, which acts as a wideband noise floor [18]; this is the dominant source of detection degradation in the whole chain, and Chapter 6 separates it cleanly from the cost of compression. Second, and more consequentially for this thesis, the scale factor is *fixed* — it is calibrated from a theoretical full-scale ADC sinusoid, not from the data. If the actual capture sits well below full scale, the top bits of every resulting `int16` are structurally zero. Section 6.4 shows that this is exactly what happens, and that it accounts for more of the reported compression ratio than the compression algorithms do.

Carry forward: the FX16 scale factor is fixed and calibrated to full scale. Whether the capture actually fills the container is a property of the sensor’s gain staging, and it silently inflates any compression ratio measured downstream.

2.7 FPGAs and high-level synthesis

2.7.1 What an FPGA is

A field-programmable gate array is a chip full of small configurable logic blocks and configurable wiring between them. Rather than executing instructions, an FPGA *becomes* the circuit you describe. The practical consequences for this application are three:

- **Parallelism.** A processor handles four receive channels one after another. An FPGA can instantiate four copies of the datapath and do all four in the same clock cycle. Every design in this thesis does exactly that.

- **Deterministic latency.** There is no cache, no operating system, no interrupt jitter. A design that takes 61,253 cycles takes 61,253 cycles every time, which is what a safety-critical pipeline needs.
- **Reconfigurability.** The design can be changed after fabrication, which is what makes an FPGA the right platform for a thesis — and what made it practical to build, measure and then substantially revise two compressors inside one project.

In production, a design proven on an FPGA is normally migrated into an ASIC or into a fixed-function accelerator block inside a larger SoC, for cost and power reasons [19, 21]. The FPGA here is a proof of concept, not a proposed product.

2.7.2 The four currencies: LUT, FF, BRAM, DSP

FPGA resource usage is reported in four units, and it is worth knowing what each is because the two algorithms in this thesis trade them against each other in an interesting way.

LUT

Look-up table. A small memory that implements an arbitrary Boolean function of a few inputs. LUTs are the raw material of combinational logic. Complicated arithmetic — square roots, arctangents — costs a lot of LUTs.

FF *Flip-flop.* A one-bit register. Flip-flops hold values between clock cycles; deep pipelines need many of them.

BRAM

Block RAM. Dedicated on-chip memory, here counted in 18 kbit units. Arrays that are too large to live in flip-flops go here. BRAMs typically have two access ports, which becomes important below.

DSP

DSP slice. A hardened multiply–accumulate unit. Using a DSP for a multiply is far cheaper than building one from LUTs.

The target device throughout is an AMD Kintex UltraScale+ `xcku5p-ffvb676-2-e`, which provides 216,960 LUTs, 433,920 flip-flops, 960 18 kbit BRAMs and 1,824 DSP slices.

2.7.3 Pipelining, initiation interval and $F_{\max}$

Three timing concepts recur throughout the results.

Pipelining splits a long computation into stages separated by registers, so several inputs can be in flight at once. The *depth* of a pipeline is how many cycles a single input takes to traverse it.

Initiation interval (II) is how many cycles must elapse between accepting one input and the next. $II = 1$ means a new sample every cycle, which is the goal. $II = 2$ halves throughput. II is limited by the tightest resource constraint in the loop body — very often, as Chapter 6 shows, by the number of access ports on a block RAM.

$F_{\max}$ is the highest clock frequency at which the longest combinational path still settles within one period. HLS estimates it before the circuit exists; the figure that counts is Vivado’s, after routing. The two can disagree badly: all four of this thesis’s main designs were estimated below 100 MHz, and all four close above it (Section 7.1).

Latency, throughput and $F_{\max}$ interact: a frame of K beats at initiation interval II and clock f takes roughly $K \cdot II / f$ seconds, *plus* the pipeline depth once per invocation. That last term is often small, but not always: DRHE pays it once per ramp, a fifth of its frame time, and Section 5.5 contains a case where it costs 65,000 cycles that a naive model misses entirely.

2.7.4 High-level synthesis

Traditionally FPGA designs are written in a hardware description language such as VHDL or Verilog, at the level of registers and wires. *High-level synthesis* (HLS) instead takes annotated C++ and generates the RTL automatically. Loops become state machines; arrays become memories; pragmas tell the tool to pipeline a loop, partition an array, or expose a port as an AXI4-Stream interface.

HLS is used throughout this thesis for a specific reason beyond convenience: because the design source is C++, the *same source* can be compiled and run as ordinary software against the MATLAB reference, before any hardware is generated. That makes the verification ladder of Chapter 4 possible, and it is how both defects in Section 5.4 were caught in seconds rather than in hours of RTL simulation.

2.7.5 AXI4-Stream

The interface used by both compressors is AXI4-Stream: a simple valid/ready handshake carrying a fixed-width data word per beat. The designs here accept 128 bits per beat — four receive channels, each a 16-bit real and a 16-bit imaginary part — and emit 256 bits per beat of compressed bitstream. A useful consequence for benchmarking is that “128 bits per cycle” is the theoretical maximum input rate, and therefore what a perfectly efficient design at $\Pi = 1$ would achieve. Section 8.1 uses this to resolve an ambiguity in a published throughput figure.

2.8 Related work and the baseline

2.8.1 Deterministic industrial schemes

Because automotive safety systems demand bounded latency, commercial radar processors use lightweight deterministic compressors. Texas Instruments uses Exponential-Golomb Encoding [20]; NXP’s S32R family uses block floating-point formats such as CP4D, in which several mantissas share one exponent [21]. These are described as “near lossless” — CP4D introduces under 0.25 dB of signal loss — and reach ratios of roughly 2 to 4.

2.8.2 Block Adaptive Quantization

BAQ originates in synthetic aperture radar [22]. It divides the data into blocks, estimates each block’s statistics, and adapts the quantiser step size accordingly, so weak blocks are quantised finely and strong blocks coarsely. It is lossy. It appears in this thesis as a comparison point, and Chapter 6 shows what its losses cost in detection terms.

2.8.3 Neural and spectral approaches

Autoencoder-based compressors reach high ratios but are expensive to deploy at the sensor [23]. AdaRadar [24] avoids this by keeping the edge side simple — a discrete cosine transform and spectral pruning — and placing the neural decoder at the SoC, with a feedback loop that adapts the rate to downstream perception performance. These are outside the scope of this thesis, which concerns itself with lossless schemes, but they define the upper end of the ratio spectrum.

2.8.4 The baseline: Kiem’s thesis

The direct predecessor of this work is Lukas Kiem’s master’s thesis [11], which performed a design-space exploration across Exponential-Golomb variants, Infineon common-exponent formats, and predictive coders, and concluded that Doppler Redundancy Hybrid Encoding

gave the best combination of compression ratio, noise-floor impact and detection accuracy. He implemented DRHE on a Zynq UltraScale+ device (XCZU3CG-SFVC784-1-e, Vitis and Vivado 2022.2), reporting a compression ratio of 3.2, an IP-core latency of 230 ns and a throughput of “11.92 Gbit/s” at 100 MHz.

That thesis is the source of the DRHE specification used here, including the filter coefficients and the Huffman dictionary, both of which were verified entry-by-entry against it (Section 3.1.6). It left three gaps — synthetic-only validation, a narrow algorithm comparison, and a compression-only implementation — which Section 1.4 listed as this thesis’s objectives.

It also, as Chapter 6 shows, contains an assumption that does not survive the move to a TDM-MIMO cascade. That is not a criticism of the work: a single-transmitter radar has no transmitter-interleaved ramp axis, and on such a radar the specification is correct as written. It becomes wrong only when applied, unmodified, to a sensor of a different kind — which is exactly what this thesis did before measuring it.

2.9 Summary of what the later chapters assume

1. The data to be compressed is the *range-FFT output*, quantised to 16-bit integers, arranged as (ramp $\times$ range bin $\times$ receiver).
2. The redundancy worth exploiting lives along *slow time*: a target at constant velocity gives nearly constant magnitude and a constant phase increment.
3. On this sensor the ramp axis is *transmitter-interleaved*, so the slow-time neighbour of ramp m is ramp $m - N_t$, not ramp $m - 1$.
4. A compression ratio is the product of a decorrelation gain and an entropy-coding gain, and the two must be measured separately.
5. The FX16 scale factor is fixed to full scale, so container occupancy is a confounding variable in any measured ratio.
6. FPGA cost is measured in LUTs, flip-flops, BRAMs and DSPs, and timing in initiation interval, pipeline depth and $F_{\max}$.

Chapter 3

The Two Algorithms

Both compressors follow the two-stage structure of Section 2.4: a predictor that turns samples into small residuals, followed by an entropy coder that turns small residuals into few bits. They share the entropy coder exactly, which is deliberate — it means any difference in compression ratio between them is attributable entirely to the predictor, and it is what makes the no-prediction baseline of Chapter 6 a fair comparison for both.

This chapter specifies the entropy coder first, then each predictor.

3.1 The shared entropy coder: turning residuals into bits

This section is deliberately detailed. The entropy coder is where the compression ratio is physically realised — it is the only part of either design that actually emits bits — so understanding exactly what it charges for is what makes the results of Chapter 6 interpretable rather than merely quotable.

3.1.1 What a residual is

Let $x[m]$ be the sample to be transmitted and let P be some rule — the *predictor* — that produces an estimate

$$\hat{x}[m] = P(x[m-1], x[m-2], \dots) \quad (3.1)$$

from samples that have *already been transmitted*. The encoder does not send $x[m]$. It sends the *residual*

$$d[m] = x[m] - \hat{x}[m]. \quad (3.2)$$

The decoder, having already reconstructed $x[m-1], x[m-2], \dots$, runs the identical rule P on them, obtains the identical $\hat{x}[m]$, and recovers

$$x[m] = d[m] + \hat{x}[m]. \quad (3.3)$$

Two things follow, and both matter.

Nothing is lost. Equation (3.2) is a *re-expression*, not an approximation. Given $\hat{x}[m]$, the map $x \mapsto d$ is a bijection. The sequence $\{d[m]\}$ carries exactly the information the sequence $\{x[m]\}$ carried — no more, no less. This is why a predictive coder can be lossless at all.

The predictor must be causal. P may only use samples the decoder already has. This is not a stylistic preference but a hard constraint: a predictor that peeked at $x[m]$ itself would be unreproducible at the far end. It is also why Section 2.4.4 makes such a point of *which* version of the past the encoder uses — the originals or the reconstructions — since those differ the moment anything is not exactly lossless.

Table 3.1: The same sixteen real samples, written two ways. Range bin 75, receive channel 1, ramps 168–183. Both rows carry identical information.

m	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183
x (raw)	−69	−52	59	−6	−29	−15	59	−20	−69	−73	−45	84	−73	−50	59	−5
d (residual)	3	−3	0	−1	1	3	−4	3	−1	−2	−1	0	−2	4	−1	1

3.1.2 Why a re-expression can be worth anything

If no information is lost and none is gained, why is the residual sequence any cheaper to write down?

Because the *distribution* has changed, and a well-chosen code charges by distribution. Table 3.1 shows this on real data: one range bin of one receive channel, sixteen consecutive ramps, taken from frame 0 of the ColoRadar capture.

The raw values swing over $[-73, +84]$; writing any of them needs eight bits. The residuals never leave $[-4, +4]$; four bits would do. Since the two rows are informationally equivalent, that factor of two is free — provided the coder can be told about it.

And there is the catch. The bound $[-4, +4]$ holds for *these* sixteen samples. Somewhere else in the frame a target appears, or the scene changes, and a residual of 400 arrives. A fixed four-bit field cannot carry it. What is needed is a code that spends few bits on the common case, many on the rare one, and lets the decoder tell which is which.

3.1.3 The parsing problem

The naive approach fails immediately. Suppose the rule were “small values get four bits, large values get twelve”. When the decoder reads four bits, it has no way to know whether it has just read a complete small value or the first third of a large one. Variable-length coding is not merely a matter of writing fewer bits; it is a matter of writing them so that the boundaries can be recovered.

There are two standard answers:

1. **A prefix-free code over every possible value.** Assign every one of the 65,536 possible 16-bit residuals its own codeword, no codeword being a prefix of another. This is optimal, and it is what a Huffman code over the raw alphabet would give — but it needs a 65,536-entry table, and the table has to be derived from the data and transmitted alongside it.
2. **Split the value into a size and a payload.** Send a small *description of how big the value is*, coded with a prefix-free code over a handful of symbols, followed by the value itself in exactly that many raw bits. The decoder reads the size first, so it always knows how many bits to read next.

Both compressors here take the second route, which is the same construction JPEG uses for DC coefficients. It is cheap in every respect that matters for hardware: the prefix-free table has sixteen entries rather than 65,536, it can be fixed at design time rather than derived per frame, and decoding a symbol is a sixteen-way comparison rather than a tree walk.

3.1.4 The category S_4

The “size” half of the split is the category

$$S_4(v) = \begin{cases} 0 & v = 0, \\ \min(15, \lfloor \log_2 |v| \rfloor + 1) & v \neq 0, \end{cases} \quad (3.4)$$

Table 3.2: The first few categories. Category $k \geq 1$ covers $|v| \in [2^{k-1}, 2^k - 1]$, which counting both signs is exactly 2^k distinct values — precisely what k raw bits can enumerate.

S_4	values covered	count	
0	$v = 0$	1	the only category with no payload
1	$v \in \{-1, +1\}$	2	
2	$ v \in \{2, 3\}$	4	
3	$ v \in [4, 7]$	8	
4	$ v \in [8, 15]$	16	
$\vdots$			
15	$ v \in [16384, 32767]$	32,768	exactly fills 15 bits

which is simply the number of bits needed to write $|v|$. Its ranges are given in Table 3.2.

The design is tight by construction: category k contains exactly 2^k values, so k payload bits identify one of them with nothing wasted and nothing left over. Section 3.1.12 shows what happens at the one place where “nothing left over” turns out to be a problem.

3.1.5 The APPEND bits

The payload half is

$$A(v) = \begin{cases} v & v \geq 0, \\ v + 2^{S_4(v)} - 1 & v < 0, \end{cases} \quad (3.5)$$

written in $S_4(v)$ raw bits.

The offset for negative values looks arbitrary but is doing something specific. Table 3.3 enumerates category 3 in full.

Table 3.3: Every codepoint of category 3. The negative values map to patterns with a clear most significant bit, the positive values to patterns with it set — so the sign is recovered from that one bit without transmitting a sign bit.

v	$A(v)$	3-bit pattern	MSB
-7	0	000	0
-6	1	001	0
-5	2	010	0
-4	3	011	0
+4	4	100	1
+5	5	101	1
+6	6	110	1
+7	7	111	1

Because positive values in category k span $[2^{k-1}, 2^k - 1]$ — exactly the k -bit patterns with the top bit set — and the offset maps the negatives onto $[0, 2^{k-1} - 1]$ — exactly those with it clear — the sign falls out of the payload for free. The decoder’s rule is therefore

$$v = \begin{cases} A & \text{if the MSB of the } S_4 \text{ payload bits is set,} \\ A - (2^{S_4} - 1) & \text{otherwise,} \end{cases} \quad (3.6)$$

which in hardware is one bit test and one subtract.

3.1.6 Cost, and the shape of the dictionary

The total cost of coding one residual is

$$L(v) = \underbrace{\ell(S_4(v))}_{\text{Huffman codeword}} + \underbrace{S_4(v)}_{\text{payload}}, \quad (3.7)$$

where $\ell(\cdot)$ is the codeword length for that category. The dictionary is *fixed* — not computed per frame — which saves transmitting it and removes a serial dependency from the hardware. Its lengths are

$$\ell = [4, 3, 2, 2, 2, 5, 6, 7, 8, 9, 10, 11, 14, 14, 13, 12] \text{ bits.} \quad (3.8)$$

The full dictionary, codewords included, is in Appendix A. Every one of the sixteen entries was verified against the published table [11, Table A.1]; all sixteen agree once the published most-significant-bit-first codes are reversed for the least-significant-bit-first packer used in this implementation.

Substituting into Equation (3.7) gives the cost curve that governs everything downstream:

S_4	$ v $ range	ℓ	payload	total
0	0	4	0	4
1	1	3	1	4
2	2–3	2	2	4
3	4–7	2	3	5
4	8–15	2	4	6
5	16–31	5	5	10
6	32–63	6	6	12
7	64–127	7	7	14
8	128–255	8	8	16

Three features of this curve deserve attention.

Everything below 16 is nearly free. A residual anywhere in $[-15, +15]$ costs between four and six bits. Against sixteen raw bits that is already a compression ratio between 2.7 and 4.0, achieved without the predictor doing anything clever — it only has to keep residuals small.

There is a cliff at category 5. Cost jumps from six bits to ten. A residual of 15 costs six bits; a residual of 16 costs ten. The coder is, by design, forgiving of small errors and severe about larger ones.

Above category 8 the code loses. A residual of 128 costs sixteen bits — exactly what the raw value would have cost. Beyond that the coder actively expands. This is not a flaw: it is the inevitable consequence of assigning short codewords to some symbols, and it is why the `random-int16` edge case of Section 4.4 expands the data by 67%.

The dictionary’s shape encodes an assumption about the data — that categories 2, 3 and 4 dominate, since those get the two-bit codewords. Section 3.1.10 measures whether that assumption holds.

3.1.7 Worked example 1: one value, in full

Take $v = -5$.

1. $|v| = 5$, and $\lfloor \log_2 5 \rfloor + 1 = 3$, so $S_4 = 3$.
2. $v < 0$, so $A(v) = -5 + 2^3 - 1 = 2$, written in three bits as 010.

3. The dictionary gives category 3 the codeword 00, two bits.
4. The emitted bits are 00 010 — five bits, against sixteen for the raw value.

Decoding runs the same steps backwards. The decoder matches 00 against the dictionary, recovers $S_4 = 3$, reads the next three bits 010, sees the most significant is 0 so the value was negative, and computes $2 - (2^3 - 1) = -5$.

3.1.8 Worked example 2: sixteen real samples, both ways

The single-value example shows the mechanism. The following example shows why it matters, and it doubles as a concrete demonstration of the central result of this thesis.

Table 3.4 traces sixteen consecutive ramps of one range bin — the same bin as Table 3.1 — through the DRHE predictor twice. The left half predicts from ramp $m - 1$, which is how the algorithm is described in the literature. The right half predicts from ramp $m - n_{\text{Tx}}$, which Section 2.2.3 argued is the true slow-time neighbour. Everything else — the filters, the coefficients, the entropy coder, the dictionary — is identical between the two halves. Only the lag differs.

Table 3.4: Sixteen real samples traced through DRHE at two prediction lags. Range bin 75, receive channel 1, frame 0, ramps 168–183; real part shown, the imaginary part behaves identically. The window is late enough in the frame that both predictors’ filters are fully converged. The two halves differ only in which earlier ramp the prediction comes from.

m	TX	x	predicted from ramp $m - 1$				predicted from ramp $m - n_{\text{Tx}}$			
			$\hat{x}$	d	S_4	bits	$\hat{x}$	d	S_4	bits
168	0	−69	67	−136	8	16	−72	3	2	4
169	1	−52	−75	23	5	10	−49	−3	2	4
170	2	59	81	−22	5	10	59	0	0	4
171	3	−6	64	−70	7	14	−5	−1	1	4
172	4	−29	24	−53	6	12	−30	1	1	4
173	5	−15	−29	14	4	6	−18	3	2	4
174	6	59	−6	65	7	14	63	−4	3	5
175	7	−20	−83	63	6	12	−23	3	2	4
176	8	−69	−93	24	5	10	−68	−1	1	4
177	9	−73	−65	−8	4	6	−71	−2	2	4
178	10	−45	−44	−1	1	4	−44	−1	1	4
179	11	84	28	56	6	12	84	0	0	4
180	0	−73	70	−143	8	16	−71	−2	2	4
181	1	−50	−75	25	5	10	−54	4	3	5
182	2	59	81	−22	5	10	60	−1	1	4
183	3	−5	67	−72	7	14	−6	1	1	4
total bits						176	66			
bits per value						11.00	4.12			
compression ratio						1.455	3.879			

The left half is a predictor that is barely working. Its residuals (−136, +23, −22, −70, ...) are frequently *larger in magnitude than the samples they replace*, they land in categories 5 through 8, and they cost 11.00 bits per value — a compression ratio of 1.455, which is to say the entropy coder is salvaging a little from a predictor that has actively made things worse.

The right half is the same predictor on the correct axis. Its residuals never exceed 4 in magnitude, they land in categories 0 through 3, and they cost 4.12 bits per value.

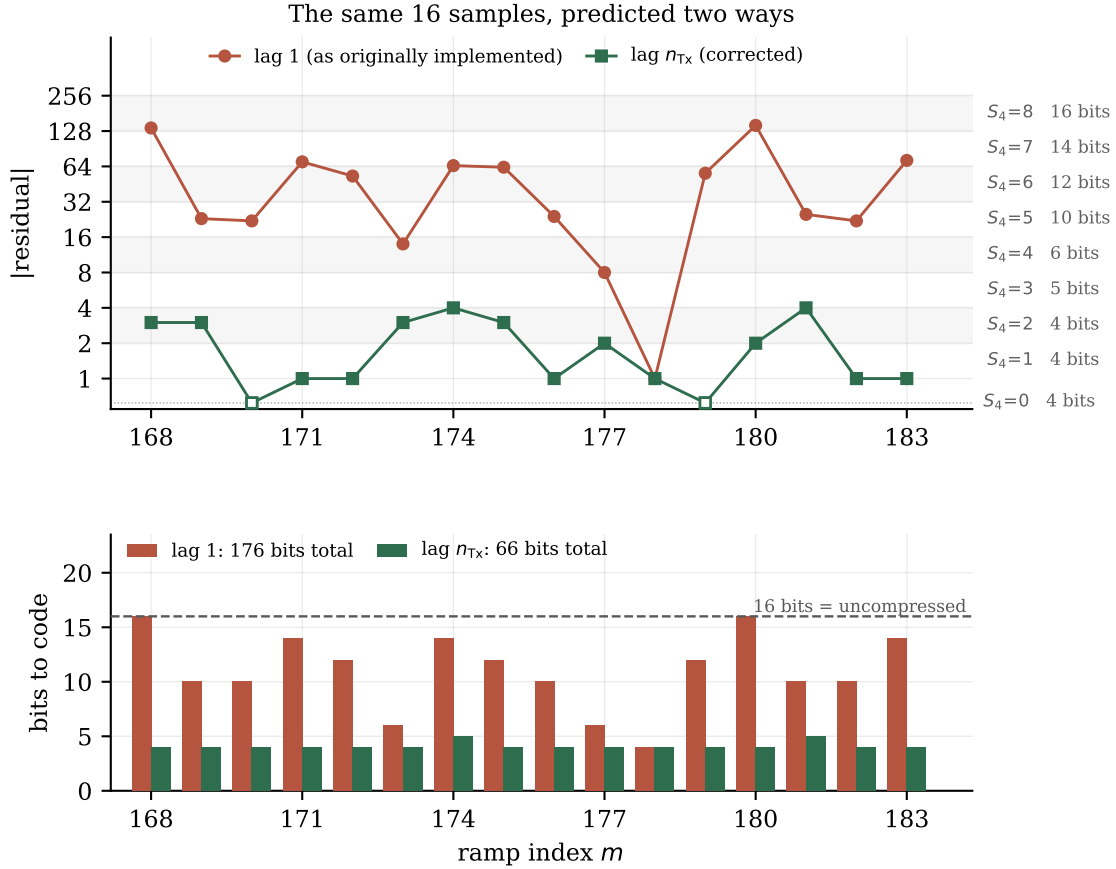


Figure 3.1: Table 3.4 drawn. Above: residual magnitude on a logarithmic scale, with the S_4 category bands and their bit costs marked on the right; hollow markers are residuals of exactly zero. Below: the bits each sample costs, against the 16 bits it would have taken uncompressed. The lag-1 predictor spends 176 bits on this window; the same predictor on the correct axis spends 66.

Why the difference is so large

Table 3.5 answers this directly, and it is worth studying because it is the mechanism behind every result in Chapter 6. Recall from Section 2.1 that DRHE predicts magnitude with a smoother and phase with a double integrator, both of which work when the quantity being predicted changes slowly and smoothly.

Look at the $\Delta\theta$ columns. Against ramp $m - 1$ the phase increment is -2.244 , then $+0.592$, then $+1.230$, then -0.678 — it swings over most of $[-\pi, \pi]$ with no pattern a two-tap filter could follow. Against ramp $m - n_{Tx}$ it is 0.002 , -0.040 , 0.017 , -0.008 : essentially zero, frame after frame, because this is a stationary target and twelve ramps later the same antenna sees the same phase.

The magnitude column tells the same story: ± 22 against the immediate predecessor, ± 4 against the same-transmitter predecessor.

The reason is the one given in Section 2.2.3. Ramps 168 and 167 were transmitted by *different antennas*, so the phase between them is set by where the target is in *angle*, via Equation (2.6) — and the twelve transmitters are at twelve different positions, so that phase jumps around on a period-12 pattern. Ramps 168 and 156 were transmitted by the *same* antenna, so the phase between them is set by how fast the target is *moving*, via Equation (2.5) — and it is not moving.

Table 3.5: The same window in polar coordinates. Against ramp $m - 1$ the magnitude moves by up to 22 and the phase jumps unpredictably over the full $\pm\pi$ range. Against ramp $m - n_{\text{Tx}}$ the magnitude moves by at most 4 and the phase increment is essentially zero — a stationary target, exactly what the double integrator is built for.

m	TX	sample		change vs. $m - 1$		change vs. $m - n_{\text{Tx}}$	
		$ x $	$\angle x$	$\Delta x $	$\Delta\theta$	$\Delta x $	$\Delta\theta$
168	0	74.1	-2.769	-21.8	-2.244	-3.2	0.002
169	1	91.3	-2.177	17.2	0.592	-0.0	-0.040
170	2	101.0	-0.947	9.8	1.230	-2.5	0.017
171	3	110.2	-1.625	9.1	-0.678	3.0	-0.008
172	4	107.0	-1.845	-3.2	-0.220	-2.2	0.004
173	5	112.0	-1.705	5.0	0.140	-0.3	0.018
174	6	97.0	0.917	-15.0	2.622	-0.4	0.023
175	7	81.5	1.819	-15.5	0.902	0.4	-0.027
176	8	73.4	2.794	-8.1	0.975	-0.1	0.043
177	9	75.0	-2.913	1.6	0.576	2.4	0.021
178	10	95.3	-2.063	20.3	0.850	0.9	0.005
179	11	97.8	-0.537	2.5	1.526	1.9	-0.013
180	0	77.5	-2.799	-20.3	-2.263	3.4	-0.031
181	1	92.6	-2.141	15.2	0.659	1.4	0.036
182	2	102.6	-0.958	10.0	1.182	1.6	-0.011
183	3	106.1	-1.618	3.5	-0.659	-4.0	0.007

The actual bits

It is worth seeing the bitstream itself. Coding the first six residuals of the corrected column, using Equations (3.4) and (3.5) and the dictionary of Appendix A:

d	S_4	$A(d)$	codeword	payload	emitted	bits
+3	2	3	10	11	10 11	4
-3	2	0	10	00	10 00	4
0	0	—	0101	—	0101	4
-1	1	0	001	0	001 0	4
+1	1	1	001	1	001 1	4
+3	2	3	10	11	10 11	4

which concatenates to the 24-bit stream

1011 1000 0101 0010 0011 1011

carrying six samples that would have occupied 96 bits raw.

The same six samples on the uncorrected axis — residuals -136 , $+23$, -22 , -70 , -53 , $+14$ — fall into categories 8, 5, 5, 7, 6 and 4 and occupy $16 + 10 + 10 + 14 + 12 + 6 = 68$ bits. Nearly three times as many, for identical information.

3.1.9 Decoding the same stream

Decoding is the mirror image, and seeing it done confirms that no side information beyond the fixed dictionary is needed. The decoder holds a bit buffer and consumes from the low end.

1. Buffer is 1011 1000. . . . Match the low bits against the dictionary. The two-bit prefix 10 is category 2's codeword; no shorter or longer entry can also match, because the reversed code set is prefix-free. Consume two bits.

Table 3.6: Where the bits go, one frame, four receive channels, 196,608 values. “% values” is how often a category occurs; “% bits” is how much of the output it accounts for. The two columns are very different, and the difference is the whole point of entropy coding.

S_4	cost	no prediction		DRHE, lag 1		DRHE, lag n_{Tx}	
		% values	% bits	% values	% bits	% values	% bits
0	4	16.71	14.61	11.78	9.81	18.85	17.87
1	4	28.57	24.97	22.01	18.33	32.94	31.22
2	4	31.45	27.49	31.95	26.60	34.40	32.60
3	5	14.66	16.02	22.21	23.12	11.31	13.40
4	6	4.31	5.65	6.46	8.07	1.49	2.12
5	10	1.75	3.82	2.36	4.92	0.50	1.18
6	12	1.15	3.01	1.43	3.58	0.29	0.82
7	14	1.14	3.47	1.09	3.17	0.18	0.59
8	16	0.19	0.66	0.65	2.17	0.03	0.13
9	18	0.07	0.29	0.06	0.23	0.01	0.05
bits/value		4.576		4.803		4.220	
compression ratio		3.497		3.331		3.792	

- Category 2 means two payload bits. Read 11, so $A = 3$. The most significant of those two bits is set, so the value is positive: $d = +3$. Consume two bits.
- Add the prediction: the decoder has already reconstructed ramp $168 - 12 = 156$ for this bin, runs the identical filters on it, obtains $\hat{x} = -72$, and outputs $x = 3 + (-72) = -69$ — which is the value in Table 3.4.
- Buffer is now 1000 0101 Match 10 again: category 2. Read payload 00, so $A = 0$, MSB clear, so $d = 0 - (2^2 - 1) = -3$. Prediction -49 , output -52 . Correct.
- Buffer is 0101 0010 Now 10 does *not* match, because the low two bits are 01, which is no entry’s codeword. The four-bit prefix 0101 is category 0’s codeword. Category 0 has no payload and means $d = 0$ exactly. Prediction 59, output 59. Correct.

Step 5 is the one that shows why prefix-freeness is doing real work: the decoder never had to be told how long the next codeword would be.

3.1.10 Where the bits actually go

The sixteen-sample window is illustrative but small. Table 3.6 gives the same accounting over an entire frame — 196,608 values, all four receive channels — which is what the compression ratio actually integrates.

Several things are visible here that the headline ratios hide.

The expensive categories are rare but not negligible. Under lag-1 prediction, categories 5 through 8 account for only 5.53% of values but 13.84% of bits. One value in eighteen is consuming one bit in seven. That asymmetry is exactly why the tail of the residual distribution matters so much, and why Figure 6.3 plots the 99th percentile rather than the mean.

The correction works by emptying the tail. Comparing the last two column pairs: the corrected predictor moves 86.19% of values into categories 0–2, all of which cost four bits, and cuts categories 5–8 from 5.53% to 1.00% of values. It is not that individual residuals got a little smaller; it is that the expensive ones almost stopped occurring.

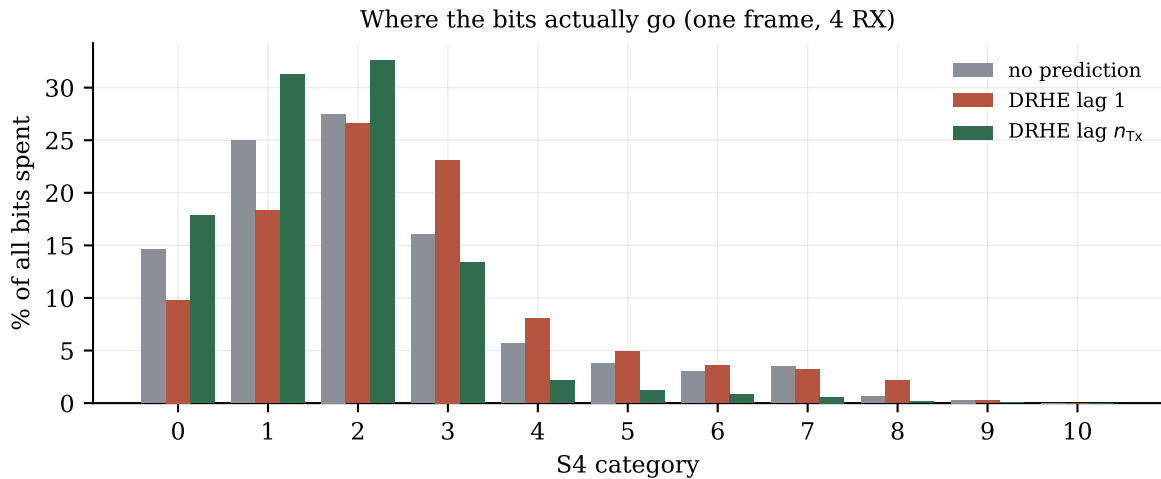


Figure 3.2: The “% bits” columns of Table 3.6 drawn. The corrected predictor concentrates spending into the three four-bit categories; the lag-1 predictor pushes it out into categories 3 through 8, where each value costs two to four times as much.

The lag-1 predictor loses to doing nothing in a specific way. Against the no-prediction column it *gains* slightly in categories 0–2 but loses heavily in 3, 4 and 8. It is redistributing values upward, not downward. A predictor that helps should hollow out the right-hand side of this table; this one fills it in.

3.1.11 How good is the coder itself?

It is worth separating the coder’s performance from the predictor’s, since the two are independent and only one of them turned out to be the problem.

The order-0 entropy of the raw FX16 values — the theoretical floor for any code that treats values independently, from Equation (2.9) — is 4.094 bits. The coder achieves 4.576 bits on the same data. That is an overhead of 0.48 bits, or 11.8 %, for a dictionary fixed at design time against a different dataset; on the five-frame average used in Chapter 6 the figure is 6.2 %.

That is a good result, and it has a clear implication: **there is very little left to win in the entropy coder.** Even a perfect, data-derived, per-frame-optimal code would improve the compression ratio by only a few per cent. All the remaining leverage is in the predictor — which is what makes a predictor that was contributing nothing such an expensive thing not to have noticed.

One caveat is worth recording for future work. The dictionary gives its two-bit codewords to categories 2, 3 and 4, which suits a distribution centred there. After the prediction-axis correction the distribution has moved *down*: categories 0, 1 and 2 now hold 86 % of values, and category 0 — exactly zero — is paying four bits for what is now the second most common symbol. Re-deriving the dictionary for the corrected residual distribution is an obvious and cheap improvement that this thesis does not make.

3.1.12 A representability hole

There is one value the code cannot represent, and it caused a real defect.

Category 15 spans $|v| \in [16384, 32767]$. Counting both signs that is exactly $2^{15} = 32,768$ values, which exactly fills the fifteen payload bits — the tightness that Table 3.2 noted as a virtue leaves no spare codepoint. But `int16` arithmetic can produce $v = -32768$. Applying Equation (3.5) gives

$$A(-32768) = -32768 + 32767 = -1, \quad (3.9)$$

which truncated to fifteen bits is 0x7FFF — bit-for-bit the same payload as $v = +32767$, with the same category and therefore the same codeword. The two are indistinguishable on the wire, and the decoder returns +32767.

This is not hypothetical. Section 5.4.1 reports it firing in a synthetic edge case with a reconstruction error of 65,535, which is precisely the distance from -32768 to $+32767$ — a diagnostic number that pointed straight back to this paragraph.

3.2 DRHE: Doppler Redundancy Hybrid Encoding

3.2.1 Why predict in polar coordinates

Section 2.1 established what a range-bin sequence looks like across slow time for a target at constant velocity: the *magnitude* is nearly constant, and the *phase* advances by a constant increment. In Cartesian (I, Q) coordinates that is a point rotating on a circle — both components oscillate, and a simple difference between consecutive samples is large. In polar coordinates the same sequence is two nearly-trivial signals: a constant and a ramp.

DRHE therefore converts to polar, predicts magnitude and phase separately with filters matched to those two shapes, converts the prediction back to Cartesian, and subtracts.

3.2.2 The two predictors

For ramp m and range bin n , with $a[m, n]$ the magnitude and $\theta[m, n]$ the phase, the predictions are

$$\hat{a}[m, n] = \alpha \hat{a}[m-1, n] + (1 - \alpha) a[m-1, n], \quad (3.10)$$

$$\hat{\theta}[m, n] = \beta \hat{\theta}[m-1, n] + (2 - \beta) \theta[m-1, n] - \theta[m-2, n], \quad (3.11)$$

with $\alpha = 0.6$ and $\beta = 0.4$ [11, Table 4.4], and $\hat{\theta}$ wrapped into $[-\pi, \pi]$.

Equation (3.10) is a first-order infinite-impulse-response smoother: it blends the previous prediction with the previous observation. For a constant magnitude it converges to that constant, and the α term gives it some immunity to noise on any single ramp.

Equation (3.11) is more interesting. Setting $\beta = 0$ reduces it to $\hat{\theta}[m] = 2\theta[m-1] - \theta[m-2]$, which is a *double integrator*: it extrapolates the last two phases along a straight line. For a phase advancing by a constant $\Delta\theta$ per ramp — exactly Equation (2.5) — that prediction is perfect. The β term mixes in the previous prediction to damp the response to noise.

The prediction is then converted back and subtracted, with int16 wraparound $\mathcal{W}(\cdot)$:

$$d_{\Re} = \mathcal{W}(x_{\Re} - \text{round}(\hat{a} \cos \hat{\theta})), \quad (3.12)$$

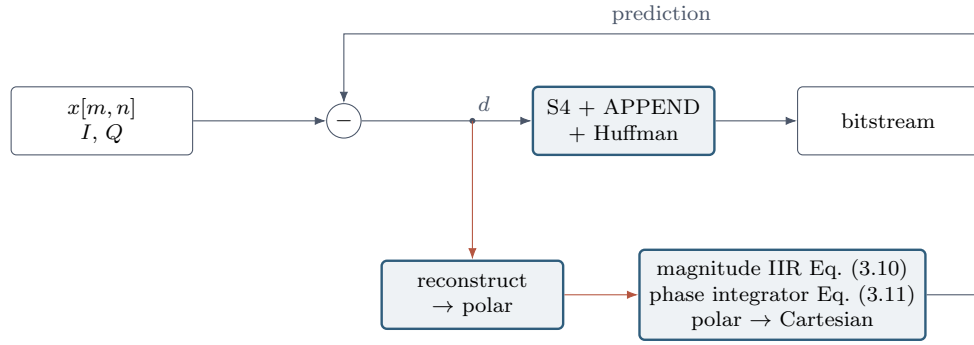
$$d_{\Im} = \mathcal{W}(x_{\Im} - \text{round}(\hat{a} \sin \hat{\theta})), \quad (3.13)$$

and $d_{\Re}$, $d_{\Im}$ are handed to the entropy coder of Section 3.1.

3.2.3 The streaming property

DRHE has a property that matters more than its compression ratio: it is *causal and streaming*. Each output depends only on the current sample and on state carried from ramp $m-1$. Concretely, the implementation holds five floating-point values per (receive channel, range bin) — the magnitude prediction, the previous magnitude, the phase prediction, and the previous two phases — and nothing else.

It therefore needs no frame buffer, and it emits its first compressed bits as soon as the first sample arrives. For a compressor sitting between an FFT and a memory bus, that is exactly the right shape. Section 6.7 returns to this when weighing DRHE against LPC.



closed loop: the encoder's state is driven by what the *decoder* will reconstruct

Figure 3.3: DRHE as implemented. Prediction happens in the polar domain with two filters matched to the expected behaviour of a constant-velocity target; the prediction is converted back to Cartesian before subtraction. The red path is the closed-loop correction of Section 5.4.1: the encoder drives its state from what the decoder will reconstruct, not from the original sample.

3.2.4 The assumption

DRHE's correctness as a *compressor* — it is lossless regardless — is not in question, but its *effectiveness* rests on one assumption made implicitly by the indices in Equations (3.10) and (3.11): that ramp $m - 1$ is the slow-time predecessor of ramp m .

Section 2.2.3 showed that on a TDM-MIMO cascade it is not.

3.3 LPC: linear prediction with Yule–Walker coefficients

3.3.1 Autoregressive modelling

The second algorithm takes a more general approach. Rather than assuming a particular signal shape and hand-designing filters for it, it *fits* a model to the data.

A second-order autoregressive (AR) model says each sample is approximately a linear combination of the previous two:

$$\hat{x}[m] = a_1 x[m - 1] + a_2 x[m - 2]. \quad (3.14)$$

Given a sequence, the coefficients that minimise the mean-square prediction error satisfy the Yule–Walker equations, which for order two are

$$\begin{bmatrix} R_0 & R_1 \\ R_1 & R_0 \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \end{bmatrix} = \begin{bmatrix} R_1 \\ R_2 \end{bmatrix}, \quad (3.15)$$

where $R_k = \sum_m x[m]x[m - k]$ is the autocorrelation at lag k . Solving directly,

$$a_1 = \frac{R_1(R_0 - R_2)}{R_0^2 - R_1^2}, \quad a_2 = \frac{R_0 R_2 - R_1^2}{R_0^2 - R_1^2}. \quad (3.16)$$

3.3.2 Stability

An AR model whose poles lie outside the unit circle is unstable: prediction errors grow rather than decay. For order two the stability triangle gives the conditions

$$|a_2| < 1 \quad \text{and} \quad |a_1| < 1 - a_2. \quad (3.17)$$

If either fails, or if the denominator $R_0^2 - R_1^2$ vanishes, the implementation sets $a_1 = a_2 = 0$, which degrades gracefully to “predict zero” — i.e. code the raw value. On the real data used here the test never fires: 100.00% of coefficient sets pass.

Table 3.7: The two algorithms compared on assumptions and structure, before any measurement.

	DRHE	LPC
Prediction domain	polar (magnitude, phase)	Cartesian, I and Q independently
Model	fixed IIR filters, coefficients α, β chosen a priori	second-order AR, coefficients fitted per frame
What it assumes	constant magnitude, constant phase increment	the sequence is well described by an order-2 AR process
Causal?	yes — streaming, first output immediate	no — needs the whole frame before any output
State	5 values per (channel, bin)	3.0 Mibit frame buffer
Side information	none	32,768 bits/frame (1.04 %)
Per-sample arithmetic	4 floating-point transcendentals (sqrt , atan2 , sin , cos)	2 fixed-point multiply–accumulates
Entropy back end	identical: S4 + APPEND + fixed Huffman	

3.3.3 Granularity and side information

The model is fitted per (*range bin, receive channel, real/imaginary part*), across all ramps of the frame. That granularity is a deliberate compromise: fitting one model to the whole frame would ignore that different range bins contain targets at different velocities, while fitting per (bin, ramp) would have no data to fit to.

The coefficients are side information: the decoder cannot reproduce them, so they must be transmitted. Two 16-bit values per (bin, channel, part) is

$$128 \times 4 \times 2 \times 2 \times 16 = 32,768 \text{ bits} \quad (3.18)$$

per frame, which is 1.04 % of the uncompressed frame. That is a small but real tax, and Section 6.5 shows a variant where it becomes the dominant cost and sinks the whole approach.

3.3.4 The cost: LPC is not causal

R_0 , R_1 and R_2 are sums over *all* ramps of the frame. The coefficients therefore cannot be known until the frame is complete, and the compressor cannot emit a single bit before then. Where DRHE holds five values per (channel, bin), LPC must buffer the entire frame: $4 \times 192 \times 128 \times 32 = 3,145,728$ bits, 3.0 Mibit.

This is the fundamental trade between the two algorithms, and Section 6.7 argues it matters more than the difference in compression ratio.

3.4 Side-by-side

Table 3.7 summarises. The expectation going in — stated explicitly here so that Chapter 6 can be checked against it — was that LPC would compress slightly better, because a fitted model should beat a fixed one, but would be the expensive design, because it contains a division and DRHE does not. Exactly one half of that expectation survived measurement.

Chapter 4

Methodology

4.1 The verification ladder

A compression algorithm that is *nearly* lossless is useless, so the method has to be able to prove exactness rather than estimate it. The approach taken is a ladder of phases, each one validating the phase above it at a lower level of abstraction. The key property is that each rung has a *reference* it can be checked against, so a discrepancy always localises to one transition.

Phase 1 — MATLAB. The entire radar chain is modelled in double precision, and all eight compression schemes are run against it. This establishes the golden reference: the exact FX16 values, the exact number of bits each algorithm should produce, and the detection metrics.

Phase 2 — HLS C-simulation. Because the hardware design is written in C++, it can be compiled and executed as ordinary software. This phase asks one question: does the C++ produce *exactly* the bits that MATLAB says it should, and does the decompressor recover *exactly* the original samples? It runs in minutes over all 50 frames, which makes it the workhorse of the whole project.

Phase 3 — RTL co-simulation. The C++ is synthesised to Verilog and the same testbench is re-run with the generated RTL substituted for the software. This checks that synthesis preserved behaviour — pipelining, array partitioning and fixed-point inference can all change results if a design is subtly wrong — and returns a cycle-accurate latency figure that no software simulation can give.

Phase 3b — implementation. The same RTL is passed through the complete Vivado flow — logic synthesis, placement and routing — as a standalone block on the target device, with `vitis-run --mode hls --impl`. This replaces the HLS resource and clock *estimates* with the figures of a real netlist: post-route LUT, FF, DSP and BRAM counts per module, the worst timing paths, and a vectorless power estimate. These are the same kind of figure Kiem reports, which is what makes a like-for-like comparison possible (Chapter 7). The phase was added late, after the estimates had been used for several conclusions; where those conclusions changed, the text says so.

Phase 4 — silicon. Out of scope here; it requires the physical board. Resource and clock figures are therefore post-route rather than measured on a running device, and power is a vectorless estimate rather than a measurement, which Section 8.4 restates as a limitation.

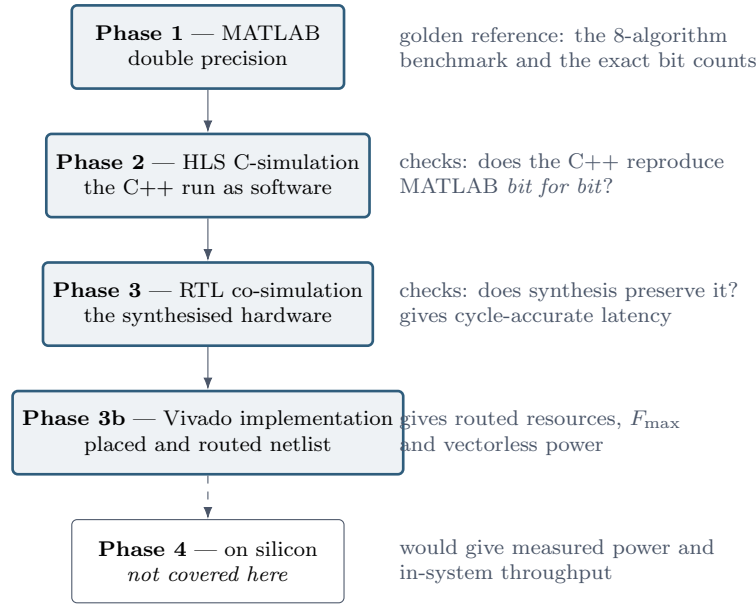


Figure 4.1: The verification ladder. Each phase is checked against the one above it, so a mismatch localises to a single transition rather than to “somewhere in the design”.

4.2 The reference signal chain

The MATLAB chain is described here in the order it executes, because several later observations depend on details of individual steps.

4.2.1 Loading

A frame is read from the ColoRadar binary and reshaped into a complex cube of (fast time $\times$ ramp $\times$ receiver), with the TDM flattening of Equation (2.7). The per-ramp mean is subtracted from each channel to remove the DC / transmitter-leakage term.

4.2.2 Windowing

A Hann window is applied along fast time to suppress spectral leakage, so that a strong nearby target does not bury a weak distant one in sidelobes.

4.2.3 Range FFT

An N_s -point FFT is taken along fast time and normalised by N_s . Only the positive half-spectrum is retained, giving $N = 128$ range bins. This halving is a deliberate choice, not a consequence of conjugate symmetry — the input here is complex I/Q , so the spectrum is *not* symmetric; the negative half corresponds to negative beat frequencies, which are not physically meaningful ranges.¹

4.2.4 FX16 quantisation

The complex FFT output is scaled by Equation (2.11) and rounded to `int16`. This is the last step before compression, and its output — 128 range bins $\times$ 192 ramps $\times$ 4 receive channels of complex `int16` — is exactly what both compressors consume.

¹A code comment in the reference implementation described this as a symmetry consequence. It is not, and the comment has been corrected.

Four receive channels rather than sixteen are exported to the hardware path, because four complex `int16` samples is 128 bits, the AXI4-Stream width. Section 8.4 notes the consequence for comparisons against the 16-channel MATLAB benchmark.

4.2.5 Detection and metrics

The reconstructed data is put through Doppler and angle processing and a threshold detector, and the resulting detections are compared against those from the double-precision reference. The threshold is the mean of the lowest 75 % of sorted magnitudes plus a fixed 15 dB offset, with 8-connected local-maximum suppression.

4.3 Metrics, defined precisely

Several metric names are used loosely in this field. They are pinned down here.

4.3.1 Compression ratio

$$\text{CR} = \frac{B_{\text{in}}}{B_{\text{out}}}, \quad B_{\text{in}} = N_{\text{ramps}} \times N \times N_{\text{RX}} \times 32, \quad (4.1)$$

with B_{out} the compressed length *including* AXI packet padding. Including the padding matters: a design that flushed its bit buffer per ramp rather than per frame would waste up to 255 bits per ramp, costing about 2.9 % of the ratio. Reporting the ratio without padding would hide that design choice.

4.3.2 Maximum absolute difference

$$\text{MaxDiff} = \max_{\text{all samples}} |x - \tilde{x}| \quad (4.2)$$

over every real and imaginary component. For a lossless coder the pass criterion is $\text{MaxDiff} = 0$. There is no tolerance.

4.3.3 Detection metrics, and one correction

Let D_{ref} be the detection set from the double-precision reference and D_{test} that from the reconstructed data. The two rates reported are

$$\text{FN}\% = \frac{|D_{\text{ref}} \setminus D_{\text{test}}|}{|D_{\text{ref}}|}, \quad \text{FP}\% = \frac{|D_{\text{test}} \setminus D_{\text{ref}}|}{|D_{\text{test}}|}. \quad (4.3)$$

The first is a false negative rate in the ordinary sense. The second is **not** a false positive rate, despite the label it carries in the literature and in this author's own earlier reporting. A false positive rate normalises by the number of true negatives — in a range–Doppler map, the number of empty cells. This quantity normalises by the number of *test detections*, which makes it a **false discovery rate**. It is a perfectly legitimate metric, and arguably the more useful one here, but it is not what its conventional name denotes. It is reported under its correct name throughout this thesis.

Two further properties bound how much weight the detection metrics can carry, and are stated rather than buried:

- **The comparison is cell-wise and exact.** A target detected one range–Doppler cell away from its reference position counts as both a false negative *and* a false discovery. There is no positional tolerance, which inflates both rates relative to a matched-detection criterion.
- **The 15 dB threshold is a free parameter.** It is not swept, and the detection rates are sensitive to it.

Table 4.1: The synthetic edge-case suite. Each case has its own allowed reconstruction error — zero for all but the two that exercise the saturation described in Section 5.4.1.

Case	What it exercises	Allowed MaxDiff
all-zeros	the $S_4 = 0$ path and the best case	0
dc-constant	a perfectly predictable sequence	0
random-int16	the worst case: incompressible input	0
tone-192ramp	a periodic sequence the AR model should fit	0
tone-1ramp	a sequence too short to fit	0
max-amplitude	full-scale values, and 128-bit correlations	0
max-alternating	± 32767 alternating: the largest residuals	1
int16-min	-32768 specifically	1

4.3.4 What the detection metrics actually measure here

This point is important enough to state on its own. FX16, DRHE and LPC are all exactly lossless *with respect to the FX16 codes*. Their reconstructed data is bit-identical, so their detection metrics are identical *by construction* — not by coincidence, and not as evidence that the compressors are equally good.

What the detection metrics measure in this thesis is therefore the cost of the **FX16 quantisation step**, which is the only lossy operation in the lossless chain. They are reported for that purpose, and to provide the contrast against the genuinely lossy BAQ.

4.3.5 Order-0 entropy

To judge the entropy coder rather than the whole compressor, Equation (2.9) is evaluated empirically on the residual distribution. The gap between the entropy and the achieved bits per value is the coder’s inefficiency; the entropy itself is the figure of merit for the *predictor*.

This is the measurement that exposed the central finding, and it is worth noting that it is not a standard part of a compression evaluation. Had it not been made, the prediction-axis error would have remained invisible behind a respectable-looking compression ratio.

4.4 Edge-case testing

Real data exercises the typical case well and the extreme case not at all. The ColoRadar residuals span roughly $[-514, +416]$, which is entirely inside category 10 of a 16-category code. Everything above that was, until deliberately tested, unexercised.

An eight-case synthetic testbench was therefore built (Table 4.1), generating its stimulus internally so that it runs identically in C-simulation and co-simulation.

Both defects reported in Section 5.4 were found by this suite and by nothing else. Neither is reachable with real radar data. The general lesson — that dataset-driven verification and adversarial synthetic verification find different classes of bug and neither substitutes for the other — is returned to in Section 8.5.

4.5 Toolchain and reproducibility

All MATLAB work used R2024b with the Signal Processing Toolbox. All hardware work used AMD Vitis HLS 2026.1 targeting `xcku5p-ffvb676-2-e` at a 100 MHz clock, with XSIM for co-simulation and Vivado 2026.1 for out-of-context synthesis, placement and routing (`vitis-run --mode hls --impl`). The replica of Kiem’s design and every ablation step (Chapter 7) use the same tool, part and constraints.

Two practical points are recorded because they cost time. First, Vitis refuses to create a project under a path containing a space, so the build tree must live somewhere like `D:\Thesis` rather than under a user directory whose name contains one. Second, the stimulus for the hardware testbenches is exported *from MATLAB as `int16`* rather than recomputed in C++. This eliminates any possibility of a floating-point FFT discrepancy between the two environments: the C++ testbench and the MATLAB reference operate on byte-identical input, so any difference in output is unambiguously a difference in the compressor.

Chapter 5

Implementation

5.1 Common infrastructure

Both compressors present the same interface: AXI4-Stream in at 128 bits per beat (four channels of 16-bit I and Q), AXI4-Stream out at 256 bits per beat of compressed bitstream, with the frame dimensions (N , N_{ramps} , N_{RX} , and for the corrected variants n_{TX}) on an AXI4-Lite control bundle.

Both share the bit-packing arrangement. Codewords accumulate into a 512-bit register, least-significant-bit first, and a 256-bit AXI word is flushed whenever at least 256 bits are available, with a final partial flush at end of frame. Flushing per frame rather than per ramp is what avoids the 2.9 % padding penalty noted in Section 4.3.

The Huffman codewords are stored bit-reversed relative to the published dictionary, because the packer is LSB-first. The reversed code set remains prefix-free, so the decoder can match candidates from the low end of its buffer without ambiguity — at most one table entry can match.

5.2 DRHE architecture

5.2.1 State and parallelism

DRHE holds five `float` arrays of shape $[N_{\text{RX}}][N]$: the magnitude prediction, the previous magnitude, the phase prediction, and the previous two phases. The channel dimension is completely partitioned, so the four receive channels are four parallel copies of the datapath inside one pipelined iteration of the sample loop — the parallelism argument of Section 2.7 made concrete.

State is zeroed on the first ramp of each frame. This makes frames independent: a frame lost in transit cannot corrupt its successors, which is a desirable property for a streaming sensor and one that the 50-frame test confirms directly.

5.2.2 Why it is expensive

Per sample, per channel, DRHE evaluates `sqrt` and `atan2` to get polar coordinates, then `sin` and `cos` to get back. Four floating-point transcendental functions, four times over for four channels, on every one of 24,576 beats per frame.

Chapter 7 shows the bill after implementation: the floating-point operators are 69 % of the design’s 35,343 LUTs and every one of its 132 DSP slices, and the two arctangent units alone are a third of the logic. They do not stop the design meeting its 100 MHz target, which the routed design passes with 2.621 ns of slack, although the HLS estimate said it would not. Replacing them with fixed-point arithmetic is the largest saving available, and Chapter 7 measures it.

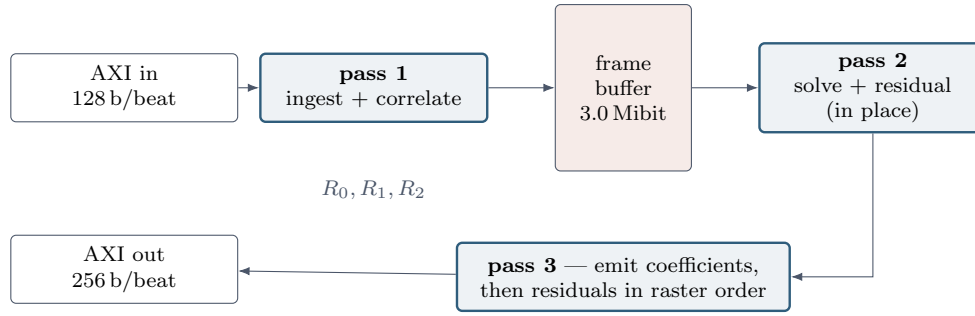


Figure 5.1: The LPC compressor. Because the Yule–Walker coefficients depend on the whole frame, the compressor must buffer it; the buffer is then rewritten in place with residuals so that only one memory is needed. Emitting residuals in the original raster order is what lets the *decompressor* avoid a frame buffer entirely.

5.2.3 The initiation-interval bottleneck

DRHE achieves $II = 2$ rather than the target $II = 1$, and the reason is instructive because it has nothing to do with arithmetic.

Each state array is partitioned on the channel dimension, so each channel’s $[N]$ history is one dual-port block RAM. Every iteration reads a value from it and writes the updated value back — two accesses, which two ports can just manage. But on the first ramp the iteration *also* writes a reset value. Three accesses against two ports forces the tool to serialise, and the loop settles at $II = 2$ for the whole frame on account of a condition that is true for one ramp in 192.

The fix is straightforward: read the state as zero on the first ramp instead of writing zeros to it, so each array sees one read and one write per cycle. It was implemented as step 3 of the ablation in Chapter 7, where it reaches $II = 1$ and raises measured throughput by 71 %, at the cost of doubling the floating-point arithmetic. It is not adopted in DRHE-1 itself, because the sensor does not need the throughput.

5.3 LPC architecture

5.3.1 Three passes

LPC cannot stream, so its structure is explicitly multi-pass (Figure 5.1):

1. **Ingest.** Read the frame into the buffer, accumulating R_0, R_1, R_2 per (channel, bin) as the samples arrive.
2. **Solve and residual.** For each (channel, bin), solve Equation (3.16), then walk the ramps again computing residuals and rewriting the buffer *in place*.
3. **Emit.** Write the coefficient table, then the residuals.

Rewriting the buffer in place matters: it means one 3.0 Mibit memory serves both purposes rather than two.

5.3.2 A useful asymmetry

Pass 3 emits the coefficient table first — the decoder needs all of it before it can reconstruct anything — and then the residuals *in the same raster order the samples arrived in*, rather than in the (channel, bin, ramp) order pass 2 produced them.

The payoff is that the decompressor needs no frame buffer at all. It holds the coefficient table and two reconstructed samples per (channel, bin) — the same *shape* of state DRHE

uses — and writes each output sample as its residual arrives. The reordering cost is paid once, by the side that already had to buffer.

This is worth noting because it inverts the usual expectation. LPC’s non-causality is a compressor-side problem only; at the receiving SoC, where memory is comparatively plentiful but latency still matters, LPC decompression is as cheap as DRHE’s.

5.3.3 Fixed-point choices

Coefficients are quantised to `ap_fixed<16,2>` for a_1 (range $[-2, 2)$) and `ap_fixed<16,1>` for a_2 (range $[-1, 1)$), which follows from the stability conditions of Equation (3.17). Measured over 50 frames, this quantisation costs 0.0000 % of compression ratio relative to double-precision coefficients — the residuals are unchanged to the bit. The fitted models are evidently not sensitive at that level, which retrospectively justifies the 16-bit side-information budget.

The autocorrelations are accumulated as exact 64-bit integers rather than in floating point. This is deliberate: $R_0^2 - R_1^2$ is a difference of two nearly equal quantities whenever the data is strongly correlated — precisely the regime the predictor exists for — so computing it in floating point would lose most of its significant digits exactly when accuracy matters most. Forming all three 128-bit quantities exactly in integer arithmetic before any division avoids the cancellation entirely.

5.4 Two defects found and fixed

Both defects described here share a pattern worth naming: each was invisible on real data, each was found by a synthetic edge case, and each would have shipped.

5.4.1 Defect 1: the -32768 residual

Symptom. The `max-alternating` edge case — input alternating ± 32767 — produced `MaxDiff = 65,535`. That number is diagnostic: it is exactly the distance from -32768 to $+32767$, which points straight at the representability hole of Section 3.1.12. Instrumenting the encoder confirmed it:

```
MISMATCH v=-32768 s4=15 append=0x7FFF decoded=32767
```

The three possible fixes.

1. **Extend the code with a category 16.** Correct in general, but it changes the bitstream format and breaks compatibility with the reference design’s dictionary.
2. **Reject the input.** Legitimate for a closed system but unhelpful for a general IP core.
3. **Saturate -32768 to -32767 .** Costs at most one LSB on the affected sample and leaves the format untouched.

Option 3 was chosen. On real data the clamp never fires, since residuals span roughly $[-514, +416]$, so behaviour on the dataset is unchanged.

Why saturation alone is not enough. Saturation makes the reconstruction differ from the original by one LSB, and that is where Section 2.4.4 becomes concrete. The encoder originally derived its next prediction from the *original* sample, while the decoder necessarily derives it from the *reconstructed* one. These agree only while reconstruction is exact. The moment the clamp fires they diverge — and because Equations (3.10) and (3.11) are IIR filters, that divergence is then carried forward through every remaining ramp of the frame, growing rather than decaying.

The complete fix is therefore to close the loop. The encoder reconstructs exactly what the decoder will produce and drives its state from that:

```
int diff_re = wrap_int16(re - pred_re);
if (diff_re == -32768) diff_re = -32767;      // saturate
int recon_re = wrap_int16(diff_re + pred_re); // what the decoder will see
float curr_mag  = hls::sqrt(recon_re*recon_re + recon_im*recon_im);
float curr_phase = hls::atan2(recon_im, recon_re); // state from recon
```

The cost. Where reconstruction is exact — which is everywhere on real data — the closed-loop and open-loop forms are identical by construction, so compression is unchanged. The cost falls entirely on synthesis: the reconstruction now sits on the dependency chain ahead of the polar conversion, so pipeline depth rises from 32 to 58 cycles and estimated registers from 40,309 to 57,105. Initiation interval is unaffected. (Implemented, the fixed-point version of the same closed loop costs four cycles and a few hundred flip-flops; Section 7.5.5.)

That is a real price — Section 7.5.3 shows that it is 26 of the 58 cycles of pipeline depth — and it is the direct cost of bounding an error that cannot occur on this dataset but could occur on another. Option 1 above would avoid it, at the price of bitstream compatibility.

5.4.2 Defect 2: 128-bit to float conversion

Symptom. The `max-amplitude` edge case produced wrong coefficients. Tracing showed a determinant of 4.4×10^{20} converting to -1.2×10^{18} — not merely imprecise but the wrong sign.

Cause. Equation (3.16) needs products of 64-bit correlations, so the numerators and the determinant are computed as `ap_int<128>`. Converting such a value directly to `float` is not correct above 2^{64} in this toolchain: `(float)(1 << 70)` evaluates to zero.

Fix. Normalise all three operands by a common arithmetic right shift into 62 bits before the division, choosing the shift from the largest of the three:

$$k = \max(\text{msb}|D|, \text{msb}|N_1|, \text{msb}|N_2|) - 61, \quad (5.1)$$

clamped at zero. Because the same shift is applied to numerator and denominator, the quotient is mathematically unchanged; only the intermediate representation is brought into a range the conversion handles.

Why it was invisible. On real ColoRadar data the correlations reach only 44 bits, comfortably inside the range where the conversion is correct. The defect is unreachable with this dataset and would have been unreachable with any similarly scaled one. It is, in other words, precisely the class of bug that a dataset-driven test plan cannot find.

5.5 The TDM-aware variants

Chapter 6 motivates these; this section describes what they are, since they are implementation rather than result.

5.5.1 DRHE

The change is confined to state indexing. The five state arrays gain a transmitter dimension, indexed by $m \bmod n_{\text{Tx}}$, and the reset condition widens from $m = 0$ to $m < n_{\text{Tx}}$, because each transmitter’s history begins on its own first ramp rather than on the frame’s. The prediction equations, the coefficients, the residual computation, the entropy coder and the bitstream format are all untouched.

The state grows by a factor n_{Tx} in principle. In practice the baseline was dimensioned for up to 1,024 range bins and the corrected version for the 128 actually used, so the realised growth is $1.5\times$: from 40 to 80 block RAMs.

Crucially, the *schedule* does not change. Indexing the state by transmitter changes which address is read, not when, so the initiation interval, the pipeline depth and the frame latency are all identical. The correction is therefore close to free.

5.5.2 LPC, and a ring buffer trick

LPC needs more work, because both the autocorrelation and the residual pass now need $x[m - n_{\text{Tx}}]$ and $x[m - 2n_{\text{Tx}}]$ rather than $x[m - 1]$ and $x[m - 2]$.

A naive shift register of depth $2n_{\text{Tx}} = 24$ would shift 24 entries every iteration. Instead, observe that a *ring* of exactly $2n_{\text{Tx}}$ entries gives both taps for free. At ramp m , let $k = m \bmod 2n_{\text{Tx}}$. Slot k still holds the value written at ramp $m - 2n_{\text{Tx}}$, because it is about to be overwritten and has not been touched since; and slot $(k + n_{\text{Tx}}) \bmod 2n_{\text{Tx}}$ holds the value from ramp $m - n_{\text{Tx}}$, since that is the most recent ramp congruent to $k + n_{\text{Tx}}$. So:

read $x[m - 2n_{\text{Tx}}]$	from slot k
read $x[m - n_{\text{Tx}}]$	from slot $(k + n_{\text{Tx}}) \bmod 2n_{\text{Tx}}$
write $x[m]$	to slot k

Nothing is shifted; the cost is $2n_{\text{Tx}}$ registers and two multiplexers.

5.5.3 A restructure that pays for itself

Accumulating the autocorrelations during ingest is no longer convenient at the deeper lags, so they were moved into the per-bin pass instead, computed from the frame buffer through the ring above. This deleted six 64-bit accumulator arrays and four history arrays outright.

The consequences, quantified in Chapter 6, are mixed and interesting: block RAM usage *falls* (by 25% in the HLS allocation, 11.5% after routing), and the ingest loop — previously stuck at $\text{II} = 3$ on accumulator port pressure — now meets $\text{II} = 1$, as does every other loop in the design. Against that, the design now makes two short passes over each (channel, bin) instead of one long one, and Section 6.6 shows that the pipeline drain this incurs costs considerably more than a naive cycle model predicts.

5.5.4 Both baselines retained

The original DRHE and LPC designs were kept alongside the corrected ones rather than replaced. This is deliberate: it allows a like-for-like comparison at every level, and it means the results in Chapter 6 are differences measured on the same machine with the same toolchain on the same day, not comparisons against remembered numbers.

Chapter 6

Results

This chapter reports what was measured. It is arranged so that the story unfolds in the order it actually did: first the benchmark and the verification, which all passed and all looked satisfactory; then the audit that asked what the numbers meant and found they did not mean what they appeared to; then the correction and its consequences.

6.1 Phase 1: the MATLAB benchmark

6.1.1 Eight algorithms on real data

Table 6.1 gives the 50-frame average over 9,934 reference detections, using all 16 receive channels. The reference against which everything is compared is the double-precision path, so FP16 — half-precision floating point — registers as effectively lossless.

Three things are worth drawing out.

The lossless coders share FX16’s detection metrics exactly. FX16, DRHE and LPC report identical FN%, FP%, noise floor and SNR. As Section 4.3.3 explained, this is not a coincidence but a tautology: all three reconstruct the FX16 codes bit-exactly, so the detector sees identical data. What the table is really showing in those rows is the cost of *quantisation*, which is 3.77 % of detections missed and 1.11 % of reported detections spurious.

The lossy comparison is instructive. BAQ reaches a nominally better ratio — 3.76 against 3.30 — but misses 10.44 % of detections instead of 3.77 %, and 9.69 % of what it does report is spurious against 1.11 %. That is nearly three times the miss rate and nearly nine times the false discovery rate, for a 14 % improvement in ratio. In a system that brakes the car, that is not a trade anyone should take. This is the empirical form of the argument made on principle in Section 1.3.

The predictive coders reach the target. DRHE’s 3.30 matches the 3.2 reported for the baseline design [11], which at the time was taken as confirmation that the implementation was correct. Section 6.4 revisits that interpretation.

6.2 Phase 2: bit-exactness

6.2.1 The headline

Over 50 frames — 4,915,200 complex range-FFT samples per design, or 157,286,400 input bits — both HLS implementations reconstruct every sample of every frame exactly: MaxDiff = 0,

Table 6.1: Phase 1: eight compression schemes, 50-frame average, 16 receive channels, 9,934 reference detections. FP% is a false discovery rate (Section 4.3.3). NF is the noise floor.

Algorithm	CR	FN%	FP%	NF (dBFS)	SNR (dB)
FP16	1.00	0.00	0.00	−105.638	35.504
DRHEfp16	1.04	0.05	0.01	−105.637	35.505
FX16	1.00	3.77	1.11	−105.374	35.359
DRHE	3.30	3.77	1.11	−105.374	35.359
BAQ	3.76	10.44	9.69	−105.080	35.081
RDVLE	1.12	24.01	4.62	−102.275	33.165
RDVLE_UDRE	2.56	28.73	4.44	−101.782	33.004
LPC	3.37	3.77	1.11	−105.374	35.359

Table 6.2: LPC, mean compression ratio over 50 frames, computed four ways. The padding model reproduces the hardware result on every frame, so the HLS bit counts are bit-exact with MATLAB.

Quantity	Value
MATLAB, double-precision coefficients	3.38541
MATLAB, 16-bit quantised coefficients	3.38541
(quantisation cost)	0.0000 %
MATLAB, quantised + 256-bit AXI padding	3.38497
Vitis HLS, measured	3.38497
Frames where the padding model matches	50 / 50
Worst-frame difference	0.00091 (tolerance ± 0.01)

mean squared error = 0, signal-to-noise ratio infinite. Compression ratios are 3.30987 for DRHE and 3.38497 for LPC.

6.2.2 How closely the hardware matches MATLAB

Agreement with the MATLAB reference is an order of magnitude tighter than the ± 0.01 criterion set in advance.

For LPC the agreement is exact. A MATLAB model that charges the same 256-bit AXI padding reproduces the Vitis bit count on 50 of 50 frames to within 4.9×10^{-6} — which means the entire discrepancy between the two is packet padding and the underlying bit counts are identical.

For DRHE the hardware bit count exceeds the MATLAB count by between 15 and 252 bits per frame, mean 134. That range sits strictly inside one 256-bit packet, which is exactly what a per-frame flush should produce, so again the residual difference is padding rather than disagreement.

6.2.3 Edge cases

All eight synthetic cases of Table 4.1 pass for both coders after the fixes of Section 5.4. Two results are worth recording because they were initially misread as failures:

- **random-int16 compresses to 0.60** — that is, it *expands* the data by 67%. This is correct behaviour, not a bug. Incompressible input has no structure to remove, and every residual lands in a high category costing a 12-bit codeword plus 15 APPEND bits. Every lossless coder expands incompressible input; it is a theorem that some input must.
- **tone-192ramp reaches only 1.10 under LPC**. The synthetic tone’s period is about 209 ramps, longer than the 192-ramp frame, so the autocorrelation estimate is poor and the

Table 6.3: Resources and timing of the two Phase 3 designs, `xcku5p-ffvb676-2-e` at a 10 ns target. “HLS” is the post-synthesis estimate; “routed” is the Vivado post-route figure for the same RTL. The device has 216,960 LUT, 433,920 FF, 960 BRAM_18K and 1,824 DSP. Comparison with Kiem’s design is deferred to Chapter 7, where it is made on post-route figures only.

Resource	DRHE-1		LPC-1,2	
	HLS	routed	HLS	routed
LUT	115,534	35,343	80,528	29,772
FF	57,105	23,546	17,393	13,601
BRAM_18K	40	40	256	244
DSP	132	132	168	168
$F_{\max}$ (MHz)	81.88	135.5	85.43	118.6
Frame latency (co-sim)	61,253 cycles		197,277 cycles	
Worst loop II	2		3	
Frame buffer	none		3.0 Mibit	

fitted coefficients are $a_1 = 1.0377$, $a_2 = -0.0484$ against the ideal $a_1 = 2 \cos \omega = 1.9991$, $a_2 = -1$. MATLAB’s solver produces the same coefficients, and its stability test does not fire either, so this is a property of the second-order Yule–Walker estimator on a truncated period, not an implementation fault.

6.3 Phase 3: synthesis and co-simulation

6.3.1 Resources

Table 6.3 gives the resources of both compressors on the `xcku5p`. When Phase 3 was run, only the left-hand number in each pair existed: the estimate that HLS makes before any circuit has been built. The right-hand number, measured after Vivado had placed and routed each design, was added later (Chapter 7). Both are kept, because two of the three conclusions that this section first drew from the estimates did not survive the measurement, and the way they failed is itself a result.

6.3.2 Timing: the estimate said no, the implementation said yes

HLS estimated both designs below 100 MHz: 81.88 MHz for DRHE and 85.43 MHz for LPC. This section originally concluded that neither design met timing, and recommended pipelining the floating-point datapath, or replacing the polar conversion with a fixed-point CORDIC, to fix it.

After placement and routing, both designs meet 100 MHz: DRHE-1 closes at 135.5 MHz and LPC-1,2 at 118.6 MHz. The HLS figure is the tool’s own model of operator delays against a budget that reserves 27 % of the period for clock uncertainty; it is not a timing analysis of the real circuit (Section 7.1). There was no timing problem to fix.

6.3.3 The expectation that did not survive, twice

Chapter 3 recorded the prior expectation that LPC would be the expensive design, because it contains a division and DRHE does not. The HLS estimates contradicted it: LPC was estimated at 30 % fewer LUTs and 70 % fewer flip-flops. The explanation offered was that per-sample cost dominates. LPC’s inner loop is two fixed-point multiply–accumulates, while DRHE’s is four floating-point transcendentals, and the division runs only 1,024 times per frame, contributing 512 of 197,277 cycles.

The routed figures keep the reason but shrink the difference: the LUT saving roughly halves and the flip-flop saving falls from 70 to 42 %. After implementation LPC-1,2 uses 0.84 times DRHE-1's LUTs and 0.58 times its flip-flops. The difference shrank because HLS over-estimated DRHE more than LPC: it costed DRHE's arctangent units and loop body at three to six times their implemented size (Section 7.1), and LPC's logic at 2.7 times. For the corrected TDM variants the two algorithms cost almost exactly the same logic (Section 7.8). The division is still not the bottleneck: in LPC- $n,2n$ the four integer dividers are 17 % of the LUTs, and they come from a modulo in the address arithmetic, not from the algorithm's single coefficient division.

What LPC does cost is *memory and latency*. Its frame buffer accounts for 240 of its 256 estimated block RAMs, and the frame must be complete before the first output bit exists. The real trade between the two algorithms is **memory and latency, against very little in logic**, which was not the trade either was expected to present.

6.3.4 Where LPC's cycles go

The cycle count decomposes exactly, which is a useful check that nothing unexpected is happening:

Stage	Iterations	II	Cycles
Ingest	24,576	3	73,728
Solve + residual	98,304	1	98,304
Emit coefficients	512	1	512
Emit residuals	24,576	1	24,576
Model total			197,120
Measured (co-simulation)			197,277

The model is accurate to 0.08 %. Two inefficiencies are visible and neither is algorithmic: the ingest loop sits at $II = 3$ on accumulator port pressure where 24,576 cycles should do, and the residual pass handles one channel at a time where ingest and emit handle all four. Fixing both would bring LPC to roughly 74,000 cycles, about 21 % more than DRHE, without touching the algorithm.

At this point every test had passed, both designs were verified bit-exact on real data, and the compression ratios matched the literature. The obvious conclusion was that the work was done.

6.4 The audit: what the compression ratio actually measures

6.4.1 The question

A compression ratio of 3.3 on radar data invites a particular reading: that the predictor is removing about 70 % of the information, and that the Doppler redundancy the algorithm was designed around is really there and really being exploited.

That reading deserves a test, and the test is simple. Both compressors use the same entropy coder. So: code the FX16 output through that coder with *no prediction at all*, and see what ratio survives. Whatever survives is attributable to the entropy coder and to the statistics of the data — not to prediction.

6.4.2 The result

Table 6.4 is the pivot of this thesis. Coding the data with *no predictor at all* compresses **better** than either predictor. DRHE costs 4.9 % more bits than doing nothing; LPC costs

Table 6.4: What happens when the predictor is removed. “No prediction” codes the FX16 values directly through the same S4 + Huffman back end. Five frames, four receive channels. LPC includes its coefficient overhead.

Configuration	bits/value	CR	vs. baseline
No prediction (entropy coder only)	4.586	3.489	—
DRHE, lag 1 (as implemented)	4.821	3.319	0.951×
LPC, lags 1 & 2 (as implemented)	4.710	3.397	0.974×

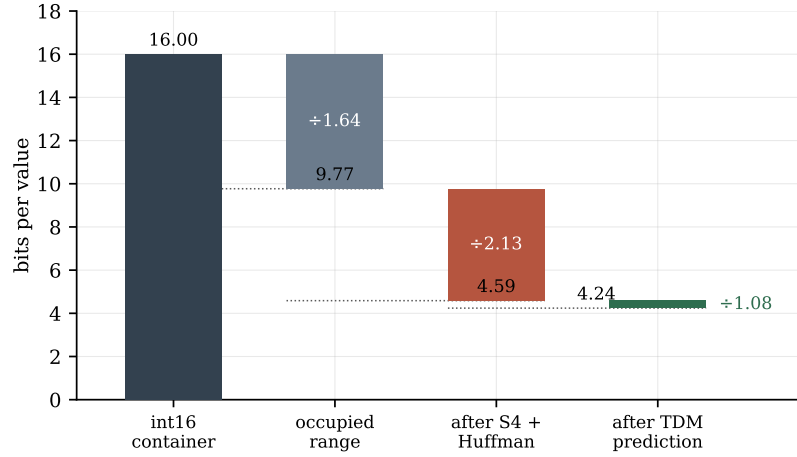


Figure 6.1: Where the compression ratio comes from. Of the 16 bits in the `int16` container, 6.2 are structurally unused because the capture sits far below full scale; the entropy coder recovers a further factor of 2.13 from spectral sparsity; and prediction *on the corrected axis* contributes the remaining 1.08. Prediction on the original ramp axis contributes nothing at all — it costs 0.24 bits per value.

2.6 % more.

Order-0 entropy confirms the direction independently of the coder:

Quantity	Value
Order-0 entropy of raw FX16 codes	4.094 bits
Order-0 entropy of DRHE residuals	4.538 bits
Achieved bits/value (no prediction)	4.586 bits
Huffman overhead above entropy	0.283 bits (6.2 %)

The predictor is *increasing* the entropy of the data it is supposed to be decorrelating. That is not a marginal inefficiency; it is a predictor that is actively wrong.

Note also that the entropy coder itself is doing fine: 4.586 bits achieved against 4.094 bits of entropy is an overhead of 6.2 %, which for a fixed dictionary designed for a different dataset is a good result. The problem is not in the second stage.

6.4.3 Where the ratio really comes from

If not prediction, then what? Two places, neither of which is Doppler redundancy (Figure 6.1).

Unused container headroom: a factor of 1.63. Equation (2.11) calibrates the FX16 scale factor to a theoretical full-scale ADC sinusoid. The ColoRadar capture sits far below full scale: the peak raw ADC sample is 914 out of a possible 32,767. After windowing, transform and scaling, the peak `int16` FFT code is 436. Only $\log_2(436) + 1 = 9.8$ of the 16 container bits ever carry information; the remaining 6.2 are zero in every single sample. An entropy

Table 6.5: Prediction axis study. Five frames, four receive channels, 128 bins, 192 ramps. Prediction gain is relative to no prediction at all; a gain below 1.0 means the predictor is making things worse.

Configuration	bits/value	CR	gain
No prediction	4.586	3.489	1.000
DRHE, lag 1 (<i>as implemented</i>)	4.821	3.319	0.951
DRHE, lag n_{Tx}	4.240	3.774	1.082
LPC, lags 1, 2 (<i>as implemented</i>)	4.710	3.397	0.974
LPC, per-transmitter blocks, $N = 16$	6.093	2.626	0.753
LPC, lags $n_{Tx}, 2n_{Tx}$	4.269	3.748	1.074

coder removes them whether or not any prediction happens, and that alone is a factor of $16/9.8 = 1.63$.

Spectral sparsity: a factor of 2.14. The remaining $9.8 \rightarrow 4.59$ bits is the genuine statistical gain, and it is a real property of radar data: a range-FFT frame is sparse. Most bins contain noise near zero and a few contain targets, so the S_4 categories concentrate heavily at the low end. This is what the entropy coder is for and it does it well.

Prediction: nothing, as implemented. The third factor should have been the predictor’s contribution. As implemented it was negative.

Consequence for reporting. The compression ratios in Table 6.1 — and, by the same argument, those reported elsewhere in this field — are substantially a property of the *sensor’s gain staging* rather than of the algorithm. A capture with 6 dB more gain would compress measurably less on the same algorithm. A no-prediction baseline should be reported alongside any such figure, and is not standard practice.

6.5 The prediction axis

6.5.1 Diagnosis

The inversion in Table 6.4 is not a property of the predictors — they are both sound algorithms — but of the *axis* they were applied along. Section 2.2.3 set this up: both were predicting from ramp $m - 1$, which by Equation (2.7) is a different physical transmitter eleven times in twelve.

The mechanism is now concrete. Two ramps from different transmitters differ by the target’s array steering phase, Equation (2.6), which is large and varies with target bearing. DRHE’s phase model, Equation (3.11), is a double integrator that predicts perfectly when the phase increment is constant; a per-ramp transmitter change makes that increment alternate on a period-12 pattern, which no double integrator can track. LPC’s autocorrelations at lags 1 and 2 likewise measure the *transmitter switching pattern* rather than target Doppler, and the fitted coefficients describe the former.

6.5.2 Testing the hypothesis

Six configurations were evaluated over five frames (Table 6.5).

Predicting on the true slow-time axis converts DRHE’s 4.9 % loss into an 8.2 % gain, and LPC’s 2.6 % loss into a 7.4 % gain. The hypothesis is confirmed.

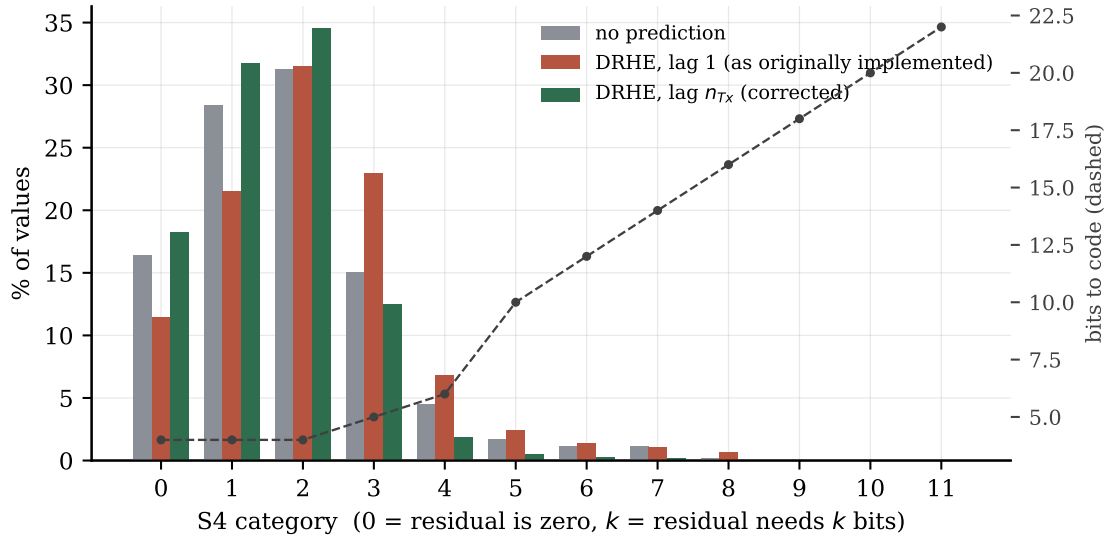


Figure 6.2: Distribution of residuals across S_4 categories, with the cost of each category (Equation (3.7)) on the right axis. Lag-1 DRHE (red) pushes mass *up* the category scale relative to not predicting at all (grey) — into categories 3 and 4 which cost 5 and 6 bits. Lag- n_{Tx} DRHE (green) pulls it back down into categories 0–2, which cost 4 bits or fewer. The steepness of the cost curve above category 4 is what makes this shift so consequential.

6.5.3 The variant that does not work, and why

Row five is worth dwelling on because it is the obvious alternative and it fails badly.

Splitting the frame into twelve independent 16-ramp per-transmitter blocks and fitting separate coefficients to each *does* put the predictor on the right axis. But it multiplies the coefficient side information by twelve — from 1.04 % to 12.5 % of the uncompressed frame — and it shortens each autocorrelation estimate from 192 samples to 16, which makes the fitted coefficients much noisier. The net result, CR 2.626, is far worse than doing nothing at all.

Keeping *one* coefficient pair per (bin, channel, part) and simply moving the lags to n_{Tx} and $2n_{Tx}$ costs no extra side information whatsoever, which is why the last row works and the fifth does not. The lesson generalises: on a TDM-MIMO radar, the right change is to the *lag*, not to the *partitioning*.

6.5.4 What the residuals look like

The bits-per-value figures are a summary; the underlying change in the residual distribution is more revealing.

Figures 6.2 and 6.3 show what happened. Lag-1 prediction *widens* the residual distribution: the 99th percentile of $|\text{residual}|$ goes from 80.6 with no prediction to 102.4 with DRHE. Predicting at lag n_{Tx} collapses it to 15.8, and the corrected LPC reaches 8.2 — roughly a tenfold reduction in tail magnitude against the original design.

The category histogram tells the same story in the currency the coder actually charges in. Lag-1 DRHE moves 23.0 % of residuals into category 3 and 6.9 % into category 4, which cost 5 and 6 bits; the corrected version puts 84.6 % of residuals into categories 0–2, all of which cost 4 bits or fewer.

6.6 The corrected implementations

The corrections described in Section 5.5 were implemented in HLS and put through the identical verification ladder as the baselines: 50-frame C-simulation, synthesis, and RTL

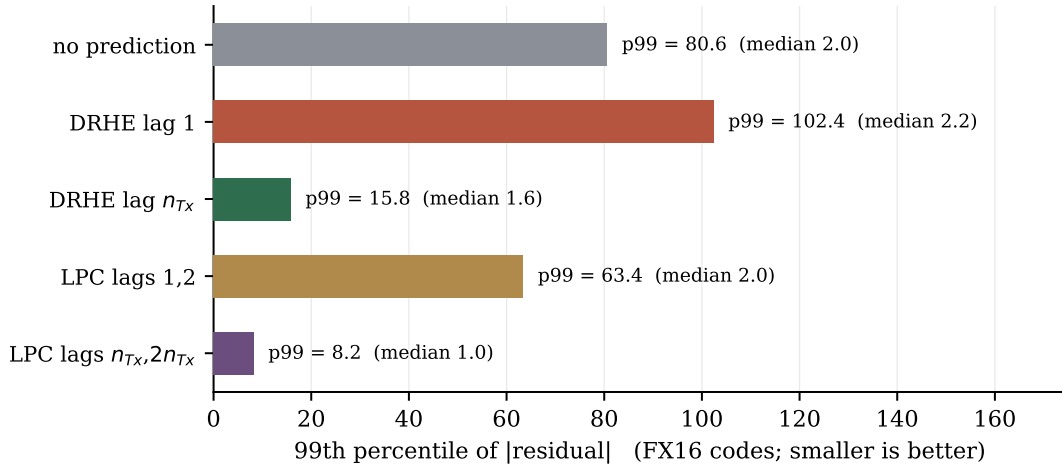


Figure 6.3: The 99th percentile of residual magnitude. This is the clearest single view of the effect: lag-1 DRHE produces a heavier tail than no prediction at all (102.4 against 80.6), while lag- n_{Tx} DRHE cuts it to 15.8 — a factor of 6.5 better than the design it replaces. LPC at lags $n_{Tx}, 2n_{Tx}$ reaches 8.2.

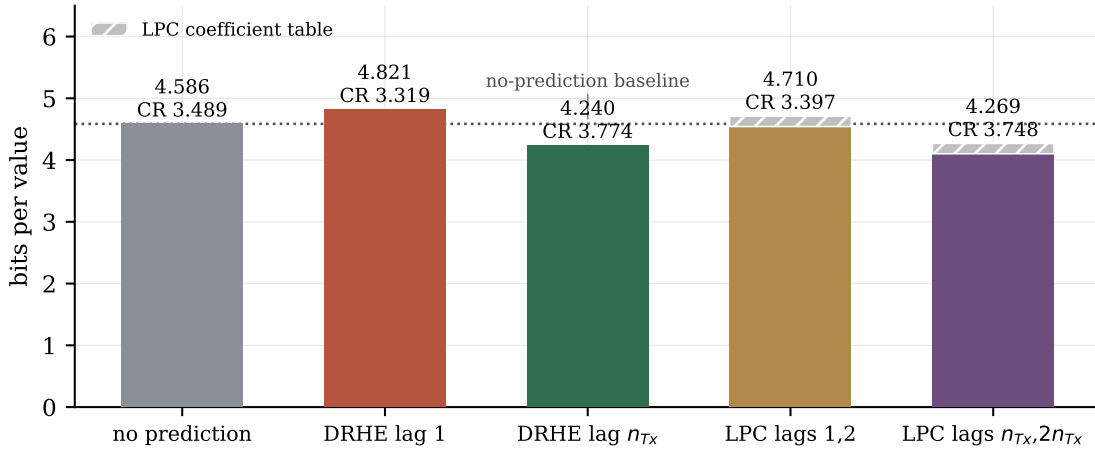


Figure 6.4: Bits per value for all five configurations, with the LPC coefficient table shown separately as it is charged whatever the residuals do. The dotted line is the no-prediction baseline: any bar above it is a predictor that is doing harm.

co-simulation.

6.6.1 Compression, measured in hardware

The measured 3.75082 for the corrected DRHE agrees with the 3.774 predicted by the five-frame MATLAB study of Table 6.5, to within the frame-to-frame spread. That agreement between an independent MATLAB prediction and a hardware measurement is the cross-check that the correction is real rather than an artefact of either environment.

Figure 6.5 shows the per-frame detail. The improvement is uniform: there is no frame on which the corrected design does worse, and the shape of the variation across the sequence is preserved, which is what one would expect if the correction improves prediction rather than merely shifting a constant.

Table 6.6: The corrected designs against their baselines. Vitis HLS C-simulation, all 50 frames, four receive channels. Every configuration reconstructs every sample exactly.

Design	CR	MaxDiff	Δ CR
DRHE, lag 1	3.30987	0	—
DRHE, lag n_{Tx}	3.75082	0	+13.3%
LPC, lags 1, 2	3.38497	0	—
LPC, lags $n_{Tx}, 2n_{Tx}$	3.69681	0	+9.2%

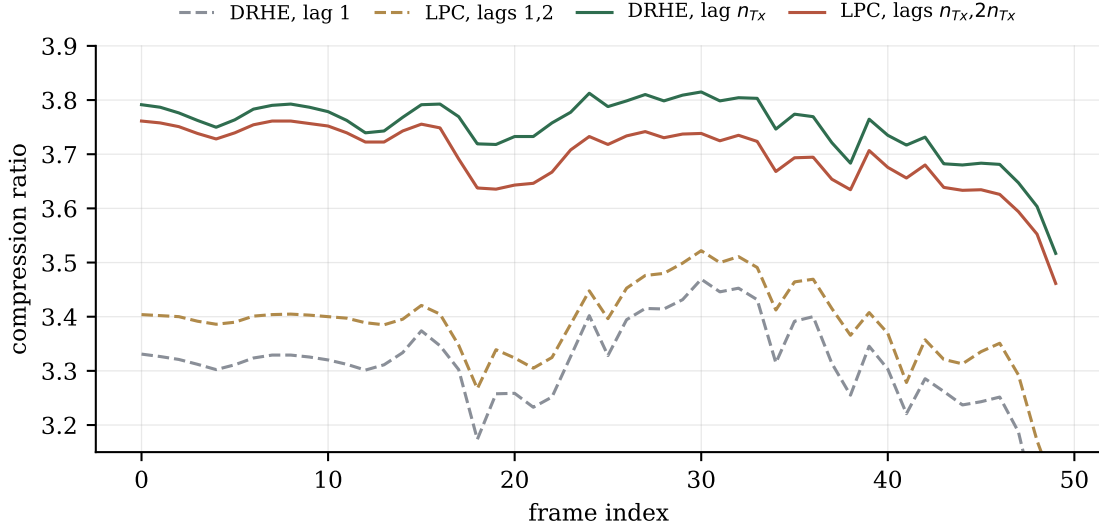


Figure 6.5: Per-frame compression ratio across the 50-frame sequence, measured in C-simulation. The corrected designs (solid) sit above their baselines on *every* frame, not merely on average, and the frame-to-frame variation — which tracks scene content — is preserved. All four configurations reconstruct every sample exactly.

6.6.2 What the correction costs

DRHE: close to free. Forty additional block RAMs — 4 % of the device — buys 13.3 % more compression. After routing, the corrected design uses 539 fewer LUTs (1.5 %) and 32 fewer flip-flops than the baseline, the same DSPs, and both close well above 100 MHz; differences this small are not interpreted, since run-to-run variation was not measured. RTL co-simulation measures the frame latency at exactly 61,253 cycles for *both* the baseline and the corrected design.

That last point is not a coincidence but a structural consequence: indexing the predictor state by transmitter changes *which* address is read, not *when*, so the schedule the tool generates is identical. The entire 13.3 % gain is bought with block RAM and nothing else.

LPC: a genuine trade. The LPC picture is more interesting, because the restructure that the deeper lags required turned out to fix an unrelated problem. Moving the autocorrelation out of the ingest pass deleted six 64-bit accumulator arrays and four history arrays, so the block RAM that HLS allocates *falls* by 25 %, from 256 to 192 (after routing, 244 to 216). More importantly, the ingest loop — previously stuck at $II = 3$ on accumulator port pressure — now meets $II = 1$, and so does every other loop in the design. The synthesis report states that all loop constraints are satisfied, which the baseline never achieved.

Against that: 4,731 more LUTs (15.9 %) and 6,585 more flip-flops (48.4 %) after routing, most of them in the four integer dividers that the ring-buffer index m % HIST generates, and — the real cost — considerably more latency.

Table 6.7: Cost of the correction, xcku5p at a 100 MHz target. Resources and $F_{\max}$ are post-route (Vivado); the HLS estimates for the same designs are in Table 7.1. II is from HLS; frame latency is measured in RTL co-simulation on a single frame.

Resource	DRHE		LPC	
	lag 1	lag n_{Tx}	lags 1, 2	lags $n_{Tx}, 2n_{Tx}$
LUT	35,343	34,804	29,772	34,503
FF	23,546	23,514	13,601	20,186
BRAM_18K	40	80	244	216
DSP	132	132	168	158
$F_{\max}$ (MHz)	135.5	131.0	118.6	106.3
Worst loop II	2	2	3	1
Frame latency (cyc)	61,253	61,253	197,277	325,166

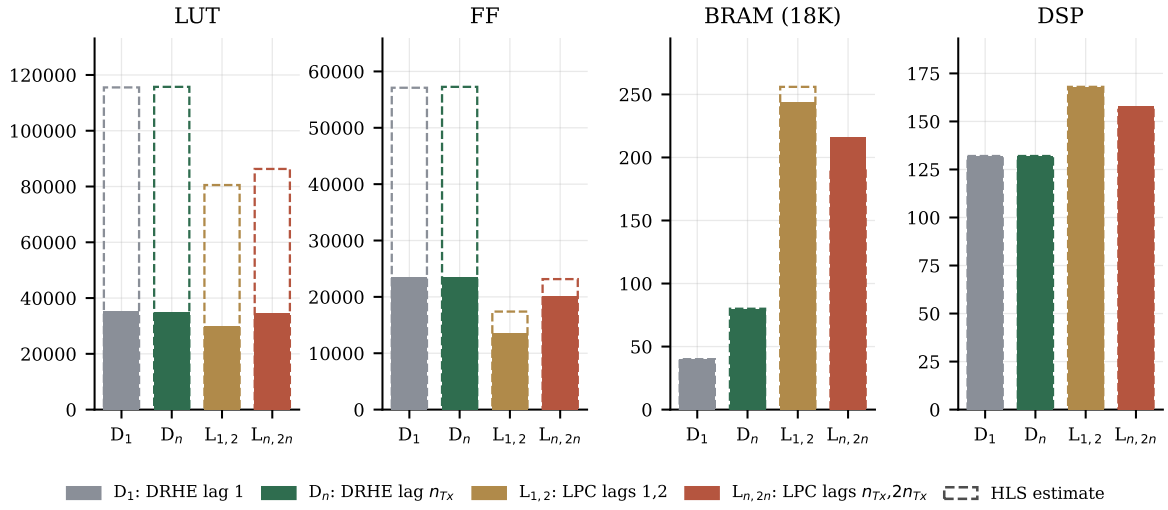


Figure 6.6: Resource usage for the four designs, post-route (solid) against the HLS estimate (dashed outline). Once implemented the four designs use similar logic; they differ in memory. The TDM correction moves DRHE’s BRAM from 40 to 80 and, by removing the ingest accumulators, moves LPC’s *down*.

6.6.3 A caution about cycle models

The latency result deserves its own discussion, because it is a methodological lesson as much as a number.

For the baseline designs the obvious cycle model — trip count times initiation interval, summed over loops — is accurate to 0.08 %, as Section 6.3 showed. Applying the same model to the corrected LPC predicts 246,272 cycles. Co-simulation measures 325,166. The model is wrong by 32 %.

The missing term is **pipeline drain**. Moving the autocorrelation into the per-bin pass replaced one long pipelined loop with *two short loops invoked 512 times each*. A pipelined loop pays its depth once on entry before it reaches steady state — 71 cycles for the correlation loop and 56 for the residual loop — so 512 invocations contribute roughly

$$512 \times (71 + 56) \approx 65,000 \text{ cycles} \quad (6.1)$$

of pure drain that no II-based model accounts for. That closes most of the gap.

So the corrected LPC is *slower* than the design it replaces: 325,166 cycles against 197,277, a factor of 1.65. This is a real cost and is stated as such rather than buried. It is, however, structural rather than algorithmic: merging the two per-bin loops, or processing several bins

Table 6.8: The two corrected designs compared. Resources and $F_{\max}$ are post-route. Bold marks the better of the two on each row.

	DRHE, lag n_{Tx}	LPC, lags $n_{Tx}, 2n_{Tx}$
Compression ratio	3.75082	3.69681
Lossless	yes	yes
LUT	34,804	34,503
FF	23,514	20,186
BRAM_18K	80	216
DSP	132	158
$F_{\max}$ (MHz)	131.0	106.3
Frame latency	61,253 cyc	325,166 cyc
Worst loop II	2	1
Compressor frame buffer	none	3.0 Mibit
First output available	immediately	after a full frame
Side information	none	1.04 %

per loop invocation so the drain amortises, would recover most of it. DRHE pays no such price at all.

6.6.4 Co-simulation

Both corrected designs pass RTL co-simulation with $\text{MaxDiff} = 0$, so losslessness survives synthesis as well as it did in software. The co-simulation is run on a single-frame stimulus, as is standard for a cycle-accurate run: 50 frames of RTL simulation with floating-point IP takes many hours of wall-clock time for no additional information, since the latency per frame is deterministic.

6.7 DRHE against LPC, after the correction

After the correction the ranking on compression ratio reverses: DRHE is ahead, 3.751 against 3.697, where before it was behind. That is because the correction helps DRHE more (13.3 %) than LPC (9.2 %), which makes sense — DRHE’s phase model is *specifically* a constant-Doppler model, so putting it on the Doppler axis recovers exactly what it was designed to find, whereas LPC’s general AR model was always going to fit *something*.

On logic the two are almost level once implemented: LPC uses 0.99 times DRHE’s LUTs and 0.86 times its flip-flops, where the HLS estimates had suggested LPC was much cheaper. On memory and latency DRHE is clearly better, and by a wide margin: no frame buffer at all, first output immediately, and a fifth of the frame latency.

For the application that motivates this thesis — a compression IP sitting between the range FFT and the memory bus in a streaming sensor front end — latency and streaming behaviour outrank a 1.5 % difference in ratio. **DRHE is the better choice**, and after the correction it happens to win on ratio too. LPC’s case rested on a logic saving that the implementation did not confirm.

6.8 Summary of results

1. Both compressors are exactly lossless over 50 frames of real data (4.9 million complex samples, 157 million bits), verified at three levels of abstraction.
2. As originally implemented, both predictors performed *worse than no prediction*: CR 3.319 and 3.397 against 3.489 for the entropy coder alone.

3. The cause is the transmitter-interleaved ramp axis of a TDM-MIMO cascade. Predicting at lag n_{Tx} raises measured compression to 3.751 (DRHE) and 3.697 (LPC), still exactly lossless.
4. For DRHE the correction costs 40 block RAMs and nothing else — logic and measured frame latency are unchanged, and the routed clock moves only from 135.5 to 131.0 MHz.
5. For LPC the correction is a trade: better ratio, 11.5 % less block RAM after routing (25 % in the HLS estimate), $\Pi = 1$ throughout, against $1.65\times$ the frame latency.
6. The ratio decomposes as $1.63\times$ from unused container headroom, $2.14\times$ from spectral sparsity, and $1.08\times$ from prediction once it is applied correctly.
7. Implemented through placement and routing, all four designs meet 100 MHz, and the HLS estimates had over-stated their logic by about three times. Chapter 7 accounts for what the implemented numbers mean, and for every difference from Kiem’s design.

Chapter 7

Accounting for the Hardware Numbers

Chapter 6 reported many hardware numbers, and Kiem’s thesis reports a comparable set for his design [11]. Put side by side, the two sets differ on almost every line: resources, clock, latency, throughput. A difference on its own explains nothing. This chapter’s job is to account for every one of them: to say what each number represents, what it should be compared against, and, wherever our design and Kiem’s differ, exactly why.

The accounting is done in three steps, and each step rests on a measurement rather than an argument.

1. **Put the numbers on the same footing.** Kiem’s resource and clock figures are measured after Vivado has placed and routed the circuit. Every hardware figure in Chapter 6 was first obtained as an HLS estimate, made before the circuit existed. We therefore ran the full Vivado implementation on all four designs (Section 7.1). The estimates turned out to be wrong by a factor of three in logic, and wrong about whether the designs meet timing.
2. **Rebuild the reference in our tool flow.** Two designs built with different tools, for different devices, from different code differ in many ways at once. To separate them, we rebuilt Kiem’s design from his description, in our tool, for our device (Section 7.4).
3. **Change one thing at a time.** Starting from our design and ending at the rebuilt reference, we changed one design decision per step and implemented every step (Section 7.5). The change in each number at each step is the cost of that one decision.

The rest of the chapter then reads each metric against the yardsticks that give it meaning: the sensor, the system’s time budget, the link and the device, as well as Kiem’s design where the comparison is like-for-like.

7.1 Two ways of counting the same circuit

HLS reports what it *expects* a circuit to cost, before the circuit has been built. Vivado reports what the circuit *did* cost, once it has been synthesised into device primitives, optimised, placed and routed. Kiem’s figures are the second kind; ours, at first, were the first.

7.1.1 Running the implementation

`vitis-run --mode hls --impl` takes the RTL that HLS generated and runs the complete Vivado flow on it: logic synthesis, optimisation, placement, routing and post-route reporting. It is the flow AMD calls “export and implement”, and its reports are the same kind of number

as Kiem’s Table 5.3. Four lines were added to each design’s configuration file and nothing else was changed:

```
vivado.flow=impl    vivado.clock=10    vivado.report_level=2
                    vivado.max_timing_paths=10
```

Each design is implemented as a standalone block (“out of context”) constrained to a 10 ns clock on the same `xcku5p-ffvb676-2-e`. One run takes about 45 min. It leaves behind a post-route utilisation report (total and per module), a timing report with the worst paths, and a power report; the script that reads all of them into this chapter’s tables is listed in Appendix C.

7.1.2 What the implementation found

Table 7.1 sets the two kinds of number side by side for all four designs.

Table 7.1: HLS estimates against the same designs after Vivado placement and routing, `xcku5p-ffvb676-2-e`, 10 ns constraint. BRAM in BRAM_18K units. $F_{\max}$ is $1/(10\text{ ns} - \text{WNS})$ after routing, and HLS’s own estimate before it. Every figure is read from the tool reports by `collect_impl.py`.

Design	LUT		FF		DSP		BRAM_18K		$F_{\max}$ (MHz)	
	HLS	impl.	HLS	impl.	HLS	impl.	HLS	impl.	HLS	impl.
DRHE-1	115,534	35,343	57,105	23,546	132	132	40	40	81.9	135.5
DRHE- n	115,736	34,804	57,263	23,514	132	132	80	80	81.9	131.0
LPC-1,2	80,528	29,772	17,393	13,601	168	168	256	244	85.4	118.6
LPC- n , $2n$	86,298	34,503	23,154	20,186	156	158	192	216	85.4	106.3

Three things stand out.

- **Logic was over-estimated by about three times.** HLS predicted 115,736 LUTs for DRHE- n ; the routed design uses 34,804. The other designs are similar. Every conclusion in earlier drafts that rested on DRHE being “2.5 times Kiem’s LUTs” rested on the estimate.
- **Both designs meet 100 MHz.** HLS estimated 81.88 MHz for DRHE and 85.43 MHz for LPC. After routing, DRHE- n closes at 131.0 MHz and LPC- n at 106.3 MHz. The timing conclusion first drawn in Section 6.3 reported an estimate, not a fact.
- **DSPs and block RAM were estimated almost exactly.** These are fixed blocks with known sizes, and HLS allocates them itself, so there is little for the later tools to change. The exceptions are the two LPC designs’ block RAM, where Vivado re-mapped some memories (LPC- n : 192 estimated, 216 used; LPC-1,2: 256 estimated, 244 used), and two DSPs in LPC- n .

7.1.3 Why the logic estimates were so far off

The per-module breakdown for DRHE- n in Figure 7.1 shows where the difference comes from. It is not spread evenly across the design.

The loop body. Most of the estimate sat in the logic HLS generates directly from C++: the bit packer, the S4 and Huffman lookup, the residual arithmetic and the control. Each of the eight inserts into the output buffer is written as `(ap_uint<512>)code << bit_count`, and HLS costs every one of them as a complete 512-bit barrel shifter, about 2,171 LUTs. Because `bit_count` is a signed `int`, it also adds a second shifter in the opposite direction for negative shift amounts, and a 512-bit select between the two. Vivado sees what HLS does

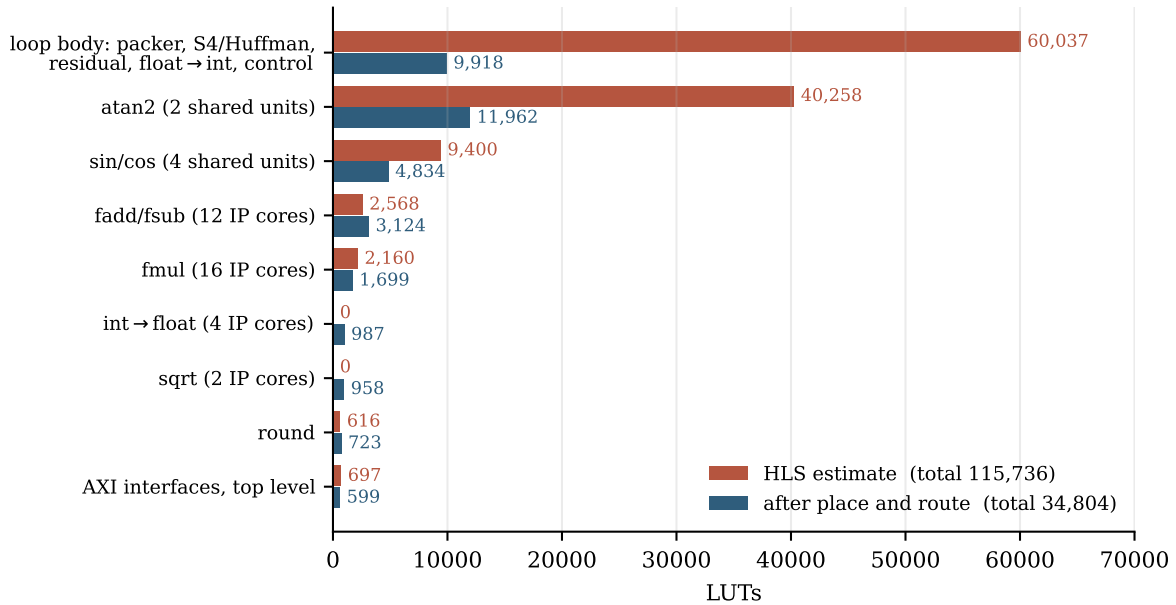


Figure 7.1: LUTs of each part of DRHE-*n*: the HLS estimate (from the module’s synthesis report) against the routed design (from the hierarchical utilisation report). The pre-built floating-point adders and multipliers were estimated well. The arctangent and the loop body, which HLS builds itself from C++, were over-estimated by factors of about three and six. The square root and the integer-to-float conversions were not estimated at all.

not: the value being shifted is at most 29 bits wide and zero everywhere else, so most of every shifter’s inputs are constant and the logic behind them disappears. The whole loop body is estimated at about 60,000 LUTs and implemented in about 9,900.

The arctangent. `hls::atan2(float, float)` is built by HLS from generic C++ — a CORDIC on a 43-bit internal fixed-point format with range reduction around it — rather than from a pre-characterised core. HLS estimated each instance at 20,129 LUTs; each implemented instance uses about 5,980. The same effect, smaller, applies to `sin` and `cos`.

What HLS did not count. The square root and the integer-to-float conversions are AMD floating-point IP cores that use no DSPs, and HLS reports them as zero LUTs. Implemented, they cost 1,945 LUTs between them. This is the one place where the estimate is too low.

The clock. HLS estimated a critical path of 12.213 ns through an integer-to-float conversion chained into a multiply. That figure is HLS’s own model of each operator’s delay, scheduled against a budget that already reserves 2.7 ns (27 per cent of the period) for clock uncertainty; it is not a timing analysis of real wires. After routing, the worst path in DRHE-*n* is 7.633 ns, starting in one of the floating-point sine/cosine units, and it passes the 10 ns constraint with 2.367 ns to spare.

What this means for the rest of the thesis. HLS estimates are useful for what they were designed for: comparing one version of a design against another in the same tool, quickly. They are not a substitute for implementation when the comparison is with a design that was implemented. From here on, every hardware number compared with Kiem is a post-route number.

7.2 Where the numbers come from, and how far to trust them

Table 7.2 sorts every hardware number in this thesis by where it is read from and what kind of number it is.

Table 7.2: Where each hardware number is read from, and what kind of number it is. *top* is the name of the design’s top-level function. All files are produced by Vitis HLS and Vivado 2026.1; `collect_impl.py` reads them all (Appendix C).

Metric	Read from	What kind of number
LUT, FF, BRAM, DSP	<code>top_export.rpt</code> , post-implementation section	<i>Measured</i> on the routed design.
Per-module resources	<code>top_utilization_hierarchical_routed.rpt</code>	<i>Measured</i> on the routed design.
Clock ($F_{\max}$)	<code>top_timing_routed.rpt</code> , worst negative slack	<i>Measured</i> : static timing analysis of routed wires; $F_{\max} = 1/(T - \text{WNS})$.
Power	<code>top_power_routed.rpt</code>	<i>Estimated</i> from the routed design with default switching activity.
HLS estimates	<code>csynth.rpt</code> , top row	<i>Estimate</i> made before the circuit exists.
II, pipeline depth	<code>csynth.rpt</code> , loop table; <code>*.verbose.sched.rpt</code>	<i>Exact</i> for the generated schedule.
Frame latency	<code>top_cosim.rpt</code> , Verilog row	<i>Measured</i> : cycle-accurate simulation of the generated RTL.
Compression ratio	C-simulation and co-simulation logs	<i>Measured</i> : bits out divided by bits in, on real data.

Two limits on the post-route numbers should be kept in mind. First, the designs are implemented out of context. Vivado therefore does not know the clock tree of a real system and cannot include its skew; the timing figures are slightly optimistic compared with an in-system build like Kiem’s. Second, the power figure uses Vivado’s default assumption that every signal toggles on 12.5 % of clock cycles, not activity recorded from real data, so it is good for comparing designs with each other but is not a prediction of a particular system’s power.

The frame latency from co-simulation is exact for the circuit simulated, but it measures a specific situation: the testbench feeds the frame back-to-back, one beat per cycle whenever the design will accept it. So 61,253 cycles is the time the compressor would take if an entire frame were waiting for it at once. The real sensor does not deliver data like that, which is why Section 7.6 separates this number from the latencies the system actually experiences.

7.3 The yardsticks

Six things can give a hardware number meaning:

1. **The sensor.** This is how fast data arrives and how much time passes between ramps and between frames. It sets the throughput requirement.
2. **The system budget.** This is how long the rest of the processing chain can afford to wait for compressed data. Kiem derived one explicitly.
3. **The link.** This is how many bits per second the output connection can carry. It sets the compression ratio requirement.
4. **The device.** This is how much of the chip the design uses, and whether it would fit on a smaller one.
5. **Kiem’s implementation.** This is the only published hardware DRHE, but it is valid only where the two numbers measure the same thing.

6. **Internal baselines.** These are the design’s own theoretical ideal, the no-prediction coder, and the entropy bound.

Table 7.3 gives the sensor and system figures. They are used throughout the rest of the chapter.

Table 7.3: Sensor and system figures used as yardsticks. ColoRadar values follow from Table 2.1 and [12]. Kiem’s values are from his Section 3.3 [11]. “Post-FFT” means the retained half-spectrum as 16-bit complex values, which is what the compressor receives.

Quantity	ColoRadar cascade	Kiem’s sensor
Range bins per ramp (retained)	128	512
Ramps per frame	192	1024
Receive channels	16 (4 per compressor)	4
Ramp interval	78.5 μ s	$\approx 20.5 \mu$ s
Frame acquisition time	15.07 ms	≈ 21 ms
Frame period	200 ms (5 Hz)	50 ms (20 Hz)
Compression time budget	—	15 ms
Post-FFT rate, 4 RX, during a frame	0.209 Gbit/s	3.20 Gbit/s
Post-FFT rate, all RX, during a frame	0.835 Gbit/s	3.20 Gbit/s
Clock cycles per ramp at 100 MHz	7,850	$\approx 2,050$

One cross-check is worth recording. Sixteen receive channels, 192 ramps and 256 complex 16-bit samples per ramp is 25.2 Mbit of raw ADC data per frame, which at 5 Hz is 125.8 Mbit/s. The ColoRadar authors quote 126 Mbps for this sensor [12]. The sensor model used in this chapter therefore agrees with the published one.

7.4 Rebuilding the reference design

7.4.1 Why a replica is needed

A comparison of two numbers explains something only if the two numbers differ in one known way. Our DRHE and Kiem’s differ in at least six ways at once: the arithmetic (floating point against fixed point), the bit packer, how the predictor state is reset, the feedback structure (closed against open loop), the tool version (Vitis 2026.1 against 2022.2) and the device (a xcku5p at speed grade -2 against an XCZU3CG-SFVC784-1-e at speed grade -1). Set side by side, the two resource tables cannot say how much of any difference comes from which cause.

The way out is to hold everything constant except one thing at a time, and that needs Kiem’s design inside our tool flow. His source code is not published, so we rebuilt it from the description in his Section 4.3.4, which is detailed enough to do so:

- **Data types.** Magnitude `ap_ufixed<16,16>`, phase `ap_fixed<16,3>`, input `ap_fixed<16,16>`, intermediate phase arithmetic `ap_fixed<32,4>`, all with rounding.
- **Functions.** `hls::sqrt` on `ap_uint<32>`; `hls::atan2` on `ap_fixed<16,16>`; `hls::sin` and `hls::cos` on `ap_fixed<16,3>`.
- **State.** Five arrays of 4×1024 entries, partitioned by channel, driven from the *input* sample (open loop).
- **Packing.** The real and imaginary codes of one channel packed into one `ap_uint<58>`; the four channel words then combined into the 256-bit output buffer.
- **Schedule.** All four channels in parallel, $II = 1$.

Where his text leaves a choice open, the replica takes the simplest reading. One such choice needs stating. Kiem does not describe how the predictor state is cleared at the start of a frame, but he reports $II = 1$, and Section 7.5.4 shows that $II = 1$ is only possible if the

clearing does not add a third memory access per cycle. The replica therefore clears the state by reading it as zero on the first ramp, which is equivalent.

7.4.2 One source, four switches

The replica and our design are built from one source file with four compile-time switches (`drhe_abl_compress.cpp`, Appendix C).

Switch	0: our design (DRHE-1)	set: Kiem’s design
ABL_PACK	eight inserts into a 512-bit buffer with a signed offset	1: the same with an unsigned offset; 2: 58-bit word per channel, one 232-bit insert
ABL_RESET	state zeroed by a write on ramp 0	state read as zero on ramp 0
ABL_ARITH	single-precision floating point	Kiem’s fixed-point types and functions
ABL_LOOP	closed loop: state from the reconstruction	open loop: state from the input

With every switch at zero the source is, statement for statement, DRHE-1: it synthesises to the same schedule (II 2, depth 58, 40 BRAM_18K, 132 DSPs) and writes exactly as many bits as DRHE-1 on every one of the 50 frames. With every switch set it is the replica. All the packing modes write the same bits, so one decoder reads them all, and the decoder follows the arithmetic switch. Every variant was decoded losslessly on real ColoRadar frames in C simulation, so each one is a working compressor, not a sketch.

7.4.3 How close is the replica to Kiem’s published design?

Table 7.4 sets the implemented replica beside Kiem’s published figures.

Table 7.4: The rebuilt design against Kiem’s published compression IP (his Table 5.3 and Section 5.2.3). Replica figures are post-route, out of context, on the `xcku5p` at speed grades -2 and -1 ; Kiem’s are post-route, in system, on the `XCZU3CG-SFVC784-1-e`. Kiem’s BRAM is given in 36 Kb tiles and converted.

	Replica (-2)	Replica (-1)	Kiem [11]
LUT	17,141	17,228	45,892
FF	5,050	5,598	9,243
DSP	48	48	44
BRAM (36 Kb tiles)	10	10	10
Initiation interval	1	1	1
Pipeline depth (HLS, cycles)	12	14	23
Achieved clock (MHz)	129.5	125.0	100 (met)
Compression ratio (ColoRadar, 10 frames)	3.31904		—

The replica agrees with Kiem’s design where the design is fully specified. Block RAM matches exactly: 10 tiles against his 10, because both store twenty arrays of 1,024 16-bit words (Section 7.5.1). The DSP count is 48 against 44. Both designs achieve $II = 1$ and both meet 100 MHz on a speed grade -1 device.

The replica does *not* reproduce Kiem’s LUT and flip-flop counts. It needs 17,141 LUTs and 5,050 flip-flops where Kiem reports 45,892 and 9,243. His design, as he describes it, does not need the difference. Section 7.5.5 tests the explanations his text leaves open — the speed grade, the packer he describes in one sentence, and his deeper pipeline — by building each into the replica. Together they account for about half of the flip-flop difference and almost none of the LUT difference; what remains most likely lies in the tool version (Vitis HLS

2022.2, whose fixed-point math library is four years older than the one used here) or in logic his thesis does not describe.

This matters for how his LUT figure is used. It measures one particular build; it is not a measure of what the DRHE algorithm costs. When the question is “what do our design decisions cost?”, the fair yardstick is the replica, built in the same tool, for the same device, from the same description, and the next section uses it that way. When the question is “how does our IP compare with the IP Kiem built?”, his published figure is the right one.

7.5 From our design to Kiem’s, one change at a time

Table 7.5 is the central result of this chapter. Each row is a complete, lossless compressor, implemented through placement and routing on the same device with the same constraints. The first row is DRHE-1 as built. Each following row changes exactly one design decision, and the last is the replica of Kiem’s design. The difference between two adjacent rows is therefore the cost, in every resource at once, of that one decision.

Table 7.5: From DRHE-1 to the replica of Kiem’s design, one design decision per step. Every row is a lossless compressor implemented through placement and routing on the xcku5p-ffvb676-2-e at 10 ns. II and depth are from HLS; everything else is post-route. BRAM in BRAM_18K units.

Step	Change from the row above	LUT	FF	DSP	BRAM	II	Depth	MHz
0	DRHE-1 as built	35,343	23,546	132	40	2	58	135.5
1	packer shift offset made unsigned	33,502	24,423	132	40	2	58	128.4
2	Kiem’s 58-bit per-channel packer	31,124	21,261	127	40	2	62	120.6
3	state cleared by read, not write	52,432	30,373	264	40	1	59	131.4
4	Kiem’s fixed-point arithmetic	16,882	5,380	48	20	1	16	133.3
5	open loop = replica of Kiem	17,141	5,050	48	20	1	12	129.5
—	Kiem, published [11]	45,892	9,243	44	20	1	23	100

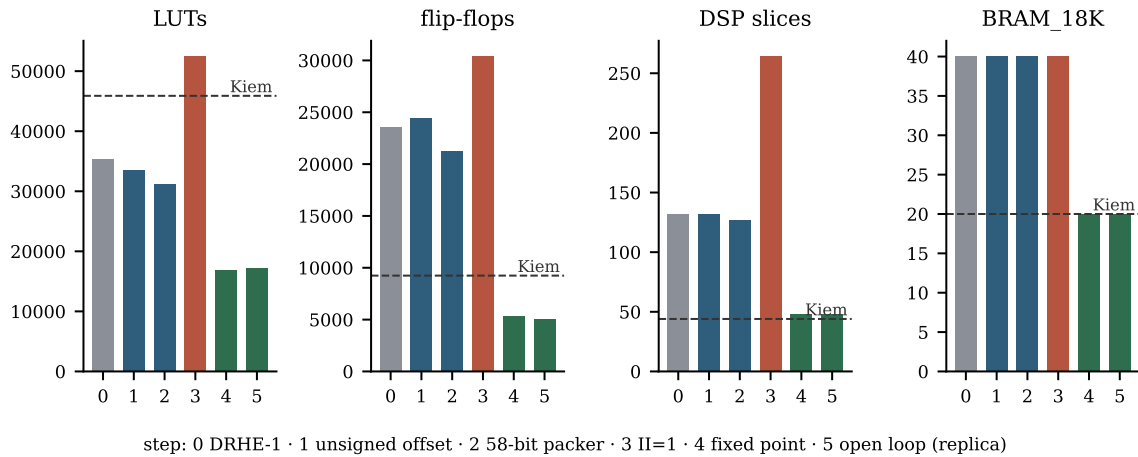


Figure 7.2: LUTs, flip-flops, DSPs and block RAM at each step of Table 7.5, all post-route on the xcku5p. The dashed line in each panel is Kiem’s published figure. Every step but one reduces the design; the exception raises it from II = 2 to II = 1, which doubles the arithmetic because the units can no longer be shared between channels.

The following subsections take each metric in turn and account for its difference from Kiem using the ladder, the per-module reports and, where relevant, the HLS schedule.

7.5.1 Block RAM: the width of a word

The difference. DRHE-1 uses 20 block-RAM tiles; Kiem’s design uses 10.

The accounting. Block RAM holds DRHE’s predictor state: five values per range bin and channel from earlier ramps (the magnitude prediction, the previous magnitude, the phase prediction and the two previous phases). Partitioned by channel, as both designs do, that is $5 \times 4 = 20$ separate arrays, each 1,024 words deep. Table 7.6 lists how each design stores them.

Table 7.6: How the predictor state is stored, from the HLS memory tables and the routed utilisation reports. A BRAM_18K holds 1,024 words of up to 18 bits; a 36 Kb tile is two of them.

Design	Arrays	Words	Bits	BRAM_18K per array	Total tiles
DRHE-1 (float)	20	1,024	32	2	20
Replica, Kiem (fixed)	20	1,024	16	1	10
DRHE- n (float, TDM)	20	1,536	32	4	40

Every DRHE-1 state word is a 32-bit `float`. A 1,024-word array of 32-bit words is 32 kbit, too much for one BRAM_18K, so each array takes a full 36 Kb tile: twenty arrays, twenty tiles. Kiem’s words are 16-bit fixed point (`ap_ufixed<16,16>` and `ap_fixed<16,3>`); a 1024×16 array fits in one BRAM_18K, so twenty arrays need ten tiles. The replica, built with his types, uses exactly ten. **The factor of two is the word width, and nothing else.**

DRHE- n keeps one copy of the state per transmitter, 1,536 words per array instead of 1,024. HLS rounds the depth up to the next power of two and gives each array four BRAM_18K, so the design uses 40 tiles: four times Kiem’s, of which two is the word width and two is the per-transmitter state. The memory is 67 % full (983,040 bits stored in 1,474,560 bits of block RAM); the rounding of the depth to a power of two and the unused parity bits are the only waste.

7.5.2 DSP slices: counted operator by operator

The difference. DRHE-1 uses 132 DSP slices; Kiem’s design uses 44.

The accounting. Every DSP in DRHE-1 belongs to an identifiable floating-point operator, and the routed hierarchy report counts them exactly (Table 7.7).

Table 7.7: Where DRHE-1’s DSP slices go, from the routed hierarchical utilisation report.

Operator	Instances	DSPs each	DSPs
float multiply (<code>fmul</code> , max-DSP)	16	3	48
float add/subtract (<code>fadd</code> , full-DSP)	12	2	24
float sine/cosine	4	13	52
float arctangent	2	4	8
Total			132

A single-precision multiply needs three DSP slices because its 24-bit significand product does not fit one 27×18 multiplier; an adder uses two, for the alignment shift and the add; the sine/cosine unit uses thirteen for its polynomial and range reduction. In fixed point, a product of two 16-bit numbers is one slice and an addition needs none. The replica uses 48 DSPs against Kiem’s 44.

Four channels at two operations each would need eight sine/cosine units and four arctangents. DRHE-1 has four and two, because at $\Pi = 2$ each unit is idle every other cycle and HLS shares it between two channels. Step 3 of the ladder removes that sharing: at $\Pi = 1$ the same float design needs 264 DSPs. **The factor of three is floating-point arithmetic; without the sharing that $\Pi = 2$ allows, it would be six.**

7.5.3 Pipeline depth: which operations wait for which

The difference. DRHE-1’s pipeline is 58 cycles deep; Kiem reports 23 (his HLS estimate, the same kind of number).

The accounting. The HLS schedule report lists, for every operation, the pipeline stage it runs in. Grouping the operations of one sample by what they do gives Figure 7.3(a). The pipeline is two chains laid end to end. The first is the prediction: load the state, run the two IIR filters, wrap the phase, take the sine and cosine, rescale to Cartesian and round, which ends around stage 31. The second is the polar conversion that updates the state: convert the sample to floating point, take $re^2 + im^2$ and its square root, and take the arctangent, which alone occupies 28 stages. In DRHE-1 the second chain *cannot start until the first has finished*, because it converts the reconstructed sample, and the reconstruction needs the prediction. That is the closed loop introduced in Section 5.4.1.

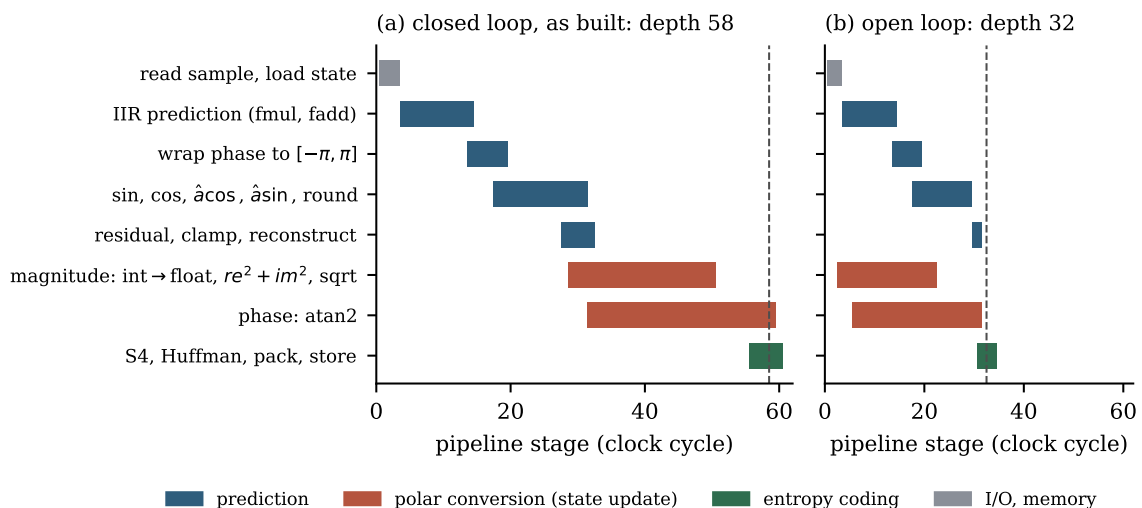


Figure 7.3: The pipeline schedule of one sample, read from the HLS schedule reports. (a) DRHE-1 as built: the state is updated from the reconstructed sample, so the polar conversion (red) waits for the prediction (blue) and the depth is 58. (b) The same float design with the state updated from the input sample (open loop): the two chains run side by side and the depth is 32. Kiem’s design is open loop and fixed point, and shorter still.

Switching the loop back open, so the state is driven from the input sample, lets the polar conversion start in stage 3 alongside the prediction. The float design’s depth falls from 58 to 32 (Figure 7.3(b); the open-loop depth is measured, not drawn). Fixed-point arithmetic then shortens both chains. The replica’s fixed-point `atan2` and `sqrt` are each a few cycles rather than 28 and 11, and the closed-loop fixed-point design (step 4) is 16 cycles deep, the open-loop replica (step 5) 12. The replica is shallower than Kiem’s reported 23 cycles; his figure comes from an older tool version and from code that split its arithmetic into extra stages to meet timing (his Section 4.4.1), so an exact match is not expected.

Design	Depth	What sets it
DRHE-1 (float, closed loop)	58	prediction, then polar conversion, in series
float, open loop	32	the longer of the two chains (atan2)
fixed point, closed loop	16	fixed-point chains in series
replica (fixed point, open loop)	12	fixed-point chains in parallel
Kiem (HLS estimate)	23	—

Of DRHE-1’s 58 cycles, 26 are the closed loop; the open-loop float design is still 20 cycles deeper than the replica, and that is the latency of floating-point operators. The closed loop is the price of guaranteeing that the encoder and the decoder can never disagree; at 260 ns it is not a price this application notices (Section 7.6).

7.5.4 Initiation interval: one write too many

The difference. DRHE-1 accepts a new sample every two cycles ($II = 2$); Kiem’s design accepts one every cycle.

The cause, as HLS reports it. The synthesis log names it:

```
Unable to schedule 'load' operation 32 bit ('pp2') on array ...s_prev_prev_phase_3
due to limited memory ports (II = 1).
```

Each state array is one block RAM, and a block RAM has two ports. In every cycle each array must be read once (the old value) and written once (the new one). DRHE-1 also clears the state at the start of a frame with an *extra* write on ramp 0 (if ($r == 0$) $s_prev_prev_phase[ch][s] = 0$;). That write only happens once in 192 ramps, but HLS must schedule for the worst case, so every cycle is scheduled as if three accesses were needed, and three accesses on two ports take two cycles.

The proof. Step 3 of the ladder replaces the write with a read that is ignored on ramp 0 ($pp2 = first ? 0 : s_prev_prev_phase[ch][s]$), which clears the state just as well with one read and one write per cycle. The same float design then schedules at $II = 1$. Steps 1 and 2, which change only the packer, stay at $II = 2$ with the identical message.

What it costs to fix. $II = 1$ doubles the theoretical throughput. In RTL co-simulation, where the step-3 design also reconstructs its frame losslessly, the measured rate rises from 5.14 to 8.76 Gbit/s (35,909 cycles per frame against 61,253); it does not quite double because the 59-cycle pipeline still drains once per ramp. Step 3 of Table 7.5 also shows the bill: the float arithmetic can no longer be shared between channels, so DSPs and logic both rise. This is the one step in the ladder that makes the design substantially larger, and it is why DRHE-1’s $II = 2$ was not a pure loss: with 24 times more throughput than the sensor needs (Section 7.7), the halved rate buys a design half the size in arithmetic.

7.5.5 LUTs and flip-flops: two gaps, not one

The difference. DRHE-1 uses 35,343 LUTs and 23,546 flip-flops. Kiem reports 45,892 and 9,243. So against his published figures our design uses *fewer* LUTs ($0.77\times$) but *more* flip-flops ($2.55\times$), which looks contradictory: a design with more registers would be expected to have more logic too.

Why one comparison cannot explain it. The two ratios mix two separate gaps. The first is between DRHE-1 and the replica: both built in the same tool for the same device, differing only in the design decisions the ladder changes. The second is between the replica and Kiem’s published build: the same design, as far as his description goes, built in a different tool version, in a different system, from code we have not seen. Measured against the replica, DRHE-1 uses $2.06\times$ the LUTs and $4.66\times$ the flip-flops, both larger, and there is no contradiction. The contradiction comes from the second gap, which is large for LUTs (Kiem’s build uses $2.68\times$ the replica’s) and smaller for flip-flops ($1.83\times$). This subsection accounts for the first gap completely and for the second as far as experiments can.

The first gap, module by module. The routed hierarchy report lists every module in DRHE-1 with its own LUTs and flip-flops (Table 7.8). The floating-point operators, which HLS instantiates as separate modules, are 69% of the LUTs and 54% of the flip-flops. The two arctangent units alone are 11,961 LUTs, a third of the design, because a single-precision `atan2` is a 43-bit CORDIC with range reduction around it. The rest is the loop body — the packer, the S4 and Huffman encoding, the residual arithmetic, and the registers that carry each value down a 58-stage pipeline until it is used — plus the AXI interfaces.

Table 7.8: Where DRHE-1’s LUTs and flip-flops go, from the routed hierarchical utilisation report.

Module	Instances	LUT	FF
float arctangent (<code>atan2</code>)	2	11,961	5,334
float sine/cosine	4	5,022	1,296
float add/subtract	12	3,126	2,778
float multiply	16	1,678	2,054
float square root, int-to-float, rounding	10	2,672	1,354
<i>all floating-point operators</i>		24,459	12,816
loop body (packer, coder, residual, pipeline registers)	1	10,258	8,278
AXI-Stream and AXI-Lite interfaces	3	549	927

The first gap, decision by decision. The ladder of Table 7.5 turns the module view into costs. Each step’s change is the cost of one decision, and the five changes add up exactly to the difference between DRHE-1 and the replica.

Step 1, the shift offset (1,841 fewer LUTs, 5 %). DRHE-1’s packer shifts each code into the output buffer by `bit_count`, a signed `int`. A shift by a signed amount has to handle negative amounts too, so every one of the eight inserts carries logic for a right shift that never happens, and Vivado keeps it, since nothing in the netlist shows that the counter is never negative. Making the counter an unsigned 10-bit number removes that logic. Flip-flops move by 877 (more), a small change that was not investigated further; run-to-run variation was not measured, so changes of this size are not interpreted.

Step 2, the packer (2,378 fewer LUTs, 3,162 fewer flip-flops). Kiem’s packer joins each channel’s real and imaginary codes into one 58-bit word, joins the four words into one 232-bit word, and inserts that once. Eight variable-offset inserts into a 512-bit buffer become three small shifts inside a 232-bit word and one insert. The loop body, which holds the packer, falls from 8,723 to 6,482 LUTs and from 9,355 to 6,622 flip-flops. Together, steps 1 and 2 show that DRHE-1’s packer costs about 4,200 LUTs more than it needs to, 12% of the design.

Step 3, the initiation interval (21,308 more LUTs, 9,112 more flip-flops). Removing the extra state write lets the loop run at $II = 1$ (Section 7.5.4). At $II = 2$ each floating-

point unit was shared by two channels; at $II = 1$ each channel needs its own. The floating-point operators grow from 23,487 to 45,407 LUTs, which is the whole of the step's increase, and the DSP count doubles, to 264. This is the only step that makes the design larger, and it is the price of doubling the throughput with floating-point arithmetic.

Step 4, fixed-point arithmetic (35,550 fewer LUTs, 68 %; 24,993 fewer flip-flops, 82 %).

Kiem's types replace every floating-point operator with fixed-point logic: 16-bit multiplies in single DSP slices, additions in carry chains, and fixed-point `sqrt`, `atan2`, `sin` and `cos` built by HLS directly into the loop body. The flip-flops fall further than the LUTs because both of their causes shrink at once: the values carried down the pipeline are 16 bits wide instead of 32, and the pipeline is 16 stages deep instead of 59, so there are fewer, narrower values to hold for fewer cycles.

Step 5, the open loop (259 more LUTs, 330 fewer flip-flops). Driving the state from the input sample instead of the reconstruction shortens the pipeline by four stages and removes 330 flip-flops; the LUT change is small (2 %). In fixed point, the closed loop that makes DRHE-1 immune to encoder–decoder divergence is almost free.

The order of the steps matters for their sizes, though not for their sum. The fixed-point step is measured at $II = 1$, where the float design has twice as many operators as DRHE-1 does, so it saves more than the same change would save at $II = 2$. To measure that directly, one more variant changes *only* the arithmetic of DRHE-1, keeping its packer, its state write and so its $II = 2$, and its closed loop. It uses 20,549 LUTs (42 % fewer than DRHE-1), 9,230 flip-flops (61 % fewer), 44 DSPs — exactly Kiem's count — and 20 BRAM_18K, with a pipeline 12 cycles deep. It reconstructs ColoRadar losslessly and meets 100 MHz, if only just (102.8 MHz): with the loop closed and only 12 stages, the fixed-point arithmetic feeding the state write is now the critical path (Section 7.5.6). This is the saving available to DRHE-1 as it stands. It still carries DRHE-1's expensive packer, which in the float design cost about 4,200 LUTs more than Kiem's (steps 1 and 2); its cost in fixed point was not measured separately.

So why does DRHE-1 have more flip-flops than Kiem, but fewer LUTs? Because the two gaps go in opposite directions for LUTs and the same direction for flip-flops. DRHE-1 needs $4.66\times$ the replica's flip-flops, for the reasons of steps 2 to 5: a 58-stage pipeline of 32-bit values, the registers inside each floating-point operator, and the packer. Kiem's build needs only $1.83\times$ the replica's, so DRHE-1 ends up with more than he has. DRHE-1 needs $2.06\times$ the replica's LUTs, but Kiem's build needs $2.68\times$, so DRHE-1 ends up with fewer. Both ratios against Kiem are therefore the product of a gap we can account for and one that we can only partly explain, which is the subject of the rest of this subsection.

The second gap: the replica against Kiem's build. Section 7.4.3 left the replica 28,751 LUTs and 4,193 flip-flops short of Kiem's published figures. His thesis does not give enough detail to reconstruct the difference, but three explanations can be tested, and each was: the replica was changed in one respect at a time and implemented again (Table 7.9).

Speed grade. Kiem's device is speed grade -1 , ours -2 . HLS schedules for the part it is given: for the slower part it inserts two more pipeline stages (14 against 12), which adds 548 flip-flops, 13 % of the flip-flop gap, and almost no LUTs. This is the effect Kiem describes in his Section 4.4.1, where he split arithmetic into extra registered steps to meet timing, measured.

The packer. Kiem describes his packer in one sentence: each channel's 58-bit word is “combined” into the 256-bit buffer. The replica joins the four words first and inserts once. The

Table 7.9: Testing explanations for the gap between the replica and Kiem’s published LUT and flip-flop counts. Each row is the replica with one thing changed, implemented and routed on the xcku5p. “Share” is the change from the replica as a fraction of the gap to Kiem (28,751 LUTs, 4,193 flip-flops).

Variant	Tests	LUT	FF	LUT share	FF share
Replica, speed grade -2	the reference point	17,141	5,050	—	—
Replica, speed grade -1	Kiem’s speed grade	17,228	5,598	0 %	13 %
Replica, literal packer	four inserts, <code>int</code> offset	18,576	6,214	5 %	28 %
Replica, 23-cycle pipeline	Kiem’s reported depth	17,060	5,217	0 %	4 %
Kiem, published [11]	in system, 2022.2	45,892	9,243	100 %	100 %

most literal alternative reading inserts each channel’s word in turn, with a signed `int` bit counter, as DRHE-1 does. That costs 1,435 more LUTs and 1,164 more flip-flops: 5 % of the LUT gap and 28 % of the flip-flop gap. So a less careful packer would account for about a quarter of his extra flip-flops, but no plausible packer accounts for his LUTs. DRHE-1’s own packer, the most expensive one measured in this work, costs about 4,200 LUTs more than Kiem’s (steps 1 and 2), still a small fraction of 28,751.

Pipeline depth. Kiem reports 23 cycles where the replica schedules 12. Forcing the replica’s loop to 23 stages with an HLS latency constraint adds only 167 flip-flops, because HLS places the extra stages where few values are live. Depth by itself therefore does not explain his flip-flop count; where his extra stages sit, inside the 32-bit intermediate arithmetic his Section 4.3.4 describes, is what would matter, and that cannot be reproduced without his code.

What is left. If their effects add, the three tested explanations account for about 45 % of the flip-flop gap but only about 5 % of the LUT gap. Two explanations could not be tested:

- **The tool version.** Kiem used Vitis HLS 2022.2. The replica’s logic is dominated by the fixed-point `sqrt`, `atan2`, `sin` and `cos` that HLS builds from its math library into the loop body (the loop body, which also holds the packer, coder and residual logic, is 12,691 of the replica’s 17,141 LUTs), and that library’s fixed-point implementations are the part of the flow most likely to have changed in four years. Only 2026.1 was available, so this is a hypothesis rather than a finding.
- **Code or debug logic his thesis does not describe.** His Section 4.2.1 and Figure 5.4 show an integrated logic analyser probing the compressor’s internal signals in the implemented system. Signals marked for debug are kept through optimisation, so a build with probes can be larger than one without; his thesis does not say whether the build he reports had them, nor describe his control logic in detail.

In-system implementation, the other difference, is unlikely to matter: Vivado synthesises each IP in a block design out of context by default, exactly as here, and placement and routing change LUT counts only slightly.

What this means for the comparison. The difference between DRHE-1 and Kiem’s published LUT count is *not* a property of either algorithm or of any design decision described in either thesis: the same design, rebuilt, needs 17,141 LUTs. The honest statement is therefore the one Section 7.4.3 made: Kiem’s published figure measures his build; the replica measures his design; and against his design, DRHE-1’s extra cost is fully accounted for by its four distinctive decisions: floating-point arithmetic, its packer, the extra state write and the closed loop.

7.5.6 Clock frequency and speed grade

What it is. $F_{\max}$ is the fastest clock at which every signal still settles within one period. Vivado’s timing analysis after routing gives the worst negative slack (WNS): how much time the slowest path has to spare at the 10 ns constraint. The slowest path can then be run at a period of 10 – WNS nanoseconds.

Why measure it. Throughput and latency in seconds both depend on the clock, and the compressor has to share a clock with the FFT and DMA around it. Kiem ran all his IP on one 100 MHz clock.

The numbers. Every design meets 100 MHz. The routed worst paths are:

Design	WNS (ns)	$F_{\max}$ (MHz)	Worst path
DRHE-1	2.621	135.5	inside a float sine/cosine unit
DRHE- n	2.367	131.0	from a float sine/cosine unit into the loop
LPC- $n, 2n$	0.595	106.3	frame-buffer read into the solver’s 64-bit multiply
DRHE-1, speed grade –1	2.769	138.3	inside a float sine/cosine unit
Fixed-point DRHE-1 (II 2)	0.272	102.8	a 26-stage carry chain into the state write
Replica, speed grade –2	2.276	129.5	a 20-stage carry chain in the fixed-point arithmetic
Replica, speed grade –1	1.999	125.0	an 18-stage carry chain in the fixed-point arithmetic

Against Kiem: now valid, with one correction. Kiem’s design met 100 MHz on a speed grade –1 part, which is slower silicon than the –2 part used here. The replica closes at 125.0 MHz on a –1 part and 129.5 MHz on a –2, so the speed grade is worth about 3.5 % of clock for the same source (HLS also schedules the slower part two stages deeper, so it is not quite the same netlist). Implemented on a –1 part, DRHE-1 itself closes at 138.3 MHz — slightly faster than on the –2 part, because HLS scheduled it 63 stages deep instead of 58 for the slower silicon — so it meets Kiem’s clock on Kiem’s grade of silicon with room to spare.

One design with little margin. The fixed-point version of DRHE-1 (Section 7.5.5) closes at only 102.8 MHz. With the loop closed and the pipeline only 12 stages deep, the path from the fixed-point arithmetic to the state write has 38 levels of logic, 26 of them carry chains. That is the cost of a short closed-loop pipeline, and adopting the fixed-point design would mean either accepting that margin or asking HLS for one or two more stages, which the replica’s schedule shows are cheap.

The HLS estimate was the wrong way round. HLS estimated 81.88 MHz for DRHE and 85.43 MHz for LPC and put the critical path in a different place from where it really is. Every design the estimate said would fail passes, with a quarter of the period to spare in DRHE’s case. The advice in earlier drafts to pipeline the float datapath or replace it with a CORDIC “to meet timing” was addressing a problem that does not exist.

When does it become a problem? If a design cannot share the system clock, it needs its own clock and a clock-domain-crossing FIFO on each side, which is extra design work and extra memory. None of these designs needs one. Of the four main designs, LPC- n has the least margin, 0.595 ns; a slower speed grade or a congested in-system build could consume it.

7.6 Latency: three different things with the same name

“Latency” means *how long something waits*. The question is always *what* waits, and for *what*. Three quantities in this project are called latency, and they answer different questions. Confusing them is the easiest way to draw a wrong conclusion, including when comparing with Kiem. Figure 7.4 puts all of them on one time axis.

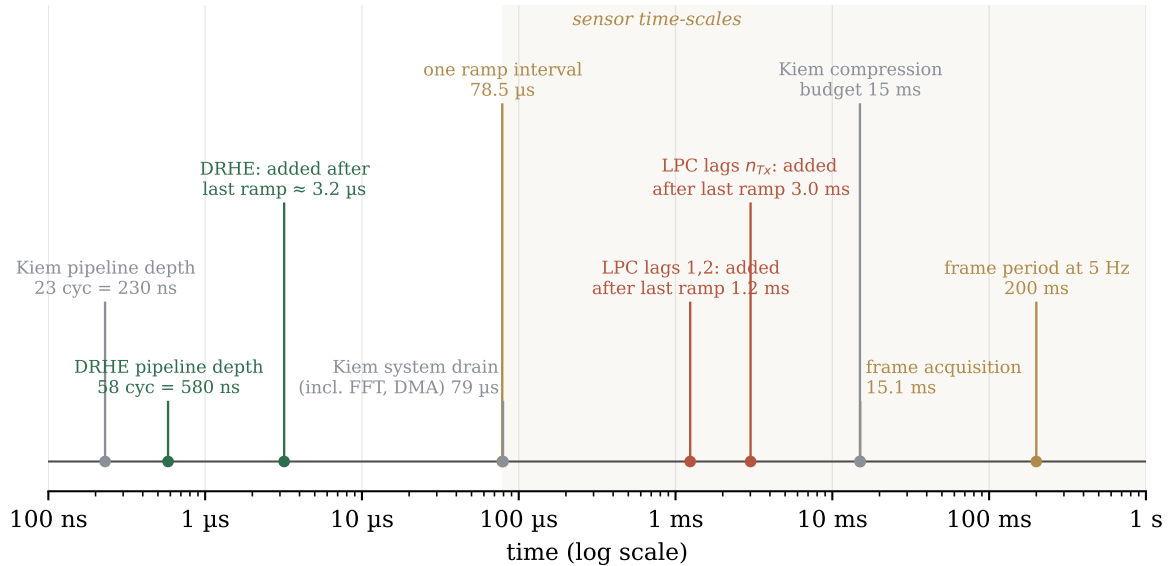


Figure 7.4: Every latency in this chapter on one logarithmic time axis. The compressor’s own delays (left) are two to five orders of magnitude shorter than the sensor’s time-scales (shaded). The only compressor delays that approach the sensor’s scale are LPC’s, because LPC must see the whole frame before it can emit anything.

7.6.1 Pipeline latency: how long one sample takes to go through

What it is. Pipeline latency is the number of clock cycles between a sample entering the compressor and its compressed bits leaving. For DRHE it is the pipeline depth, 58 cycles, or 580 ns at 100 MHz. Many samples are in flight at once, so this is a delay, not a rate.

Why measure it. It fixes two things: how much data is “inside” the compressor at any moment, and so how deep any buffering around it must be; and the smallest possible delay the compressor can add to anything.

Against Kiem: valid. Kiem reports 230 ns, which is 23 cycles. This is his HLS estimate, the same kind of number as ours. He also measured 31 cycles with an integrated logic analyser, the extra 8 cycles being the time to fill the first 256-bit output word. Estimate against estimate, this design is 35 cycles (350 ns) deeper, and Section 7.5.3 accounted for them: 26 cycles are the closed loop, which guarantees the encoder and decoder never disagree, and the rest is the latency of floating-point operators.

When does it become a problem? In this application it does not. 580 ns is 0.7 % of one ramp interval. It would matter only in a control loop that must react within about a microsecond, and no radar perception chain works on that time-scale. The difference from Kiem is real, but it has no practical consequence here.

7.6.2 Frame latency: how long a whole frame takes, if it all arrives at once

What it is. Frame latency is the co-simulation figure: 61,253 cycles for DRHE, 325,166 for the corrected LPC. As Section 7.2 explained, it is measured with the frame fed back-to-back. It is really a *throughput* measurement expressed in cycles: the time to process one frame's worth of work at full speed.

What it is not. It is not a delay the system ever experiences. The real sensor delivers a frame over 15.07 ms. At 100 MHz DRHE needs 612.5 μ s of work to process it, so the compressor is busy 4.1 % of the time during a frame. The rest of the time it waits for the next ramp.

Against Kiem: no equivalent number. Kiem does not report a per-frame processing time. His 0.08 ms is a different quantity, discussed next.

7.6.3 Added latency: how much later the answer arrives because of compression

This is the latency that matters to the system, and the right answer to the question “what does the latency represent?”

What it is. Without compression, the downstream processor could start work on a frame the moment its last ramp had been through the range FFT. With compression, it must wait until the compressor has emitted the last bit of that frame. The difference is the *added latency*.

- **DRHE** is a streaming design. It processes each ramp as that ramp arrives, so when the last ramp arrives only that one ramp remains: 319 cycles, or 3.2 μ s at 100 MHz.
- **LPC** cannot emit anything until it has seen the whole frame, because its coefficients are computed from every ramp. When the last ramp arrives, it still has all of its solve and emit passes to run. That is 1.24 ms for the baseline and 3.01 ms for the corrected design, at 100 MHz.

This is where the two algorithms differ most. DRHE adds microseconds and LPC adds milliseconds, a factor of about a thousand.

Kiem's 0.08 ms: what it measures. Kiem's system figure, 79,410 ns, is measured from the DMA interrupt that marks the last raw ramp being *read* from memory to the DMA interrupt that marks the last compressed word being *written* back. That span includes the range FFT of the last ramp (1,787 cycles, 17.9 μ s), DMA transfers and interrupt handling as well as the compressor. It is therefore an *added-latency* measurement for his whole processing chain, not a compression time and not a per-frame time. This project has no FFT, DMA or processor in its test system, so it has no measurement of the same span. The compressor's share of such a span would be the 3.2 μ s above. The comparison with 0.08 ms should therefore not be made directly.

7.6.4 When does latency become a problem?

There are three thresholds, from hardest to softest.

1. The throughput threshold: failure. If the compressor needs longer to process one ramp than the sensor takes to produce the next one, a backlog builds up, and no finite buffer can hold it for ever. Data is lost. This is the only threshold at which the design stops working, so it is the first to check. For this sensor the limit is 7,850 cycles per ramp at 100 MHz. DRHE uses 319, so it would still keep up at a clock about 25 times slower. LPC's ingest pass runs at

one beat per cycle, so it also keeps up easily. If frames could be buffered whole, the equivalent limit would be the frame period, 200 ms, which both designs meet by a very wide margin.

2. The budget threshold: the system’s schedule. The downstream processor must receive each frame in time to finish its own work before the next frame arrives. Kiem made this explicit for his sensor. It has a 50 ms cycle and 21 ms of acquisition, which leaves 29 ms, and he allocated 15 ms of that to compression. For ColoRadar at 5 Hz the gap after acquisition is about 185 ms, and both designs use a tiny fraction of it.

The budget becomes informative when the designs are *scaled to Kiem’s sensor*, whose frame is 21 times larger (512 bins by 1024 ramps against 128 by 192):

- **DRHE** still processes one ramp at a time. At 512 bins and $\Pi = 2$ a ramp takes about 1,090 cycles, or 10.9 μs , against a ramp interval of about 20.5 μs . It keeps up, and its added latency stays in microseconds.
- **LPC**’s post-ingest passes scale with the frame. Scaling the measured cycle counts gives roughly 53 ms of added latency at 100 MHz, which is *3.5 times over* Kiem’s 15 ms budget. Its frame buffer would need 67 Mbit. That exceeds the entire block RAM of Kiem’s ZU3CG (8.0 Mbit) and of the xcku5p (17.7 Mbit, or 36 Mbit with its UltraRAM), so it would have to move to external DRAM.

So on ColoRadar, LPC’s latency is acceptable. On a longer-frame sensor like Kiem’s it is the limiting factor, and it is the strongest argument for DRHE in this thesis. The scaled figures are extrapolations from measured cycle counts, not measurements. They are accurate enough to show the budget is exceeded, because the margin is large.

A related, quieter problem is *double buffering*. If one frame follows another with no idle gap, LPC is still emitting frame k while frame $k + 1$ is arriving. It then needs a second frame buffer, doubling its largest cost. DRHE has no frame buffer to double.

3. The application threshold: reaction time. Finally, any added delay lengthens the time between something happening in front of the vehicle and the vehicle knowing about it. A physical scale helps. At 100 km h^{-1} a car travels 2.78 cm per millisecond. LPC’s 3.0 ms is 8.4 cm of travel. DRHE’s 3.2 μs is under a tenth of a millimetre. Both are small next to the frame period itself: a 5 Hz radar is already up to 200 ms, or 5.6 m, behind reality. Added latency becomes an application problem when it is a noticeable fraction of the frame period. LPC on ColoRadar is 1.5 %, so it is not a problem. LPC scaled to Kiem’s sensor would exceed his entire 50 ms cycle, so it is.

7.7 Throughput and initiation interval

What they are. Throughput is how many bits per second the compressor can accept. Initiation interval (Π) is how many cycles it needs between accepting one input word and the next. With a 128-bit input word at clock f , the *theoretical* throughput is $128f/\Pi$. The *measured* throughput is the bits in a frame divided by the co-simulated frame time, and is lower because of pipeline fill and drain.

Why measure it. Throughput tells us whether the compressor keeps up with the sensor, which is the throughput threshold of Section 7.6.4. Π is the main thing that sets throughput, and it is the thing a designer can change.

Against Kiem: a correction. Kiem’s 11.92 Gbit/s is calculated as stream width times clock: $128 \times 100 \text{ MHz}$, expressed in binary units. It is the rate his design would reach if it accepted a word every cycle without pause. It is *not* a measured figure. Earlier drafts of this

Table 7.10: Throughput in decimal Gbit/s. “Measured” is from co-simulation, including fill and drain; “theoretical” is $128f/II$. Kiem’s figure is theoretical: his 11.92 Gbit/s is 12.8 Gbit/s.

	Theoretical	Measured	bits/cycle (meas.)
DRHE, 100 MHz, $II = 2$	6.40	5.136	51.4
DRHE- n at its routed $F_{\max}$ (131.0 MHz)	8.38	6.73	51.4
DRHE with $II = 1$ (step 3), 100 MHz	12.80	8.76	87.6
Replica of Kiem, $II = 1$, 100 MHz	12.80	11.54	115.4
LPC lags 1,2, frame average	—	1.595	15.9
LPC lags n_{Tx} , frame average	—	0.967	9.7
Any design at $II = 1$, 100 MHz	12.80	—	128
Kiem [11]	12.80	not reported	—
<i>Required:</i> ColoRadar, 4 RX	0.209		
<i>Required:</i> ColoRadar, all 16 RX	0.835		
<i>Required:</i> Kiem’s sensor	3.20		

work compared it against our *measured* 5.136 Gbit/s and reported a gap of 2.5 times. That mixed a theoretical number with a measured one. Theoretical against theoretical, the gap is exactly 2, which is $II = 2$ against $II = 1$. The remaining 20 % is per-ramp pipeline drain. It appears in our measured number, and it would not appear in Kiem’s theoretical one even if his design had it too.

Against the sensor: the comparison that matters. Figure 7.5 shows capacity against requirement. DRHE accepts data 24.6 times faster than four ColoRadar channels produce it at 100 MHz, and 32 times at its routed $F_{\max}$. Throughput beyond the requirement does not improve compression. It is spare capacity, and it can be spent in three ways:

- **Time-multiplexing.** One DRHE instance could serve all 16 receive channels with 6.2 times headroom at 100 MHz, instead of four instances. It would need four times the state memory, about 320 BRAM_18K, but only one set of logic.
- **A slower clock.** Dynamic power scales roughly with clock frequency. A design with 24 times headroom does not need to run at 100 MHz.
- **Larger sensors.** On Kiem’s sensor, which needs 3.2 Gbit/s, DRHE’s measured 5.1 Gbit/s still has 1.6 times headroom.

Against the architecture’s own ideal. The 128-bit input could in principle carry 128 bits per cycle. DRHE achieves 51.4, which is 40 % of the input width and 80 % of what $II = 2$ allows. This is the fairest measure of implementation efficiency, because it needs nothing external. It also points at the two fixes: the II , which Section 7.5.4 traced to one write and removed, and the per-ramp drain.

When does it become a problem? When headroom falls below 1. For a single 4-channel ColoRadar stream that would take a design about 25 times slower than this one. The practical risk is elsewhere. At $II = 2$, the compressor accepts data at half the rate the FFT core can deliver it within a ramp, so the FFT must be allowed to pause (AXI back-pressure), or there must be a small FIFO between them. One ramp for four channels is 16 kbit, about one BRAM_18K. At $II = 1$ the question would not arise.

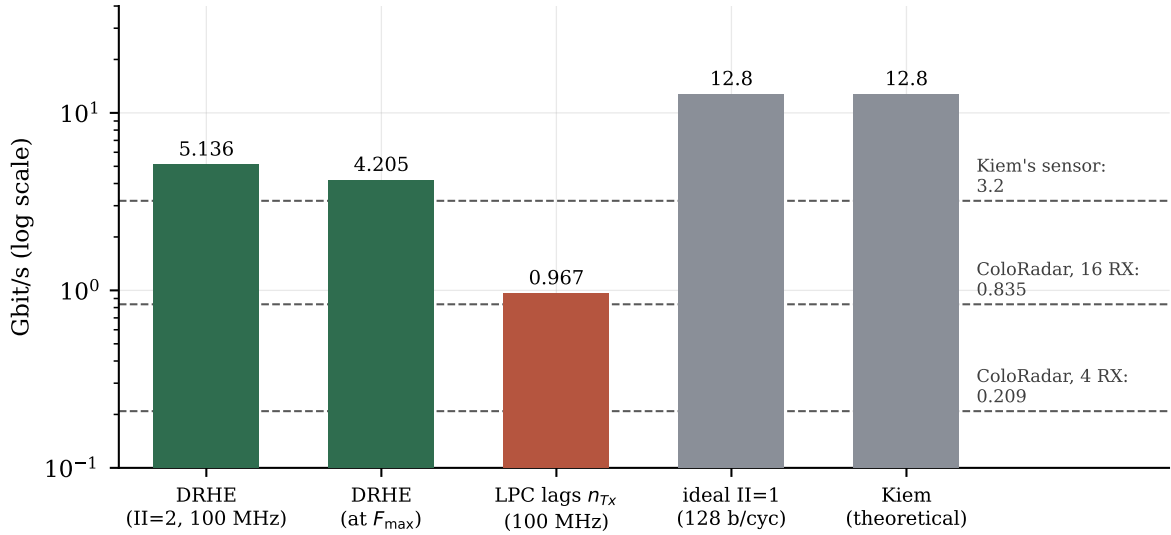


Figure 7.5: Throughput capacity (bars) against three requirements (dashed). Capacity above a dashed line is headroom. DRHE clears every requirement, including Kiem’s much faster sensor. The corrected LPC, measured over a whole frame, clears ColoRadar’s full 16-channel rate only narrowly, which matters only if frames arrive back-to-back.

7.8 Resources against the device

Why measure resources at all? Every resource number answers three questions. *Does it fit* on the chip that would actually be used, next to everything else that chip must do? *What does it cost* compared with another design that does the same job? And *how does it scale* when the sensor gets bigger? Section 7.5 answered the second. This section answers the first and third.

Table 7.11: Post-route resources of each design as a fraction of the `xcku5p` used here and of Kiem’s `XCZU3CG` (70,560 LUT, 141,120 FF, 432 BRAM, 18K, 360 DSP). Kiem’s compression IP is shown for reference.

Design	% of <code>xcku5p</code>				% of <code>XCZU3CG</code>			
	LUT	FF	BRAM	DSP	LUT	FF	BRAM	DSP
DRHE-1	16	5	4	7	50	17	9	37
DRHE- n	16	5	8	7	49	17	19	37
LPC-1,2	14	3	25	9	42	10	56	47
LPC- n , $2n$	16	5	22	9	49	14	50	44
Replica of Kiem	8	1	2	3	24	4	5	13
Kiem’s IP [11]	21	2	2	2	65	7	5	12

It fits Kiem’s device. Every design fits the `XCZU3CG`, and DRHE uses fewer of its LUTs than Kiem’s own IP does, though more of its flip-flops, DSPs and block RAM. An earlier draft concluded, from HLS estimates, that DRHE needed 164 % of that device’s LUTs and could not be built on it at all; implemented, it needs half. Kiem’s complete system (FFT, DMA, processor interface and compressor) used 59,353 LUTs, so everything except his compressor used 13,461. DRHE- n in place of his compressor would bring the system to about 48,265 LUTs, or 68 % of the device, inside the usual 70–80 % guideline above which placement and routing become congested.

Scaling to the whole sensor. The designs compress four receive channels; the ColoRadar cascade has sixteen. Two ways of covering all sixteen follow from the numbers:

- **Four DRHE- n instances**, about 139,216 LUTs and 160 BRAM tiles. That fits the `xcku5p` (64 % of its LUTs) but not the `XCZU3CG`.
- **One DRHE- n instance, time-multiplexed.** Its throughput headroom of 6.2 at 100 MHz over the 16-channel rate makes this possible. Logic stays at one instance; the state memory grows four-fold to 160 tiles, which fits the `xcku5p` (33 %) comfortably and the `XCZU3CG` only tightly (74 % of its block RAM, before the rest of the system is placed).

LPC’s memory is its limit: its frame buffer alone is 3.1 Mibit per four channels, so all sixteen channels would need about 430 tiles: 90 % of the `xcku5p` and twice the `XCZU3CG`. At Kiem’s frame size it could not be held on chip at all (Section 7.6.4).

7.9 Power

What it is. The routed design’s power report splits consumption into *device static* power, which the chip draws whenever it is switched on, whatever it is doing, and *dynamic* power, which the design draws by switching its logic, registers, DSPs, memories and wires every clock cycle. Only the dynamic part belongs to the design.

Why measure it. A compressor exists to save link bandwidth, and in a vehicle bandwidth and power are both paid for. A compressor that costs more power than the bits it saves is not worth fitting.

Table 7.12: Power from the routed designs’ reports at 100 MHz, with Vivado’s default switching activity. Static power is the `xcku5p`’s own and is the same for every design. Energy per frame is dynamic power multiplied by the co-simulated time to compress one frame; energy per bit divides that by the frame’s 3,145,728 input bits.

Design	Total (W)	Static (W)	Dynamic (W)	μJ per frame	pJ per bit
DRHE-1	1.017	0.455	0.562	344	109
DRHE- n	1.113	0.456	0.657	402	128
LPC-1,2	1.165	0.457	0.709	1399	445
LPC- $n,2n$	1.011	0.455	0.555	1805	574
Replica of Kiem	0.738	0.453	0.286	78	25

Against Kiem: nothing to compare. Kiem does not report power. The replica is therefore the only reference. DRHE-1 draws about twice the replica’s dynamic power, which is what its resource counts predict, since dynamic power scales with the logic, registers and DSPs that switch; and because the replica finishes a frame in 27,270 cycles against DRHE-1’s 61,253, DRHE-1 spends about 4.4 times as much energy per frame. LPC- n is the most expensive per frame by a wide margin, because its dynamic power is similar to DRHE’s but it takes five times as long.

Against the job. At the sensor’s actual rate the compressor is busy only a few per cent of the time. Energy per frame is therefore the useful figure: at 5 frames/s, DRHE-1’s 344 μJ per four-channel frame is about 1.7 mW of average switching power (the clock network keeps toggling between ramps unless it is gated, and is about a ninth of the dynamic figure), negligible next to the 0.46 W the device draws simply by being switched on. That static power

is a property of the development board’s large FPGA, not of the compressor; in a production ASIC or a small device it would be far lower.

How far to trust it. These are estimates from routed netlists with an assumed 12.5 % toggle rate on every signal, not measurements on a board and not driven by recorded switching activity. Vivado rates their confidence as “medium”. They are good for ranking designs against each other and for the order of magnitude; a board measurement would be needed for an absolute figure.

7.10 Compression ratio: what counts as good

What it is. Compression ratio (CR) is bits in divided by bits out, measured on real data. It is the reason the compressor exists.

Against the link. The link sets the requirement. Kiem’s sensor produces 2.4 Gbit/s of raw ADC data. To fit a Gigabit Ethernet link (1000BASE-T1) it needs a CR of 2.4, or 2.7 once protocol headers are included. The corrected DRHE’s 3.75 would clear that. For ColoRadar the picture is different, and it should be stated plainly: its post-FFT rate for all 16 channels is 0.835 Gbit/s while a frame is being acquired. That already fits under 1 Gbit/s, if only just, before compression. After compression it is 0.223 Gbit/s. On this sensor, compression buys *margin* and room for more channels, not the ability to use Gigabit Ethernet at all.

Against Kiem: not valid. Kiem’s hardware CR was 3.17, on his own data. The number looks comparable, but it is not. Section 6.4 showed that 1.63 times of this thesis’s ratio comes from unused headroom in the 16-bit container. That is a property of how ColoRadar’s data is scaled, not of the algorithm. A CR measured on a different sensor with different scaling measures a different thing, so a higher or lower number on another dataset says nothing about which implementation is better.

The valid yardsticks. These are the internal ones:

- **No prediction.** The same entropy coder applied to the raw values gives 3.489. This is the test of whether prediction helps at all. The original designs failed it; the corrected designs beat it by 7.5 % (DRHE) and 6.0 % (LPC).
- **The entropy bound.** The order-0 entropy is the least any coder of this type could spend per value. The fixed Huffman code runs 6.2 % above it (Section 6.4), so little is lost to the coder itself.
- **Each other.** DRHE and LPC run on the same data through the same coder, so their ratios are directly comparable: 3.751 against 3.697.

When does it become a problem? When the compressed rate, plus protocol overhead, no longer fits the link. The caution from Section 6.4 matters here. A sensor whose ADC output filled the 16-bit container would lose the 1.63 times headroom factor. A rough estimate, multiplying the sparsity factor (2.14) by the corrected prediction gain (1.08), puts its ratio near 2.3, which is *below* the 2.4 Kiem’s link requires. That is an estimate from factors measured on ColoRadar, not a measurement on such a sensor. It shows that a CR of 3.75 on this dataset is not a guaranteed pass on every sensor.

7.11 Every difference from Kiem, accounted for

Table 7.13 closes the chapter’s account. It lists every number in which our design and Kiem’s differ, says whether the comparison is valid as published, names the cause of the difference and points to the measurement that shows it. Where the cause is one design decision, the evidence is the step of the ladder that changes only that decision. Where it is not, the table says so.

The story in one paragraph. The published comparison said our design was two and a half times larger than Kiem’s, slower, and too big for his device. None of that survives. The size and clock figures came from HLS estimates set against his post-route figures; implemented, DRHE-1 uses fewer LUTs than his IP, meets the same clock with a quarter of the period to spare, and fits his device at half its logic. Against a replica of his design built in the same tool, DRHE-1 is larger, and by exactly the amounts its four distinctive decisions cost: floating-point arithmetic (most of the logic and registers, every extra DSP, twice the block RAM), a less economical packer (about 4,200 LUTs), a closed loop (26 cycles of depth in floating point, almost nothing in fixed point), and one extra state write (II 2, which halves throughput but also halves the arithmetic). The closed loop buys a guarantee that the encoder and decoder can never diverge, and II 2 buys a smaller design the sensor does not notice; the float datapath buys only 0.07 % of compression ratio. What remains unexplained is the replica’s distance from Kiem’s own LUT count, and Table 7.9 shows how much of it the testable explanations cover.

Table 7.13: Every difference between DRHE-1 and Kiem’s published design, with its cause and the evidence for it. “Valid” says whether the two published numbers measure the same thing. Replica figures are post-route on the `xcku5p`; the replica is the fair yardstick for what our design decisions cost.

Metric	DRHE-1	Replica	Kiem	Valid	Cause of the difference	Evidence
LUT	35,343	17,141	45,892	yes	DRHE-1 against the replica: floating-point operators (69 % of DRHE-1’s LUTs), then the packer. The replica against Kiem: about 5 % explained (Table 7.9).	ladder steps 1, 2, 4; Table 7.9
FF	23,546	5,050	9,243	yes	Registers inside float operators (54 %) and along a 58-stage pipeline. Kiem’s extra over the replica: about 45 % explained (speed grade, packer, depth), the rest not.	ladder steps 4, 5; Table 7.9
DSP	132	48	44	yes	Every DSP belongs to a float operator; each float multiply needs 3, each sine/cosine 13. Sharing at $II = 2$ halves the count.	Table 7.7; step 3 (264 at $II = 1$)
BRAM (36 Kb)	20	10	10	yes, in tiles	32-bit state words need a full tile per array; 16-bit words need half.	Table 7.6; replica matches exactly
II	2	1	1	yes	One extra write per cycle to clear the state; three accesses on two ports.	HLS 200-885; step 3 reaches II 1
Depth (cycles)	58	12	23	yes, both HLS	Closed loop: polar conversion waits for the prediction (26 cycles); the rest is float operator latency.	Figure 7.3; open-loop float schedule (depth 32); steps 4, 5
$F_{\max}$ (MHz)	135.5	129.5	≥ 100	partly	Both meet 100 MHz; Kiem reports only that he met it. His part is speed grade -1 .	routed timing; -1 runs
Throughput (Gbit/s)	6.40 th., 5.14 meas.	12.80 th., 11.54 meas.	12.80 th.	theoretical only	II 2 against II 1. Kiem’s 11.92 is binary units and theoretical.	Eq. 5.1 of [11]; co-simulation
Latency	580 ns pipeline	—	230 ns pipeline; 0.08 ms system	pipeline only	Pipeline: the depth above. His 0.08 ms spans FFT, DMA and interrupts.	Section 7.6
Power	0.562 W dyn.	0.286 W dyn.	not reported	—	Scales with switching resources; float design $\approx 2\times$ replica.	Table 7.12
Compression ratio	3.310 (50 frames; 3.321 on the first 10)	3.319 (first 10 frames)	3.17	no	Different sensor and data scaling; the float and fixed designs differ by 0.07 % on the same data.	Section 7.10
Device fraction	50 % of ZU3CG LUTs	24 %	65 %	yes	Follows from the LUT row.	Table 7.11

Chapter 8

Discussion

8.1 Comparison with the baseline design

Chapter 7 accounted for every difference between this design and Kiem’s [11] individually. This section draws the threads together, because the overall picture changed twice during the project and the reasons are worth recording.

8.1.1 Three corrections to the comparison

Three errors of comparison were made and corrected along the way. Each made this design look worse than it is.

Estimates against implemented figures. Every early comparison set HLS estimates against Kiem’s post-route figures. HLS over-estimated DRHE’s logic by a factor of 3.3 and put its clock at 81.88 MHz; the implemented design uses 35,343 LUTs, fewer than Kiem’s 45,892, and closes at 135.5 MHz. The “2.5 times Kiem’s LUTs” and “does not fit his device” conclusions of earlier drafts both came from the estimate.

Block RAM units. Kiem reports 10 block RAMs, counted in 36 Kb tiles. In the 18 Kb units HLS uses, that is 20, so DRHE-1’s 40 BRAM_18K is twice his figure, not four times.

Throughput units, and what kind of throughput. Kiem reports 11.92 Gbit/s at 100 MHz. Taken as decimal gigabits that is $11.92 \times 10^9 / 10^8 = 119.2$ bits per cycle on a 128-bit input, an awkward number with no architectural meaning. Taken as gibibits it is

$$\frac{11.92 \times 2^{30}}{100 \times 10^6} = 127.99 \text{ bits per cycle}, \quad (8.1)$$

the full width of the input stream at $II = 1$. His Equation 5.1 confirms it is calculated this way, as stream width times clock, so it is a *theoretical* rate in binary units, 12.80 Gbit/s decimal. It should be compared with this design’s theoretical rate (6.40 Gbit/s at $II = 2$), not with its measured one (5.14 Gbit/s).

8.1.2 What the comparison finally says

Built by the same tool for the same device, our DRHE-1 and a faithful replica of Kiem’s design differ as follows, and every factor has one identified cause:

- **DSPs, 2.75×:** floating-point multipliers, adders and sine/cosine units need several DSPs each where fixed point needs one, partly offset by sharing each unit between two channels at $II = 2$.

- **Block RAM, 2×:** 32-bit state words instead of 16-bit.
- **Flip-flops, 4.66×:** the registers inside a 58-stage floating-point pipeline.
- **LUTs, 2.06×:** floating-point operators, above all the arctangent, and to a lesser extent the packer.
- **Pipeline depth, 58 against 12 cycles:** 26 cycles of closed loop and the latency of floating-point operators.
- **Initiation interval, 2 against 1:** one extra write per cycle to clear the state.

Against Kiem’s *published* numbers the picture is different in one respect: his implementation uses 2.68 times the LUTs and 1.83 times the flip-flops the replica needs. Building his speed grade, the most literal reading of his packer and his pipeline depth into the replica accounts for about half of the flip-flop difference and almost none of the LUT difference (Section 7.5.5); the rest is not a property of the design he describes, and most likely of the tool version or of logic his thesis does not mention. That is why our float design, twice the replica’s size, still uses fewer LUTs than the IP Kiem built.

The conclusion for anyone continuing this work is that the float datapath is the decision that costs most, and that it is not needed. Changing only DRHE-1’s arithmetic to Kiem’s fixed point removes 42 % of its LUTs, 61 % of its flip-flops, 67 % of its DSPs and half its block RAM, and the fixed-point designs compress ColoRadar to within 0.07 % of the float design’s ratio (Section 7.10). With Kiem’s packer and state clearing as well, the complete replica needs half DRHE-1’s LUTs, a third of its DSPs and a fifth of its flip-flops.

8.2 What the compression ratio means elsewhere

Section 6.4 showed that 1.63 of the 3.489 no-prediction ratio is unused container headroom. This has a consequence that should be stated plainly: **these ratios do not transfer to a differently-scaled sensor.**

A capture whose ADC output filled the `int16` container would have no structurally-zero top bits for the entropy coder to remove. Its compression ratio, on identical algorithms, would be closer to 2.1 — the sparsity factor alone — before prediction. That is still a useful ratio, and still enough to make the Gigabit Ethernet substitution of Section 1.2 work. But it is not 3.5.

Conversely, and more usefully, the headroom represents an opportunity rather than only a caveat. If the FX16 scale factor were made adaptive per frame — computed from the actual peak rather than from a theoretical full-scale sinusoid — the same 16 bits would carry roughly 6 more bits of genuine dynamic range. That directly addresses the quantisation-error problem that motivated this thesis’s first objective, since weak returns currently being crushed into the rounding noise of Equation (2.11) would survive. The reported compression ratio would fall, but the data being compressed would be better.

8.3 Does the correction generalise?

The prediction-axis result is specific in its details and general in its principle.

It is *specific* in that $n_{Tx} = 12$, and in that the fix assumes a fixed TDM rotation. A radar that changes its MIMO schedule between frames would need the lag re-loaded; one using Doppler-division or code-division multiplexing instead of TDM has no interleaved ramp axis and does not have this problem at all — though it has others.

It is *general* in that the underlying mistake is a category error that any slow-time predictive coder can make: treating an index that is *physically* a time axis as though it were *semantically* a time axis. The ramp index genuinely is time-ordered. It is simply not the case that adjacent entries are comparable, because the thing being multiplexed onto that axis changes between

them. Any compression scheme inherited from a single-transmitter context and applied to a multiplexed one is exposed to the same error, and the check — measure the entropy coder alone, and see whether the predictor beats it — costs almost nothing.

8.4 Threats to validity

These bound how far the results generalise and are stated explicitly rather than left implicit.

- **The compression ratios are capture-specific.** As above: most of the no-prediction ratio comes from an under-filled container, which is a property of this recording’s gain staging.
- **The detection metrics do not measure compression.** All three lossless schemes share identical detection metrics with FX16 by construction. What the metrics measure is the cost of the FX16 quantisation step.
- **FP% is a false discovery rate.** It normalises by test detections, not by true negatives. It is a legitimate metric but not the one its conventional name denotes, and it is reported under its correct name here.
- **Detection matching has no positional tolerance.** A target displaced by one range-Doppler cell is scored as both a miss and a false discovery, which inflates both rates relative to a matched-detection criterion.
- **The detection threshold is an unswept free parameter.** The mean of the lowest 75 % of magnitudes plus 15 dB is not varied, and the metrics are sensitive to it.
- **Channel count differs between phases.** The MATLAB benchmark of Table 6.1 uses all 16 receive channels; the hardware path uses 4, the AXI4-Stream width. Compression ratios between the two are therefore compared through a like-for-like MATLAB helper rather than directly.
- **Hardware numbers are post-route, not silicon.** Resource counts and $F_{\max}$ come from out-of-context Vivado implementation; latency and throughput from RTL co-simulation. Phase 4 remains outstanding. An in-system build shares the device with other logic and can route less favourably; the margins reported in Chapter 7 (at least 0.595 ns of slack for the four main designs) are the allowance for that.
- **Power is a vectorless estimate.** Vivado’s power report assumes a default switching activity rather than the activity of real radar data. It is used to compare designs with one another, not as an absolute figure.
- **The reference design is rebuilt from its description, not its source.** Kiem’s code was not available. The replica follows every type, function and structure his thesis states, and matches his block RAM exactly and his DSP count to within four, but the parts he does not describe (the packer’s exact form, the control logic) are our reading of his text. The chapter reports the residual gap in LUTs and flip-flops rather than attributing it to anything it cannot measure.
- **Tool version and device differ from Kiem’s.** He used Vitis 2022.2 on an XCZU3CG; this work uses 2026.1 on an xcku5p. The two devices share the same UltraScale+ fabric (six-input LUTs, the same flip-flops, DSP48E2 slices and 36 Kb block RAM), so a netlist costs the same primitives on either, and the speed grade is tested separately. The tool version could not be varied: only 2026.1 was installed.
- **The ablation variants were checked on fewer frames.** The control variant was C-simulated on all 50 frames, the fixed-point replica and its closed-loop sibling on 10, and the other variants on 5; each was lossless on every frame it saw, and step 3 was also lossless in RTL co-simulation. The variants are used for their resource figures, which do not depend on the data.

- **Co-simulation covers one frame.** The 50-frame result is from C-simulation, which exercises identical logic but not identical timing. Nothing in either design is frame-count dependent, but this is an assumption rather than a measurement.
- **Two source citations required correction.** The BAQ baseline is properly attributed to Kwok and Johnson [22] — IEEE TGRS vol. 27, no. 4, pp. 375–383, 1989, on *synthetic* aperture radar. An earlier version of this work carried an author name, title and page range that do not correspond to any such paper. The ColoRadar citation is likewise vol. 41, no. 4, pp. 351–360 (2022) [12], not vol. 40 (2021). Both are corrected here.
- **The literature description of DRHE does not match the implementation.** A common summary of DRHE describes differencing *raw ADC* samples across chirps and coding the deltas with Golomb–Rice. The algorithm implemented and evaluated here — following [11] — operates on the *range-FFT output*, predicts in the polar domain with two IIR filters, and codes with a fixed Huffman dictionary over magnitude categories. The two make different assumptions about where the redundancy lives, and should not be conflated.

8.5 Lessons about verification

Three methodological observations seem worth recording, since they generalise beyond this project.

Dataset testing and adversarial testing find different bugs. Fifty frames of real data — 157 million input bits — found neither defect in Section 5.4. An eight-case synthetic suite found both in minutes. The real data exercises the typical case exhaustively and the extreme case not at all: ColoRadar residuals occupy category 10 of a 16-category code, so a third of the coder was simply never executed. Conversely, the synthetic suite would never have found the prediction-axis error, because synthetic data has no transmitter structure to get wrong. Neither substitutes for the other.

A passing test suite is not a correctness argument. Everything passed. The compressors were bit-exact at three levels, the ratios matched the literature, the resources fitted the device. The prediction-axis error was invisible to every test in the plan, because no test asked whether the predictor was *helping* — only whether the system was lossless and whether the ratio was in the expected range. Adding a single measurement, the no-prediction baseline, exposed it immediately.

Cycle models are a good tool until they silently stop being one. Trip count times initiation interval predicted the baseline LPC latency to 0.08 %, which built entirely unwarranted confidence in it. Applied to a restructured design with many short loop invocations it was wrong by 32 %, because it omits pipeline drain — a term that is negligible in one topology and dominant in another. The lesson is not that the model is bad, but that a model validated on one design shape carries no guarantee on another.

Chapter 9

Conclusions and Future Work

9.1 What was done

This thesis set out to extend the radar compression framework established in [11] by validating it on real data, broadening the algorithm comparison, and implementing the complete compression and decompression pipeline. All three were done.

Two lossless compressors for radar range-FFT data — Doppler Redundancy Hybrid Encoding and a second-order linear predictive coder with Yule-Walker coefficients — were specified, modelled in MATLAB, implemented in C++ for Vitis HLS, synthesised, placed and routed for an AMD Kintex UltraScale+ device, and verified bit-exactly against a double-precision reference over 50 frames of real cascaded MIMO radar data at three levels of abstraction. Both reconstruct every one of 4,915,200 complex samples exactly in software simulation, and the frame run in RTL co-simulation exactly as well. Matching decompressors were implemented for both, which is what made that verification possible.

9.2 What was found

9.2.1 The prediction axis

The principal finding is a correction rather than a confirmation. Applied the way the literature describes — predicting each ramp from its immediate predecessor — *neither predictor was contributing anything*. Coding the quantised FFT output through the identical entropy coder with no prediction whatsoever reaches a compression ratio of 3.489, against 3.319 for DRHE and 3.397 for LPC. Both were making the data measurably harder to compress, and the order-0 entropy of the residuals confirmed it independently: 4.538 bits for DRHE residuals against 4.094 bits for the raw codes.

The cause is that a twelve-transmitter time-division MIMO cascade interleaves its transmitters along the ramp axis, so “the previous ramp” belongs to a different physical antenna eleven times out of twelve — separated in space by the array baseline rather than in time by a pulse interval. Predicting instead from the previous ramp of the *same* transmitter puts both algorithms back on the genuine slow-time axis and recovers the gain they were designed to deliver. Measured in hardware simulation over the full dataset, compression rises from 3.310 to 3.751 for DRHE and from 3.385 to 3.697 for LPC, in both cases with every sample still reconstructed exactly.

For DRHE the correction costs 40 additional block RAMs — 4% of the device — with logic and measured frame latency unchanged and the clock still well above 100 MHz. For LPC it is a genuine trade: a better ratio, 11.5% less block RAM after routing and $II = 1$ on every loop, against $1.65\times$ the frame latency.

9.2.2 What a compression ratio actually measures

The audit that exposed the above also decomposed the ratio into its genuine sources. Of the no-prediction factor of 3.489, roughly 1.63 comes from an under-filled fixed-point container — the ColoRadar capture peaks at 436 of a possible 32,767, so 6.2 of every 16 bits are structurally zero — and 2.14 from the genuine spectral sparsity of a range-FFT frame. Prediction, correctly applied, adds a further 1.08.

The practical recommendation that follows is simple and, as far as the author can establish, not current practice: **report a no-prediction baseline alongside any radar compression ratio**. Without it, a ratio conflates algorithm performance with sensor gain staging, and — as this thesis demonstrates — can conceal a predictor that is doing active harm.

9.2.3 What the hardware numbers mean

Every hardware figure was first an HLS estimate, and the estimates were wrong in ways that mattered: they over-stated logic by about three times and put every design below 100 MHz. Implemented through placement and routing, every design meets 100 MHz, and DRHE-1 uses 35,343 LUTs, fewer than the 45,892 of Kiem’s published IP.

To explain every remaining difference from that IP, it was rebuilt from its description in the same tool flow and connected to DRHE-1 by a ladder of single-change variants, each implemented and verified lossless. Against the rebuilt reference, every factor has one measured cause: block RAM $2\times$ from 32-bit state words, DSPs $2.75\times$ from floating-point operators (shared two ways at $II = 2$), the initiation interval from one extra write per cycle, and the pipeline depth from the closed loop and the latency of float operators. Most of the logic and registers is the floating-point datapath itself. The fixed-point arithmetic compresses ColoRadar to within 0.07% of the float design; the float datapath is the decision that costs most, and it buys almost nothing here.

9.2.4 Two defects, and two corrections to the record

Two implementation defects were found and fixed, neither reachable with real data and both caught by an eight-case synthetic suite: a residual value of -32768 that the entropy code cannot represent and which aliased silently onto $+32767$, resolved by saturation combined with closed-loop prediction so that encoder and decoder cannot diverge; and a 128-bit to single-precision conversion in the Yule-Walker solver that returned zero above 2^{64} , resolved by normalising all operands through a common arithmetic shift.

Separately, the reference design’s published throughput was found to be expressed in gibibits rather than gigabits: 11.92 Gibit/s is 12.80 Gbit/s decimal, which is exactly 128 bits per cycle at 100 MHz — a full-rate AXI4-Stream at $II = 1$. It is a theoretical rate, not a measured one; compared theoretical against theoretical the gap to this design is exactly two, the cost of $II = 2$. Its BRAM figure is in 36 kbit tiles, which halves the apparent block-RAM gap (Chapter 7). A detection metric conventionally labelled a false positive rate was identified as a false discovery rate, and two source citations were corrected.

9.2.5 Which algorithm

After the correction, DRHE is the better choice for the application that motivates this work. It compresses slightly better (3.751 against 3.697), it needs no frame buffer at all where LPC needs 3.0 Mibit, it emits its first bits immediately where LPC must wait a full frame, and its frame latency is a fifth of LPC’s. LPC was estimated to save 30% of the LUTs, but that saving was mostly an artefact of the estimate: implemented, the corrected designs use almost the same logic (LPC- $n, 2n$ $0.99\times$ DRHE- n ’s LUTs and $0.86\times$ its flip-flops). LPC’s real costs

are memory and a frame of latency, and neither is worth paying for a streaming sensor-edge IP.

The expectation recorded in Section 3 — that LPC would be the expensive design because it contains a division — did not survive measurement. The division runs 1,024 times per frame and contributes 0.26 % of the cycles. The expensive operations in this problem are the floating-point `atan2` and sine/cosine units, not division: in the implemented DRHE-1 the two arctangents alone use 11,961 LUTs, a third of the design.

9.3 Future work

9.3.1 Immediate

1. **Move DRHE to fixed-point arithmetic.** This is the change the ablation of Chapter 7 recommends most strongly. With the closed loop and everything else kept, Kiem’s fixed-point types remove 42 % of DRHE-1’s LUTs, 61 % of its flip-flops and 67 % of its DSPs, halve the block RAM, and shorten the pipeline from 58 to 12 cycles, for a compression-ratio cost of 0.07 %. Adopting Kiem’s packer as well should save more; in the float design it saved about 4,200 LUTs, and the saving in fixed point was not measured separately. It stays exactly lossless, because the closed loop makes the decoder follow whatever arithmetic the encoder uses. One caution is measured too: the fixed-point closed loop at 12 stages closes at only 102.8 MHz, so a production version should let HLS add a stage or two. The earlier recommendation to use a CORDIC to meet timing is withdrawn: the float design already meets it.
2. **Fix the initiation interval, if the throughput is needed.** Reading the state as zero on the first ramp, instead of writing zeros to it, removes the port conflict and reaches $\text{II} = 1$ (demonstrated in step 3 of the ladder). In floating point this doubles the arithmetic, because units can no longer be shared between channels; in fixed point it is nearly free. The sensor does not need it: the $\text{II} = 2$ design already has $24.6\times$ headroom.
3. **Amortise the LPC pipeline drain.** Merging the two per-bin loops, or processing several bins per invocation, should recover most of the 65,000 cycles of drain identified in Section 6.6.3.
4. **Complete Phase 4.** Implementation is done; what remains is the board. On silicon the compressor would be measured in its real system, and power would be measured, or at least estimated with switching activity from co-simulation rather than the vectorless default used here.

9.3.2 Longer term

1. **Make the FX16 scale factor adaptive.** This is the highest-value single change the decomposition suggests. Using 9.8 of 16 bits wastes both dynamic range and honest reporting; a per-frame adaptive scale would let weak returns survive quantisation — directly addressing the quantisation-error problem that motivated this project’s first objective — and would make the reported ratio reflect algorithm performance rather than capture gain.
2. **Extend the correction to other multiplexing schemes.** Doppler-division and code-division MIMO do not produce an interleaved ramp axis, but they distribute the slow-time structure differently, and whether a slow-time predictor helps on them is an open question that the no-prediction baseline test would answer cheaply.
3. **Revisit the entropy coder for the corrected residuals.** The fixed Huffman dictionary was designed for a different residual distribution. After the prediction-axis correction, 84.6 % of residuals fall in categories 0–2 — a much more concentrated

distribution than the dictionary assumes. Re-deriving the dictionary for this distribution, or adding a category-16 extension to close the representability hole of Section 3.1.12, would both be worth measuring.

4. **Evaluate the corrected algorithms against the neural and spectral approaches** of Section 2.8, which were out of scope here but define the upper end of achievable ratios.

9.4 Closing remark

The most useful thing this project produced was not either compressor. It was the habit of asking what a number means rather than whether it matches expectation. Every test passed, every ratio agreed with the literature, and the work appeared finished — at which point a single additional measurement, the cost of coding the data with no predictor at all, showed that the central mechanism of both algorithms had never been working. That measurement takes about twenty lines of MATLAB. It should be standard.

Bibliography

- [1] C. Waldschmidt, J. Hasch and W. Menzel, “Automotive radar — from first efforts to future systems,” *IEEE Journal of Microwaves*, vol. 1, no. 1, pp. 135–148, 2021.
- [2] S. Yao *et al.*, “Exploring radar data representations in autonomous driving: a comprehensive review,” arXiv preprint, 2023.
- [3] M. I. Skolnik, *Radar Handbook*, 3rd ed. New York, NY: McGraw-Hill, 2008.
- [4] V. Bandur *et al.*, “Making the case for centralized automotive E/E architectures,” *IEEE Transactions on Vehicular Technology*, vol. 70, no. 2, pp. 1230–1245, 2021.
- [5] Z. Liu, W. Zhang and F. Zhao, “Impact, challenges and prospect of software-defined vehicles,” *Automotive Innovation*, vol. 5, no. 2, pp. 180–194, 2022.
- [6] I. Barabás *et al.*, “Rethinking vehicle architecture through softwarization and servitization,” *IEEE Access*, vol. 12, pp. 110782–110795, 2024.
- [7] Texas Instruments, “FPD-Link III serializer/deserializer (SerDes) for sensor applications,” Application Note SNLA344, 2021.
- [8] Analog Devices, “GMSL SerDes technology for automotive applications,” White Paper, 2022.
- [9] K. Matheus and T. Königseder, *Automotive Ethernet*. Cambridge, UK: Cambridge University Press, 2021.
- [10] IEEE, “IEEE P802.3bp 1000BASE-T1 PHY task force,” 2016.
- [11] L. Kiem, “Design and implementation of data compression methods for SoC-based radar systems,” Master’s thesis, Institute of Technical Informatics, Graz University of Technology, Graz, Austria, May 2025.
- [12] A. Kramer, K. Harlow, C. Williams and C. Heckman, “ColoRadar: the direct 3D millimeter wave radar dataset,” *The International Journal of Robotics Research*, vol. 41, no. 4, pp. 351–360, 2022.
- [13] Texas Instruments, *AWR2243 Single-Chip 76- to 81-GHz FMCW Transceiver Datasheet* (Rev. D), February 2024.
- [14] K. Sayood, *Introduction to Data Compression*, 5th ed. Morgan Kaufmann, 2017.
- [15] D. A. Huffman, “A method for the construction of minimum-redundancy codes,” *Proceedings of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.
- [16] B. Parhami, *Computer Arithmetic: Algorithms and Hardware Designs*, 2nd ed. Oxford University Press, 2010.
- [17] IEEE, “IEEE standard for floating-point arithmetic,” IEEE Std 754-2019, 2019.

- [18] B. Widrow and I. Kollár, *Quantization Noise*. Cambridge University Press, 2008.
- [19] I. Kuon, R. Tessier and J. Rose, “FPGA architecture: survey and challenges,” *Foundations and Trends in Electronic Design Automation*, vol. 2, no. 2, pp. 135–253, 2008.
- [20] A. Mani and Z. Yang, “Memory compression and decompression engine for TI mmWave radar,” Technical Report, Texas Instruments, 2019.
- [21] NXP Semiconductors, “S32R RADAR signal compression,” Application Note AN5375, 2021.
- [22] R. Kwok and W. T. K. Johnson, “Block adaptive quantization of Magellan SAR data,” *IEEE Transactions on Geoscience and Remote Sensing*, vol. 27, no. 4, pp. 375–383, July 1989.
- [23] F. Cheng *et al.*, “Autoencoder-based feature compression for radar and vision sensor fusion in autonomous driving,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 8, pp. 12345–12355, 2021.
- [24] J. Park, S. Y. Chun and M. Seok, “AdaRadar: rate adaptive spectral compression for radar-based perception,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2026.
- [25] Z. Zivkovic and L. Li, “On-the-fly range-velocity radar data compression exploring Doppler spectrum redundancy,” 2015.

Appendix A

The Huffman Dictionary

Table A.1 gives the complete fixed dictionary used by both compressors. The published codes [11, Table A.1] are most-significant-bit first; the implementation here packs least-significant-bit first, so the stored codewords are bit-reversed. Both forms are listed, and all sixteen entries were verified to agree.

The reversed code set remains prefix-free, which is what allows the decoder to match candidates from the low end of its bit buffer with no ambiguity.

Table A.1: The fixed Huffman dictionary. “Cost” is the total bits to code one residual in that category, $\ell + S_4$, from Equation (3.7).

S_4	$ v $ range	ℓ	code (MSB-first)	stored (LSB-first)	cost
0	$v = 0$	4	1010	0x0005	4
1	1	3	100	0x0001	4
2	2–3	2	01	0x0002	4
3	4–7	2	00	0x0000	5
4	8–15	2	11	0x0003	6
5	16–31	5	10110	0x000D	10
6	32–63	6	101110	0x001D	12
7	64–127	7	1011110	0x003D	14
8	128–255	8	10111110	0x007D	16
9	256–511	9	101111110	0x00FD	18
10	512–1023	10	1011111110	0x01FD	20
11	1024–2047	11	10111111110	0x03FD	22
12	2048–4095	14	10111111111111	0x3FFD	26
13	4096–8191	14	10111111111110	0x1FFD	27
14	8192–16383	13	1011111111110	0x0FFD	27
15	16384–32767	12	101111111110	0x07FD	27

Note the non-monotonicity at categories 12–15: the longest codewords are assigned to categories 12 and 13 rather than to 15. This is a property of the published dictionary and is reproduced faithfully. It has no effect on this dataset, where no residual exceeds category 10.

Appendix B

Bitstream Layouts

B.1 DRHE

DRHE’s bitstream is a single continuous sequence of coded residuals, in the raster order the samples arrive:

$$\underbrace{\text{enc}(d_{\mathfrak{R}}) \text{ enc}(d_{\mathfrak{S}})}_{\text{ramp 0, bin 0, ch 0}} \underbrace{\text{enc}(d_{\mathfrak{R}}) \text{ enc}(d_{\mathfrak{S}})}_{\text{ramp 0, bin 0, ch 1}} \cdots$$

where $\text{enc}(v)$ is the Huffman codeword for $S_4(v)$ followed by $S_4(v)$ APPEND bits. There is no header: the frame dimensions arrive over the AXI4-Lite control interface. The final AXI word is zero-padded.

B.2 LPC

LPC’s bitstream has two sections:

$$\underbrace{[a_1^{\mathfrak{R}} \ a_2^{\mathfrak{R}} \ a_1^{\mathfrak{S}} \ a_2^{\mathfrak{S}}]_{\text{ch,bin}}}_{\text{coefficient table, } 4 \times 16 \text{ bits each}} \underbrace{\text{enc}(d_{\mathfrak{R}}) \text{ enc}(d_{\mathfrak{S}}) \cdots}_{\text{residuals, raster order}}$$

The coefficient table comes first because the decoder needs all of it before it can reconstruct anything. The residuals follow in the original raster order — not in the order the compressor computed them — which is what allows the decompressor to operate without a frame buffer (Section 5.3).

Appendix C

Reproducing the Results

C.1 Source files

File	Purpose
MATLAB reference	
loadColoRadarFrame.m	reads a frame, applies the TDM flattening
preprocessing.m	DC removal, bit shift, Hann window
rangeFFT.m	range FFT, positive half retained
compress_fx16.m	FX16 quantisation, Eq. (2.11)
export_frames_for_vitis.m	writes the HLS testbench stimulus
Audit scripts	
audit_decomp.m	the no-prediction baseline and entropy figures
audit_order.m	ramp-ordering demonstration
audit_predaxis.m	the six-configuration prediction-axis study
thesis_figdata.m	S_4 histograms and residual quantiles
HLS designs	
drhe_common.h	shared types, constants, Huffman table
drhe_compress.cpp	DRHE compressor (baseline)
drhe_decompress.cpp	DRHE decompressor
drhe_tdm_*.cpp/h	DRHE, corrected to lag n_{Tx}
lpc_common.h	LPC types and the Yule-Walker solver
lpc_compress.cpp	LPC compressor (baseline)
lpc_decompress.cpp	LPC decompressor
lpc_tdm_*.cpp/h	LPC, corrected to lags $n_{Tx}, 2n_{Tx}$
*_tb.cpp	50-frame testbenches
*_tb_edge.cpp	the eight-case synthetic suite
Implementation and the Kiem ablation (Chapter 7)	
impl_*.cfg	each design's configuration plus the four <code>vivado.*</code> lines
drhe_abl_common.h	ablation switches, fixed-point types, shared prediction and polar helpers
drhe_abl_compress.cpp	one compressor, every ladder step and side experiment
drhe_abl_decompress.cpp, drhe_abl_tb.cpp	matching decoder and lossless check for every variant
mkabl.m, mkabl2.m	create one work folder and configuration per variant
grab_impl.m	copies each design's routed reports into one tree
collect_impl.py, hier.py	read every report, write <code>hw_numbers.json</code> , the \HW macros and the chapter's tables
sched_parse.py	maps the HLS schedule to pipeline phases (Figure 7.3)

C.2 Build commands

C-simulation, synthesis and co-simulation respectively, for the corrected DRHE:

```
vitis-run --mode hls --csim    --config test_tdm.cfg --work_dir hls_tdm
v++ -c      --mode hls        --config test_tdm.cfg --work_dir hls_tdm
vitis-run --mode hls --cosim  --config test_tdm.cfg --work_dir hls_tdm
```

Implementation through Vivado, after synthesis, reads the same configuration with four lines added:

```
vivado.flow=impl
vivado.clock=10
vivado.report_level=2
vivado.max_timing_paths=10
```

```
vitis-run --mode hls --impl --config impl_tdm.cfg --work_dir hls_tdm
```

Its reports are written to `hls/impl/verilog/report/` (routed utilisation, hierarchical utilisation, timing and power) and summarised in `hls/impl/report/verilog/top_export.rpt`. Every hardware number in Chapter 7 is produced from these files by `collect_impl.py`; none is typed by hand.

Two practical notes. Synthesis must be run before co-simulation, since co-simulation substitutes the generated RTL for the top function. And the project path must not contain a space, which Vitis rejects.

C.3 Co-simulation stimulus

Co-simulation is run against a single-frame export, produced by setting `DRHE_MAX_FRAMES=1` before running `export_frames_for_vitis.m`. Fifty frames of RTL simulation with floating-point IP takes many hours and returns no information the single-frame run does not, since the per-frame latency is deterministic. The 50-frame file should be restored afterwards for C-simulation.